CS504 Midterm mcqs

100+ mcqs correct answer

Orange Monkey Team

Important NOTE:-

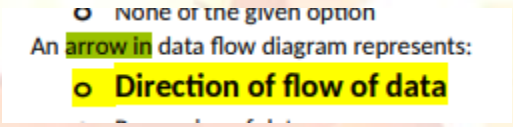
Maybe toppers are successful in school or college carrier but backbenchers are successful in their life because toppers run according to books while backbenchers don't follow rules and regulation that is why backbenchers are successful. For life, there are no books and another reason why toppers can't succeed because they fear failure and don't try anything new. Backbenchers don't fear failure as they have faced the failure in their school or college carrier so that is the reason they try new things and at last gain success.

As Abdul Kalam nicely quoted, "The best brains of the nation may be found on the last benches of the classroom"



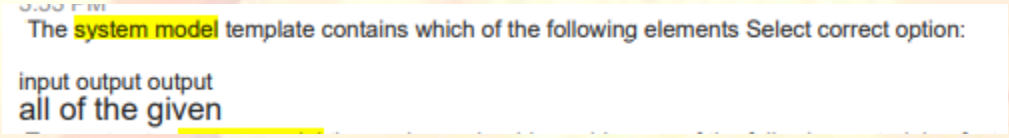
1. **An arrow in data flow diagram represents:**

   Direction of flow of data…..confirm



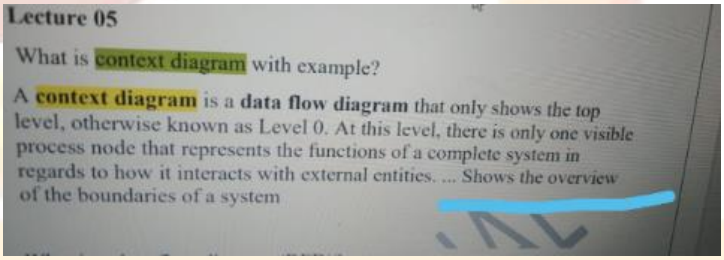
2. **The system model template contains which of the following elements.**

   Input/output



3. **A context diagram:**

   Is a DFD which gives an overviews of the system…..confirm



4. **In _____ phase of software development requirement analyst focuses on possible design of the proposed solution**

   Definition

In ------- phase of software development, requirement analyst focuses on possible design of the proposed solution.

- Maintenance
- Development
- **Definition**
- Vision

5. **System _____ are built to allow the system engineer to evaluate the system components in relationship to one another.**

Models….confirm

System models are built to allow the system engineer to evaluate the system components in relationship to one anothe

6. **_____ diagram does not capture control flow information, it just shows the flow of the data in a system**

Data flow diagram…..confirm

flow in an algorithm. Flow charts are quite detailed. Whereas DFD does not captures control flow information, it just shows the flow of the data in a system. Flow charts show

7. **In data flow diagram (DFD), Create, Update, delete and read operations are normally called.**

CRUD operation…..confirm

In data flow diagram (DFD), Create, Update, Delete and Read operations are normally called:

1. DURC operations
2. CDUR operation
3. RUDC operations
4. CRUD operations

8. **More powerful hardware resulted into the development of _ powerful and _____ software.**

More,complex…..confirm

days but were far more powerful than the computers of early fifties. More powerful hardware resulted into the development of more powerful and complex software. Those

9. **Which elements of business processing engineering are the responsibilities of the software engineer?**

Business system design…..confirm

Which elements of business processing engineering are the responsibilities of the software engineer?
Select correct option:
business area analysis
business system design ok

10. **The use case diagram shows that which ____ interact with each use case.**

Actor…..confirm

The Use case diagram shows that which _____ interact with each use case.
Select correct option

Use case

Actor

Maham Ji

11. **A prototype is not the real product but just a real looking _____ of what would be eventually delivered and might not do anything useful**

Mock-up……confirm

A prototype is not the real product. It is rather just a real looking mock-up of what would be eventually delivered and might not do anything useful. However, the presence of a

12. **A _____ is not the real product but just a real looking mock-up of what would be eventually delivered.**

Prototype……confirm

A prototype is not the real product. It is rather just a real looking mock-up of what would be eventually delivered and might not do anything useful. However, the presence of a

13. **In data flow diagram the entity or system outside the boundary of this system is called**

External Agent….confirm

External Agent

External systems which are outside the boundary of this system.

14. The project manager would need _ document to monitor and track the progress of the project and if needed.

Requirement….confirm

On the other hand, the project manager would need this document to monitor and track the progress of the project and if needed, change the project scope by modifying this

15. By leveling a DFD(adding more levels of abstraction) we mean

Splitting it into different levels……confirm

By levelling a DFD (adding more levels of abstraction) we mean:

- **Splitting it into different levels**

16. Data cannot flow from one external entity to other external entity because

It is not allowed in DFD…..confirm

Data cannot flow from one external entity to other external entity because:

1. It will get corrupted
2. It is not allowed in DFD

17. A __ is not the real product but just a real looking mock up of what would be eventually delivered.

Prototype…..confirm

A prototype is not the real product. It is rather just a real looking mock-up of what would be eventually delivered and might not do anything useful. However, the presence of a

18. A context diagram is used

As an aid to system design….confirm

A context diagram is used
Select correct option:

as the first step in developing a detailed DFD of a sy
very complex systems
as an aid to system design

19. _____ the analyst determines all the sources of requirements and where do these requirements consume.

Source and sink analysis…..confirm

In source and sink analysis the analyst determines all the sources of requirements and where do these requirements consume (sinks). Now evaluate a report which displays

20. _____ is a technique that can be used to reduce customer dissatisfaction at the requirement stage.

Prototyping ,,,,confirm

Prototyping is yet another technique that can be used to reduce customer dissatisfaction at the requirement stage. The idea is to capture user's vision of the product and get early

21. An external entity that interacts with the system is called a(n):

Actor…..confirm

22. At which stage of software development loop results are delivered?

Status quo…..confirm

- Vision

At which stage of software development loop, results are delivere

- Problem definition
- Solution integration
- Technical development
- **Status quo**

23. Requirement engineering mainly deals with the _____ of the system

Definition phase…..confirm

Requirement engineering mainly deals with the definition phase of the system.

24. A use case represents

The role a user plays when interacting with the system….confirm

A use case represents:

- A class, its attributes and operations
- An operation's interface and signature
- **The role a user plays when interacting with the system**

25. ___ requirements are often called products features.

Non-functional……confirm

------ requirements are often called product features.

- Functional
- **Non-functional**

26. All the documents related to the software are also considered as part of the _____.

Software….confirm

documentation. All the documents related to the software are also considered as part of the software.

27. Include and extend relationship is used in UML notation of a/an_____.

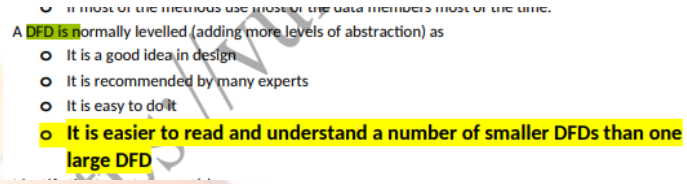
Use case model…..confirm

3.9 Relationship among Use Cases

The UML allows us to extend and reuse already defined use cases by defining the relationship among them. Use cases can be reused and extended in two different fashions: extends and uses. In the cases of *"uses"* relationship, we define that one use case invokes

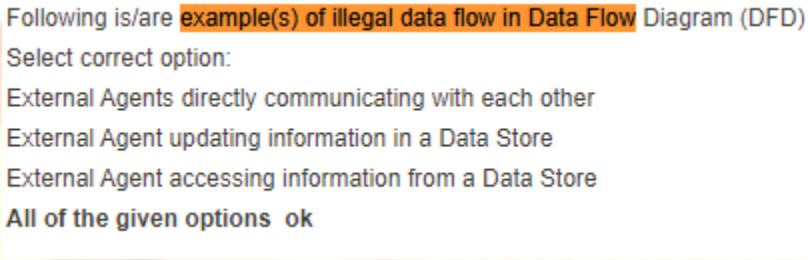
28. A DFD is normally leveled (adding more levels of abstraction) as

It is easier to read and understand a number of smaller DFDs than one large DFD…..confirm



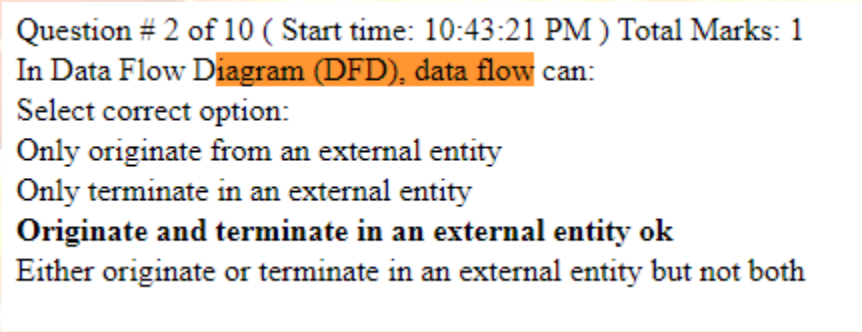
29. From the following which is/are not the example(s) of illegal data flow in data flow diagram.

There must be an intermediate process which should transform data received from one external entity and then send the transformed data to the other external entity….confirm



30. In data flow diagram (DFD) data flow can

Originate and terminate in an external entity….confirm



31. To construct a system model the engineer should consider one of the following restraining factors?

Assumptions and constraints…..confirm

Question # 6 of 10 (Start time: 10:36:57 PM) Total Marks: 1

To construct a system model the engineer should consider one of the following restraining factors?

Select correct option:

assumptions and constraints ok

32. In a top-down system analysis a/an _ is required to develop high level view of the system at first

Analyst……confirm

In a top-down system analysis, an analyst is required to develop high level view of the system at first. In data flow modeling, this high-level view is the Context level data flow

33. In data flow diagram (DFD) one data store cannot directly copy the data from another _____.

Data store…..confirm

In Data Flow Diagram (DFD), one data store can directly copy the data from another data store

34. _____ of the total cost of the software development is spent on maintenance.

Two third….confirm

_____ of the total cost of the software development is spent on maintenance.

Select correct option:

one third

two third ok

35. The system specification describes the

Function and behavior of a computer-based system…..confirm

The system specification describes the:

- **Function and behavior of a computer-based system**

36. In software engineering paradigm, any engineering approach must be founded on organizational commitment to _____.

Quality…..confirm

Any Engineering approach must be founded on organizational commitment to quality.

37. Which one is not a part of software development phase?

Project Vision….confirm

Question # 8 of 10 (Start time: 10:28:49 PM) Total Marks: 1
Which one is not a part of Software Development phase ?
Select correct option:
Construction
Scope
Project Vision ok
Definition

38. A process in Data flow diagram (DFD) represents

Transmission of data…..confirm

Question # 3 of 10 (Start time: 10:15:18 PM) Total
A Process in Data Flow Diagram (DFD) represents
Select correct option:
Flow of data
Transformation of data ok

39. In use case diagram, an ellipse signifies an

Use case…..confirm

In use case diagram, an ellipse signifies a(n)

1. actor
2. class
3. use case

40. Which of the given component of software engineering framework provides different techniques that can be used to perform a particular task.

Method…..confirm

Methods: Methods provide the technical "how-to's" to carryout these tasks. There could be more than one technique to perform a task and different techniques could be used in different situations.

41. Which of the given component of software engineering framework demands rational and timely development of a software?

Quality focus….confirm

organizational commitment to quality. The quality focus demands that processes be defined for rational and timely development of software. And quality should be

42. Arranging information in _____ form makes it easy to read, understand and comprehend as compared to streams of text.

Tabular ….confirm

Arranging information in **tabular form** makes it easy to read, understand and comprehend as compared to streams of text.

43. The data on which the program operates is also considered as part of the _____.

Software…..confirm

***Data*:** The data on which the program operates is also considered as part of the software.

44. Construction activities are directly related to software _____.

Construction or Development….both are confirm

second is management. The construction activities are those that are directly related to the construction or development of the software. While the management activities are those

45. Many of the problems encountered in Software development are attributed to shortcoming in _____.

Requirement gathering……confirm

Many of the problems encountered in SW development are attributed to shortcoming in requirement gathering and documentation process. We cannot imagine start building a house without being fully satisfied after reviewing all the requirements and developing all

46. ____ component of software engineering framework provides automated or semi-automated support in a software development

Tools…..confirm

Tools: Tools provide automated or semi-automated support for software processes, methods, and quality control.

47. Use case construction is a technique used for

Requirements structuring…..confirm

Use case construction is a technique used for:
Select correct option:

requirements determination.

requirements structuring.

48. The context diagram is used as the top level abstraction in a _ developed according to principles of structured analysis.

Dataflow diagram…..confirm

system. The context diagram is used as the top level abstraction in a dataflow diagram developed according to principles of structured analysis. The context diagram can be

49. Which one of the following is the external quality of a software product.

Low coupling…..confirm

Which one of the following is the external quality of a software product?

Select correct option

Correctness

Concision

Cohesion

Low Coupling

50. A software requirement document describes all the ___ provided by the system along with the constraints under which it must operate.

Services….confirm

same thing: a software requirement is a document that describes all the services provided by the system along with the constraints under which it must operate.

51. If you try to make software more user-friendly then the _____ may suffer.

Effiency……confirm

may be the case that if you try to make it more user-friendly then the efficiency may suffer. And if you try to make it more cost-effective then reliability may suffer. Therefore

52. According to caper jones analysis of project activities coding only has _____ affect part in software development

13-14%.....confirm

but it is not more than 13-14% of

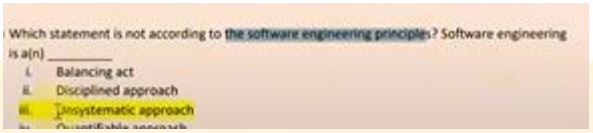
53. In _____ phase of software development, requirement engineer focuses on realizing the business object of an under developed product

development…..confirm

Requirement engineering mainly deals with the definition phase of the system. Requirement engineering is the name of the process when the system services and constraints are established. It is the starting point of the development process with the focus of activity on what and not how.

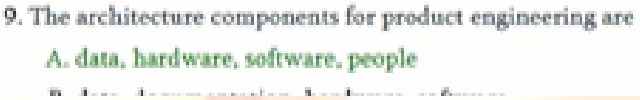
54. Which statement is not according to the software engineering principles? Software engineering is an _____.

Unsystematic approach…..confirm



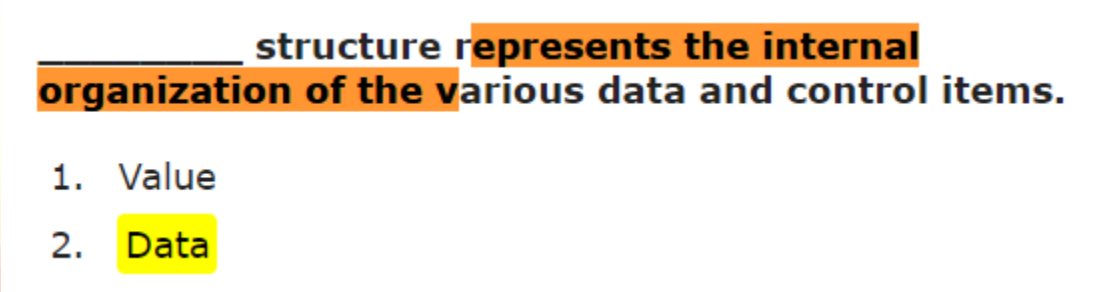
55. The architecture components for product engineering are

Data, hardware, software, people…..confirm



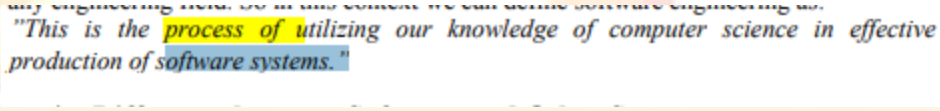
56. _____ structure represents the internal organization of the various data and control items.

Data….confirm



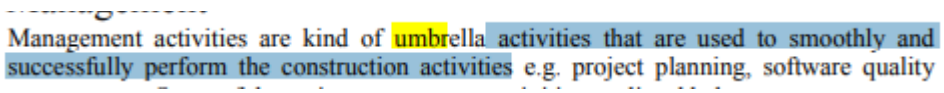
57. The process of utilizing our knowledge of computer science in effective production of software system is called.

Software Engineering…confirm



58. __ are kind of umbrella activities that are used to smoothly and successfully perform the connection activities.

Management activites…..confirm



59. Software Engineering in the combination of tools techniques and _____.

Processes…..confirm

Engineering is the combination of all the tools, techniques, and processes t

60. In use case diagram, the scope of the system is defined by

System boundary…..confirm

Use Diagram for a Library System

As an example, consider the following use case diagram for a library management system. In this diagram, there are four actors namely Book Borrower, Librarian, Browser, and Journal Borrower. In addition to these actors, there are 8 use cases. These use cases are represented by ovals and are enclosed within the system boundary, which is represented by a rectangle. It is important to note that every use case must always deliver

61. The ___ relationship is kind of a generalization specialization relationship.

Extends….confirm

makes a call to the other function. The *"extends"* relationship is kind of a generalization-specialization relationship. In this case a special instance of an already existing use case

62. Data cannot flow from one external entity to other external entity because:

It is not allowed in DFD…..confirm

Data cannot flow from one external entity to other external entity because:

1. It will get corrupted
2. It is not allowed in DFD

63. The best way to conduct a requirements validation review is to.

Use a checklist of questions to examine each requirement….confirm

The best way to conduct a requirements validation review is to

A. examine the system model for errors

B. have the customer look over the requirements

C. send them to the design team and see if they have any concerns

D. use a checklist of questions to examine each requirement

64. External entity may be

Source of input data only…..confirm

External Entity may be

Select correct option

source of input data only

65. Which of the following is used for multi level commenting?

/*Comment*/….confirm

Which of the following is used for multi level commenting?

Select correct option:

// Comment

/* Comment */ ok

66. Return values in Synchronous messages are represented by

A dotted line with label….confirm

Return values in Synchronous messages are represented by:

Select correct option:

A solid line

A dotted line with label ok

67. _____ is a role that each actor plays in the system under.

Participant…..confirm

A participant is a role that each actor plays in the system under consideration.

68. Modules with high cohesion and low coupling can be treated and analyzed as.

Black boxes……confirm

Modules with high cohesion and low coupling can be treated and analyzed as black boxes. This approach therefore allows us to analyze these boxes independent of other

69. Any Engineering approach must be founded on organizational commitment to _____.

Quality….confirm

Any Engineering approach must be founded on organizational commitment to quality.

70. Return values in synchronous messages are: (V.V.V)

May not used when response is obvious…..confirm

Return values in synchronous messages are:

1. Compulsory
2. represented by solid lines
3. Not used at all
4. May not used when response is obvious

71. According to Caper Jones analysis of project activities, coding only has _____ affect part in system development.

13-14%...confrim

that, though coding is very important but it is not more than 13-14% of the whole effor
of software development.

72. In multi-threaded or multiprocessing applications where different execution threads may pass information to one another by sending _____ to each other.

Asynchronous messages…..confirm

processing applications where different execution threads may pass information to one another by sending asynchronous messages to each other. Asynchronous messages

73. Which of the following is not among the four layers of the object-oriented pyramid?

The abstract layer……confirm

of modularity. The four layers of the OO design pyramid are:

1) **The subsystem layer.** Contains a representation of each of the subsystems that enable the software to achieve its customers defined requirements and to implement the technical infrastructure that supports customer requirements.
2) **The class and object layer.** Contains the class hierarchies that enable the system to be created using generalization and increasingly more targeted specializations. The layer also contains design representations for each object.
3) **The message layer.** Contains the details that enable each object to communicate with its collaborators. This layer establishes the external and internal interfaces for the system.
4) **The responsibility layer.** Contains the data structures and algorithmic design for all attributes and operations for each object.

74. System models include(V.V.V)

All options….confirm

diagrams to model different business processes. System models include the following

- User business processes
- User activities for conducting the business processes
- Processes that need to be automated
- Processes which are not to be automated

75. In the architecture trade-off analysis method the architectural style should be described using the.

All options…confirm

In the architecture trade-off analysis method the architectural style should be described using the

Select correct option:

data flow view

module view

process view

all of the given ok

76. _____ is concerned with decomposing the system into interacting sub-system.

System structuring…..confirm

System structuring is concerned with decomposing the system into interacting sub-systems. The system is decomposed into several principal sub-systems

77. Software development is a step-by-step process and in _____ phase of software development Business Objective of an organization gets cleared.

Vision….confirm

Vision: Here we determine why are we doing this thing and what are our business objectives that we want to achieve.

78. When two components of a system are using the same global data area, they are related as.

Content coupling….confirm

When two components of a system are using the same global data area, they are related as

Select correct option:

Data Coupling

Content Coupling ok

79. The goal of _____ is to translate the customer's desire for a set of defined capabilities into a working product.

Product engineering….confirm

The goal of _____ is to translate the customer's desire for a set of defined capabilities into a working product.

1. Electrical Engineering
2. Product Engineering

80. The condition that must be met before the use case can be invoked, is called

Precondition…..confirm

Precondition: the condition that must be met before the use case can be invoked.

81. In software engineering paradigm any engineering approach must be founded an organizational commitment to _____.

Quality ……confirm

Any Engineering approach must be founded on organizational commitment to quality.

82. There are _____ most important characteristics of an object.

Three….confirm

An object has state, behavior, and identity.

83. _____ requirements cause frequent modifications in user interface

Unstable….confirm

---------- requirements cause frequent modifications in user interface.

- Functional
- Non-functional
- Unstable

84. Prototyping is used when there is _____ regarding requirements.

Uncertainty…confirm

is used when there is uncertainty regarding requirements.

85. UML is a language for

Modeling and design…confirm

UML is a language for ____

a. high level programming

b. low level programming

c. modeling and design ✓

86. UML stands for _____.

Unified Modeling Language.

popular of these include, Rumbaugh, Booch, and Coad, and UML(Unified Modeling Language). We will be using UML to document our design. Although the notation is very

87. Requirement engineering focuses on _____ aspect of the software development process.

What….confirm

Requirement engineering mainly deals with the definition phase of the system. Requirement engineering is the name of the process when the system services and constraints are established. It is the starting point of the development process with the focus of activity on what and not how.

88. Many of the problems encountered in Software development are attributed to shortcoming in _____.

Requirement gathering and documentation process…..confrim

Many of the problems encountered in SW development are attributed to shortcoming in requirement gathering and documentation process. We cannot imagine start building a

89. In object Oriented design _____ layer contains the details that enable each object to communicate with its collaborators.

Message….confrim

The message layer. Contains the details that enable each object to communicate with its collaborators. This layer establishes the external and internal interfaces for the

90. In Object Oriented design _____ layer contains a representation of each of the subsystem that enable the software to achieve its customers defined requirements and to implement the technical infrastructure that supports customer requirements.

Subsystem….confirm

The subsystem layer. Contains a representation of each of the subsystems that enable the software to achieve its customers defined requirements and to implement the technical infrastructure that supports customer requirements.

91. In object oriented design _____ layer contains the class hierarchies that enable the system to be created using generalization and increasingly more targeted specializations.

Class and object layer…..confrim

The class and object layer. Contains the class hierarchies that enable the system to be created using generalization and increasingly more targeted specializations. The

92. In object oriented design _____ layer contains the data structures and algorithmic design for all attributes and operations for each object.

Responsibility …..confrim

The responsibility layer. Contains the data structures and algorithmic design for all attributes and operations for each object.

93. Normally a system will be more easy to modify if its modulus have:

Low coupling and high cohesion….confirm

Normally a System will be more easy to modify if its modules have:
Select correct option:
High coupling and High cohesion.
High coupling and Low cohesion.
Low coupling and High cohesion. ok

94. The best way to conduct a requirements validation review is to.

Use a checklist of question to examine each requirement….confrim

Que.	The best way to conduct a requirements validation review is to
a.	examine the system model for errors
b.	have the customer look over the requirements
c.	send them to the design team and see if they have any concerns
d.	use a checklist of questions to examine each requirement
	Answer: use a checklist of questions to examine each requirement

95. Software design discusses _____ aspect of software development.

How….confirm

Software Design discusses _____ aspect of software development
Select correct option:
What
How ok

96. Software Engineering is the combination of tools, techniques and _____.

Processes….confrim

Engineering is the combination of all the tools, techniques, and processes t

97. _____ requirements cause frequent modifications in user interface.

Unstable…..confirm

Unstable requirements cause frequent modifications in UIs

98. In abbot's Textual analysis technique, different parts of speech are identified within the text of the specification and these parts are modeled using different _____.

Components…..confrim

initially developed by Abbot and then extended by Graham and others. In this technique different parts of speech are identified within the text of the specification and these parts are modeled using different components. The following table shows this scheme.

99. Which statement is not according to the software engineering principles? Software engineering is a(n)_____.

Unsystematic approach…..confrim

Which statement is not according to the software engineering principles?
Software engineering is a(n) _____

- Balancing act
- Disciplined approach
- **Unsystematic approach**

100. OOD results in a design that achieves a number of different levels of _____.

Modularity…..confrim

software construction. OOD results in a design that achieves a number of different levels of modularity. The four layers of the OO design pyramid are:

101. The architecture components for product engineering are

Data, hardware, software, people….confrim

Que.	The architecture components for product engineering are
a.	data, hardware, software, people
b.	data, documentation, hardware, software
c.	data, hardware, software, procedures
d.	documentation, hardware, people, procedures
	Answer: data, hardware, software, people

102. GUI stands for _____.

Graphical User interface…..confirm

graphical user interface (GUI)

103. In the case of _____ intra component linkages are stronger white inter component linkages are weak.

Low coupling……confirm

connected with everything else. On the other hand, in the system with low coupling modules can be identified easily. In this case intra component linkages are stronger while inter component linkages are weak.

104. In the case of _____ module boundaries are not well defined as everything seems to be connected with everything else.

Highly coupled system……confirm

coupling. The lines depict linkages between different components. In the case of highly coupled system, module boundaries are not well defined, as everything seems to be connected with everything else. On the other hand, in the system with low coupling

105. A prototype is not the real product. It is rather just a real looking _____ of what would be eventually delivered and might not do anything useful.

Mock-up…..confirm

A prototype is not the real product. It is rather just a real looking mock-up of what would be eventually delivered and might not do anything useful. However, the presence of a

106. Coupling is a measure of _____ of a module or component.

Independence……confirm

Coupling is a measure of independence of a module or component.

107. In UML, based object Oriented model of a system the demand sign is used to depict _____ relations between two objects/classes.

Composition and Aggregation……confirm

In UML based Object Oriented Model of a system, the diamond sign is used to depict ———

- relations between two objects/classes.
- Aggregation and Association
- Inheritance and Association
- **Composition and Aggregation**

108. _____ and _____ are important short circuiting logical operators.

a.And b. OR c. Not d. NOR

a & b….confirm

The logical and operator, &&, and logical or operators, ||, are special due to the C/C++ short circuiting rule, i.e. a || b and a && b are short circuit evaluated. That is, logical

109. One of the main reasons to make functions is _____.

Reusability…..confrim

Reusability is one of the prime reasons to make functions

110. Is prefix should be used for _____ variables and methods.

Boolean…..confrim

2. is prefix should be used for boolean variables and methods.

```
isSet, isVisible, isFinished, isFound, isOpen
```

111. The terms get/set must be used where an _____ is accessed directly.

Attribute……confirm

1. The terms *get/set* must be used where an attribute is accessed directly.

```
employee.getName();
matrix.getElement (2, 4);
```

112. Code should not be.

Commented…..confirm

As Fowler puts it, comments should not be used as deodorants. Tricky code should not be commented but rewritten. In general, the use of comments should be minimized by

113. The size of _____ plays a significant role in making the program easy or difficult to understand

Individual functions……confirm

in managing and mastering the complexity of a program. We also discussed that the size of individual functions plays a significant role in making the program easy or difficult to understand. In general, as the function becomes longer in size, it becomes more difficult

114. _____ is a tool that can help us in reducing the size of individual functions.

Modularity…..confrim

to understand. Modularity is a tool that can help us in reducing the size of individual functions, making them more readable. As an example, consider the following selection

115. N-tier architecture stems from the struggle to find a _____ between the fat-client architecture and the thin-client architecture.

Middle ground……confirm

N-tier architecture stems from the struggle to find a middle ground between the fat-client architecture and the thin-client architecture. In this case the idea is to enhance

116. For (i=0, col = 0; i<27; i++, j++). In the above line of the code, 27 is representing_____.

Constant…..confrim

Consider the following code segment:

```
fac = lim / 20;
if (fac < 1)
        fac = 1;
for (i =0, col = 0; i < 27; i++, j++) {
        col += 3;
        k = 21 – (let[i] /fac);
        star = (let[i] == 0) ? ' ' : '*';
        for (j = k; j < 22; j++)
                draw(j, col, star);
}
draw(23, 1, ' ');
for (i='A'; i <= 'Z'; i++)
        cout << i;
```

Can you tell by reading the code what is meant by the numbers 20, 27, 3, 21, 22, and 23. These are constant that mean something but they do not give any indication of their importance or derivation, making the program hard to understand and modify. To a

117. In case of using unrelated operators in a single expression_____ would be the best choice to prevent the logical errors.

Parenthesize….confrim

specifying grouping. It is especially important to use parentheses when different unrelated operators are used in the same expression as the precedence rules are often assumed by the programmers, resulting in logical errors that are very difficult to spot. As an example consider the following statement:

118. In the switch statement, cases should always end with a ____ statement.

Break….confirm

In the switch statement, cases should always end with a break.

119. Floating point constants should always be written with decimal point and at least.

One decimal….confrim

2. Floating point constants should always be written with decimal point and at least one decimal.

120. In case of header files construction is to avoid __ errors. The construction should appear in the top of the file (before the file header).

Compilation……confirm

The construction is to avoid compilation errors. The construction should appear in the top of the file (before the file header) so file parsing is aborted immediately and compilation time is reduced.

121. Names representing methods and functions should be ___ and written in mixed case starting with _____ case.

Verbs, lower….confrim

4. Names representing methods and functions should be verbs and written in mixed case starting with lower case.

122. Names representing types must be nouns and written in mixed case starting with _____.

Upper case….confrim

Names representing types must be nouns and written in mixed case starting with upper case.

123. Variable names must be in mixed case starting with _____.

Lower case….confrim

Variable names must be in mixed case starting with lower case.

124. Names representing constants must be all uppercase using underscore to _____ words.

Separate….confrim

Names representing constants must be all uppercase using underscore to separate words.

125. Names representing template types in C++ should be a _____ letter.

Single uppercase…..confirm

Names representing template types in C++ should be a single uppercase letter.

126. Global variables in C++ should always be referred to by using the _____ operator.

(::) …..confrim

Global variables in C++ should always be referred to by using the :: operator.
::mainWindow.open() , ::applicationContext.getName()

127. Private class variable should have _____ suffix.

Underscore(_)…..confirm

Private class variables should have _ suffix.

128. **MVC stands for**

Model View Controller….confirm

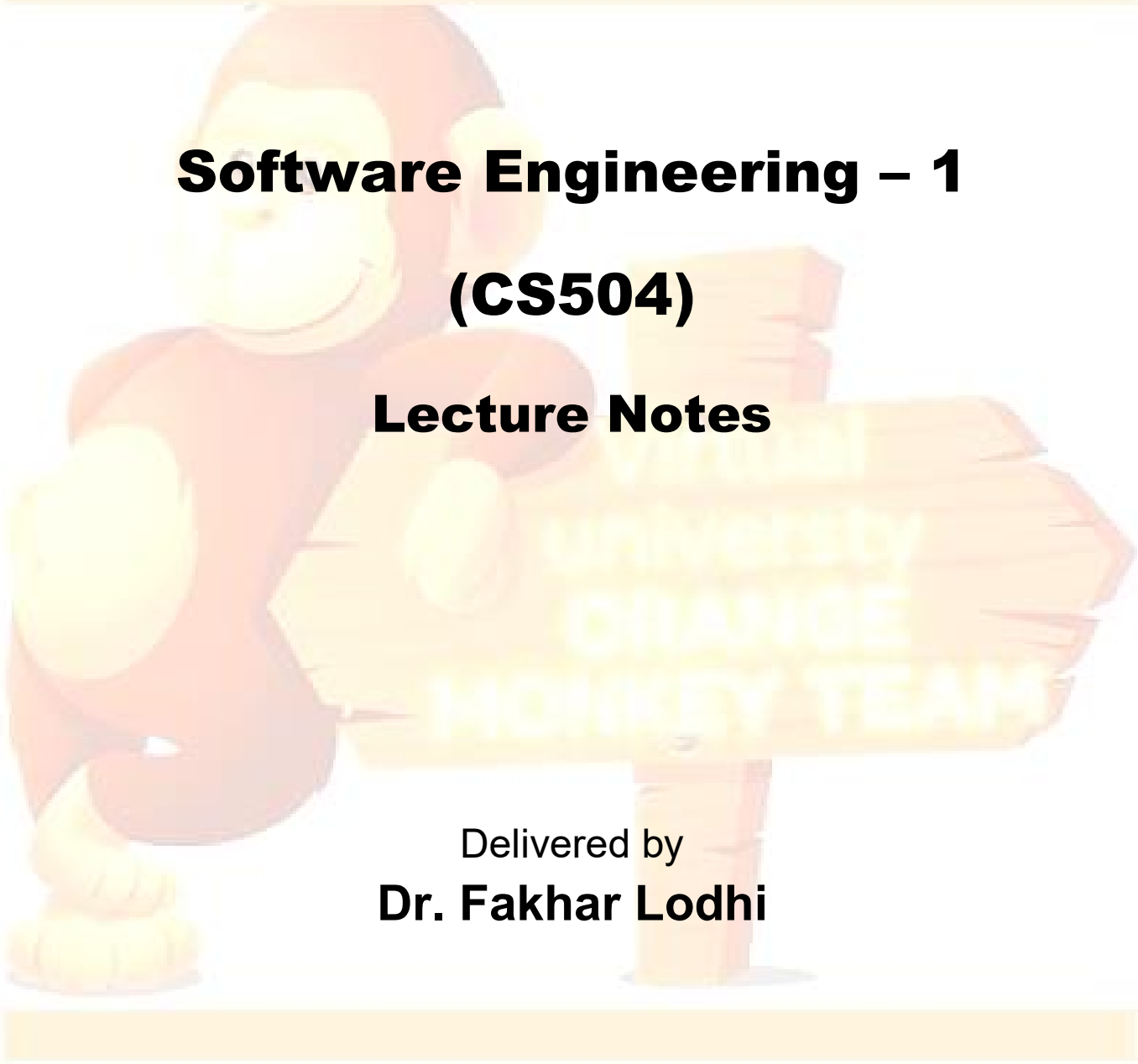
Model View Controller (MVC)



Software Engineering – 1

(CS504)

Lecture Notes



Delivered by

Dr. Fakhar Lodhi

TABLE OF CONTENTS

.....62

Lecture 11: Software Design 69

Lecture 12: Coupling and Cohesion.....72

Lecture 13: Object Oriented Analysis and Design 83

Lecture 14: Object Oriented Analysis and Design-2.....89

Lecture 15: UML Object Model Notations.....92

Lecture 16: Derivation of Object Model-Coad Methodology..... 93

Lecture 17: Derivation of Object Model-Coad Methodology -2 95

Lecture 18: CASE STUDY: Connie's Convenience Store.....97

Lecture 19: Identify Structure 100

Lecture 20: Interaction Diagrams 106

Lecture 21: Sequence Diagrams (Message Types) 108

Lecture 22: Software and System Architecture 1

© Copyright Virtual University of Pakistan

Lecture No. 11

Software Design

Introduction

Recalling our discussion of software construction process, once the requirements of a software system have been established, we proceed to design that system. During the design phase, the focus shifts from what to how. That is, at this stage we try to answer the question of how to build the system. The objective of the design process is to analyze and understand the system in detail so that features and constituent components of at least one feasible solution are identified and documented. The design activity provides a roadmap to progressively transform the requirements through a number on stages into the final product by describing the structure of the system to be implemented.

It includes modeling of the data structures and entities, the physical and logical partitioning of the system into components, and the interfaces between different components of the system as well as interfaces to the outside world. Sometimes design of algorithms is also included in this activity.

Managing Complexity of a Software System

A complex system that works is invariably found to have evolved from a simple system that worked. The structure of a system also plays a very important role. It is likely that we understand only those systems that have hierarchical structure and where intra-component linkages are generally stronger than inter component linkages. To manage the complexity of the system we need to apply the principles of separation of concern, modularity, and abstraction. This leads to designs that are easy to understand and hence easy to maintain.

Separation of concern, modularity, and abstraction are different but related principles.

Separation of concern allows us to deal with different individual aspects of a problem by considering these aspects in isolation and independent of each other.

A complex system may be divided into smaller pieces of lesser complexity called modules. This is the classic divide-and-conquer philosophy – if you cannot solve a complex problem, try to break it into smaller problems that you can solve separately and then integrate them together in a systematic fashion to solve the original problem. One major advantage of modularity is that it allows the designer to apply the principle of separation of concern on individual modules.

© Copyright Virtual University of Pakistan

Software Design Process

Software design is not a sequential process. Design of a software system evolves through a number of iterations. The design process usually involves developing a number of different models, looking at the system from different angles and describing the system at various levels of abstraction. Like the various different models used during requirement engineering domain models, these models complement each other. As stated earlier, software design provides a road map for implementation by clearly describing how the software system is to be realized.

A activities performed at this stage include design of the software architecture by showing the division of system into sub-systems or modules, the specification of the services provided by these sub-systems and their interfaces with each other, division of each sub-system into smaller components and services and interfaces provided by each one of these components. Data modeling is also an essential activity performed during the design phase. This includes the identification of data entities and their attributes, relationships among these entities, and the appropriate data structures for managing this data.

Software Design Strategies

Software design process revolves around decomposing of the system into smaller and simpler units and then systematically integrates these units to achieve the desired results. Two fundamental strategies have been used to that end. These are functional or structured design and object oriented design.

In the functional design, the structure of the system revolves around functions. The entire system is abstracted as a function that provides the desired functionality (for example, the main function of a C program). This main function is decomposed into smaller functions and it delegates its responsibilities to these smaller functions and makes calls to these functions to attain the desired goal. Each of these smaller functions is decomposed into even smaller functions if needed. The process continues till the functions are defined at a level of granularity where these functions can be implemented easily. In this design approach, the system state, that is the data maintained by the system, is centralized and is shared by these functions.

The object-oriented design takes a different approach. In this case the system is decomposed into a set of objects that cooperate and coordinate with each other to implement the desired functionality. In this case the system state is decentralized and each object is held responsible for maintaining its own state. That is, the responsibility of marinating the system state is distributed and this responsibility is delegated to individual objects. The communication and coordination among objects is achieved through message passing where one object requests the other object if it needs any services from that object.

© Copyright Virtual University of Pakistan

The object-oriented approach has gained popularity over the structured design approach during the last decade or so because, in general, it yields a design that is more maintainable than the design produced by the functional approach.

Software Design Qualities

A software design can be looked at from different angles and different parameters can be used to measure and analyze its quality. These parameters include efficiency, compactness, reusability, and maintainability. A good design from one angle may not seem to be suitable when looked from a different perspective. For example, a design that yields efficient and compact code may not be very easy to maintain. In order to establish whether a particular design is good or not, we therefore have to look at the project and application requirements. For example, if we need to design an embedded system for the control of a nuclear reactor or a cruise missile, we would probably require a system that is very efficient and maintainability would be of secondary concern. On the other hand, in the case of an ordinary business system, we would have a reversal in priorities.

Maintainable Design

Since, in general, maintenance contributes towards a major share of the overall software cost, the objective of the design activity, in most cases, is to produce a system that is easy to maintain. A maintainable design is the one in which cost of system change is minimal and is flexible enough so that it can be easily adapted to modify exiting functionality and add new functionality.

In order to make a design that is maintainable, it should be understandable and the changes should be local in effect. That is, it should be such that a change in some part of the system should not affect other parts of the system. This is achieved by applying the principles of modularity, abstraction, and separation of concern. If applied properly, these principles yield a design that is said to be more cohesive and loosely coupled and thus is easy to maintain.

© Copyright Virtual University of Pakistan

Lecture No. 12

Coupling and Cohesion

Coupling is a measure of independence of a module or component. Loose coupling means that different system components have loose or less reliance upon each other. Hence, changes in one component would have a limited affect on other components.

Strong cohesion implies that all parts of a component should have a close logical relationship with each other. That means, in the case some kind of change is required in the software, all the related pieces are found at one place. Hence, once again, the scope is limited to that component itself.

A component should implement a single concept or a single logical entity. All the parts of a component should be related to each other and should be necessary for implementing that component. If a component includes parts that are not related to its functionality, then the component is said to have low cohesion.

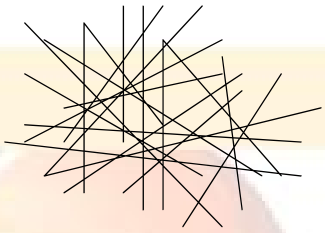
Coupling and cohesion are contrasting concepts but are indirectly related to each other. Cohesion is an internal property of a module whereas coupling is its relationship with other modules. Cohesion describes the intra-component linkages while couple shows the inter-component linkages. Coupling measures the interdependence of two modules while cohesion measures the independence of a module. If modules are more independent, they will be less dependent upon others. Therefore, a highly cohesive system also implies less coupling.

A good example of a system with a very high cohesion and very less (almost nil) coupling is the electric subsystem of a house that is made up of electrical appliances and wires. Since each one of the appliances has a clearly definable function that is completely encapsulated within the appliance. That means that an appliance does not depend upon any other appliance for its function. Therefore, each appliance is a highly cohesive unit. Since there are no linkages between different appliances, they are not coupled. Let us now assume that we have added a new centralized control unit in the system to control different appliances such as lights, air conditioning, and heating, according to certain settings. Since this control unit is dependent upon the appliances, the overall system has more coupling than the first one.

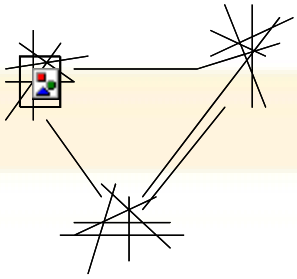
Modules with high cohesion and low coupling can be treated and analyzed as black boxes. This approach therefore allows us to analyze these boxes independent of other modules by applying the principle of separation of concern.

© Copyright Virtual University of Pakistan

Coupling and cohesion can be represented graphically as follows.



High Coupling



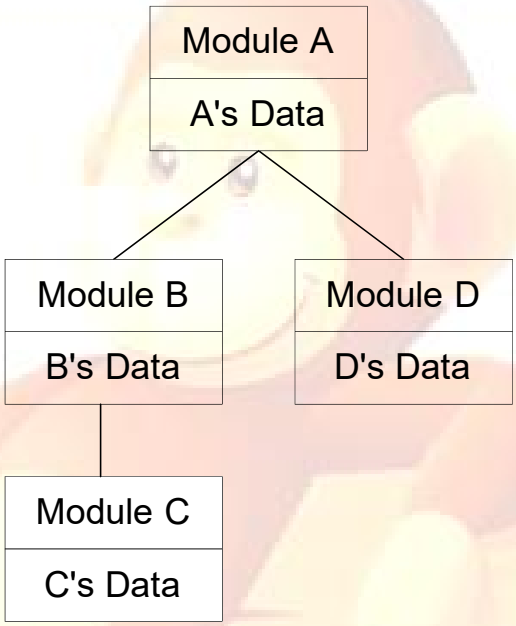
Low Coupling

This diagram depicts two systems, one with high coupling and the other one with low coupling. The lines depict linkages between different components. In the case of highly coupled system, module boundaries are not well defined, as everything seems to be connected with everything else. On the other hand, in the system with low coupling modules can be identified easily. In this case intra component linkages are stronger while inter component linkages are weak.

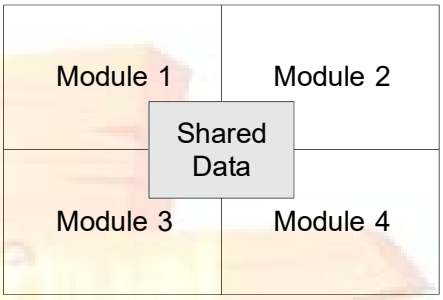
© Copyright Virtual University of Pakistan

Example of Coupling

The modules that interact with each other through message passing have low coupling while those who interact with each other through variables that maintain information about the state have high coupling. The following diagram shows examples of two such systems.



Low Coupling



High Coupling

In order to understand this concept, let us consider the following example. In this example, we have a class vector in which the data members have been put in the public part.

```
class vector {
  public:
        float x;
        float y;
        vector (float x, float y);
        float getX();
        float getY();
        float getMagnitude();
        float getAngle();
};
```

Now let us assume that we want to write a function to calculate dot product of two vectors. We write the following function.

© Copyright Virtual University of Pakistan

```
float myDotProduct1(vector a, vector b)
{
        float temp1 = a.getX() * b.getX();
        float temp2 = a.getY() * b.getY();
        return temp1 + temp2;
}
```

Since the data members are public, one could be enticed to use these members directly (presumably saving some function calls overhead) and rewrite the same function as follows:

```
float myDotProduct2(vector a, vector b)
{
        float temp1 = a.x * b.x;
        float temp2 = a.y * b.y;
        return temp1 + temp2;
}
```

So far, there does not seem to be any issue. But the scenario changes as soon as there are changes in the class implementation. Now let us assume that for some reason the class designer changes the implementation and data structure and decides to stores the angle and magnitude instead of the x and y components of the vector. The new class looks like as follows:

```
class vector {
  public:
        float magnitude;
        float angle;
        vector (float x, float y);
        vector (float magnitude, float angle);
        float getX();
        float getY();
        float getMagnitude();
        float getAngle();
};
```

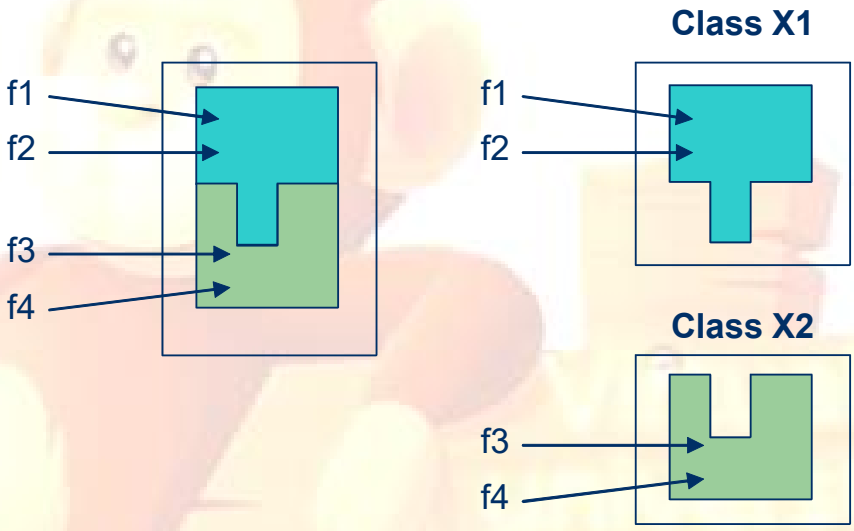
Now we see the difference in the two implementations of the dot product function written by the user of this class. In the first case, as the dot product function is dependent upon the public interface of the vector class, there will be no change while in the second case the function will have to be rewritten. This is because in the first case the system was loosely coupled while in the second case there was more dependency on the internal structure of the vector class and hence there was more coupling.

© Copyright Virtual University of Pakistan

Example of Cohesion

As mentioned earlier, strong cohesion implies that all parts of a component should have a close logical relationship with each other. That means, in case some kind of change is required in the software, all the related pieces are found at one place.

A class will be cohesive if most of the methods defined in a class use most of the data members most of the time. If we find different subsets of data within the same class being manipulated by separate groups of functions then the class is not cohesive and should be broken down as shown below.



© Copyright Virtual University of Pakistan

As an example, consider the following order class:

```
class order {
  public:
        int getOrderID();
        date getOrderDate();
        float getTotalPrice();
        int getCustometId();
        string getCustomerName();
        string getCustometAddress();
        int getCustometPhone();

        void setOrderID(int oId);
        void setOrderDate(date oDate);
        void setTotalPrice(float tPrice);
        void setCustometId(int cId);
        void setCustomerName(string cName);
        void setCustometAddress(string cAddress);
        void setCustometPhone(int cPhone);
        void setCustomerFax(int cFax)
  private:
        int oredrId;
        date orderDate;
        float totalPrice;
        item lineItems[20];
        int customerId;
        string customerName;
        int customerPhone;
        int customerFax;
};
```

The Order class shown above represents an Order entity that contains the attributes and behavior of a specific order. It is easy to see that this contains information about the order as well as the customer which is a distinct entity. Hence it is not a cohesive class and must be broken down into two separate classes as shown. In this case each on of these is a more cohesive class.

© Copyright Virtual University of Pakistan

```
class order {
   public:
        int getOrderID();
        date getOrderDate();
        float getTotalPrice();
        int getCustometId();

        void setOrderID(int oId);
        void setOrderDate(date oDate);
        void setTotalPrice(float tPrice);
        void setCustometId(int cId);
        void addLineItem(item anItem);
   private:
        int oredrId;
        date orderDate;
        float totalPrice;
        item lineItems[20];
        int customerId;
};

class customer {
   public:
        int getCustometId();
        string getCustomerName();
        string getCustometAddress();
        int getCustometPhone();
        int getCustomerFax();

        void setCustometId(int cId);
        void setCustomerName(string cName);
        svoid setCustometAddress(string cAddress);
        void setCustometPhone(int cPhone);
        void setCustomerFax(int cFax)
   private:
        int customerId;
        string customerName;
        int customerPhone;
        int customerFax;
};
```


© Copyright Virtual University of Pakistan

Abstraction and Encapsulation

Abstractions is a technique in which we construct a model of an entity based upon its essential characteristics and ignore the inessential details. The principle of abstraction also helps us in handling the inherent complexity of a system by allowing us to look at its important external characteristic, at the same time, hiding its inner complexity. Hiding the internal details is called encapsulation. In fact, abstraction is a special case of separation of concern. In this case we separate the concern of users of the entity who only need to understand its external interface without bothering about its actual implementation.

Engineers of all fields, including computer science, have been practicing abstraction for mastering complexity. Consider the following example.

```
void selectionSort(int a[], int size)
{
        int i, j, min, temp;
        for(i = 0; i < size –1; i++)
        {
                min = i;
                for (j = i; j < size; j++)
                {
                        if (a[j] < a[min])
                                min = j;
                }
                temp = a[i];
                a[i] = a[min];
                a[min] = temp;
        }
}
```

This function can be rewritten by abstracting out some of the logical steps into auxiliary functions. The new code is as follows.

```
viod swap(int &x, int &y)
{
        int  temp;
        temp = x;
        x = y;
        y = temp;
}
```


© Copyright Virtual University of Pakistan

```
int indexOfMinimumValue(int a[], int from, int to)
{
        int i, min;
        min = from;
        for (i = from+1; i < to; i++)
                if (a[i] < a[min]) min = i;
        return min;
}

void selectionSort(int a[], int size)
{
        int i, min;
        for (i = 0; i < size; i++)
        {
                min = indexOfMinimumValue(a, i, size);
                swap(a[i], a[min]);
        }
}
```

In this function we have abstracted out two logical steps performed in this functions. These functions are finding the index of the minimum value in the given range in an array and swapping the minimum value with the value at the ith index in the array. It is easy to see that the resultant new function is easier to understand than the previous version of the selection sort function. In the process, as a by-product, we have created two auxiliary function mentioned above, which are general in nature and hence can be used elsewhere as well. Principle of abstraction thus generates reusable self-contained components.

Function Oriented versus Object Oriented Design

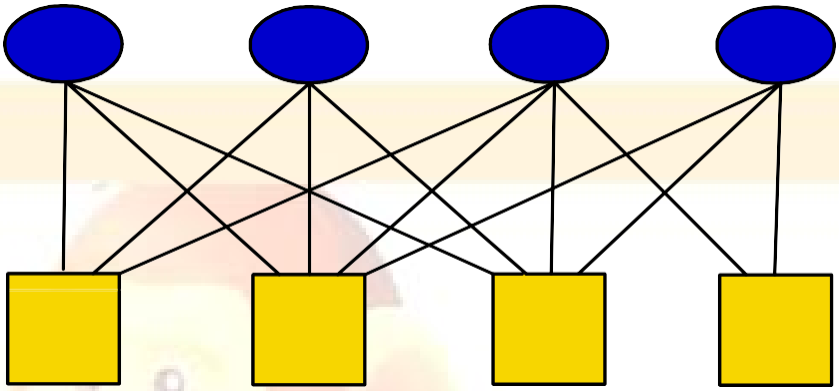
Let us now try to understand the difference between object-oriented and function oriented (or action oriented) approach.

In the case of action-oriented approach, data is decomposed according to functionality requirements. That is, decomposition revolves around function. In the OO approach, decomposition of a problem revolves around data. Action-oriented paradigm focuses only on the functionality of a system and typically ignores the data until it is required. Object-oriented paradigm focuses both on the functionality and the data at the same time. The basic difference between these two is decentralized control mechanism versus centralized control mechanism respectively. Decentralization gives OO the ability to handle essential complexity better than action-oriented approach.

This difference is elaborated with the help of the following diagram:

© Copyright Virtual University of Pakistan

Functions

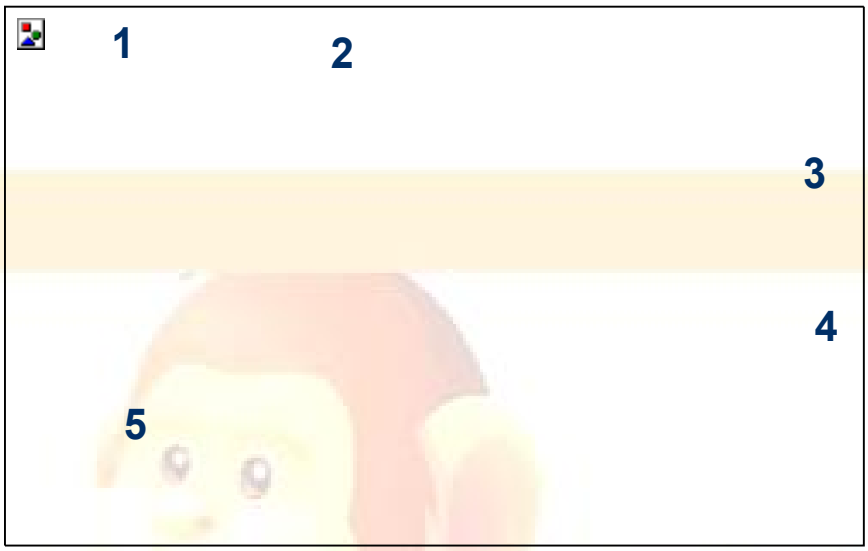


Data

In this diagram, the ovals depict the function while rectangles/squares depict data. Since a function contains dynamic information while data contains only static information, if the function and data are managed separately, the required data components can be found by scanning a function but the functions that use a particular data cannot be found by just looking at the data. That is, the function knows about the data it needs to use but the data do not know about the functions using it. That means it is easy to make a change in a function since we would know which data components would be affected by this change. On the other hand, changing a data structure would be more difficult because it would not be easy to find all the functions that are using this data and hence also need to be modified.

Object oriented approach solves this problem by putting the relevant data and functionality together at one place. Hence, in case of a change, the effected components can be identified easily and the effect of change is localized. Therefore, maintenance becomes relatively easy as compared to function-oriented approach. This is made possible because the data is not shared in this case. Anyone needing any information contained in there would request the encapsulating object by sending it a message through the interface provided by the object. In this case we create highly cohesive objects by keeping the related data and function at one place and spinning-off non-related information into other classes. This can be elaborated with the help of the following diagram.

© Copyright Virtual University of Pakistan



Let us assume that the circles represent sets of functions and rectangles represent data that these function use to carry out their operation. In the object-oriented design, the data areas that are common among different sets of functions would be spun-off into their own classes and the user function would use these data through their interfaces only. This is shown in the following diagram.



© Copyright Virtual University of Pakistan

Lecture No. 13

Object Oriented Analysis and Design

Object Oriented Design - Why?

Software is primarily used to represent real-life players and processes inside a computer. In the past, software was considered as a collection of information and procedures to transform that information from input to the output format. There was no explicit relationship between the information and the processes which operate on that information. The mapping between software components and their corresponding real-life objects and processes was hidden in the implementation details. There was no mechanism for sharing information and procedures among the objects which have similar properties. There was a need for a technology which could bridge the gap between the real-life objects and their counter-parts in a computer. Object oriented technology evolved to bridge the gap. Object-oriented technology helps in software modeling of real-life objects in a direct and explicit fashion, by encapsulating data and processes related to a real-life object or process in a single software entity. It also provides a mechanism so that the object can inherit properties from their ancestors, just like real-life objects.

A complex system that works is invariably found to have evolved from a simple system that worked. The structure of a system also plays a very important role. It is likely that we understand only those systems which have hierarchical structure and where intra-component linkages are generally stronger than inter component linkages. That leads to loose coupling, high cohesion and ultimately more maintainability which are the basic design considerations. Instead of being a collection of loosely bound data structures and functions, an object-oriented software system consists of objects which are, generally, hierarchical, highly cohesive, and loosely coupled.

Some of the key advantages which make the object-oriented technology significantly attractive than other technologies include:

- Clarity and understandability of the system, as object-oriented approach is closer to the working of human cognition.
- Reusability of code resulting from low inter-dependence among objects, and provision of generalization and specialization through inheritance.
- Reduced effort in maintenance and enhancement, resulting from inheritance, encapsulation, low coupling, and high cohesion.

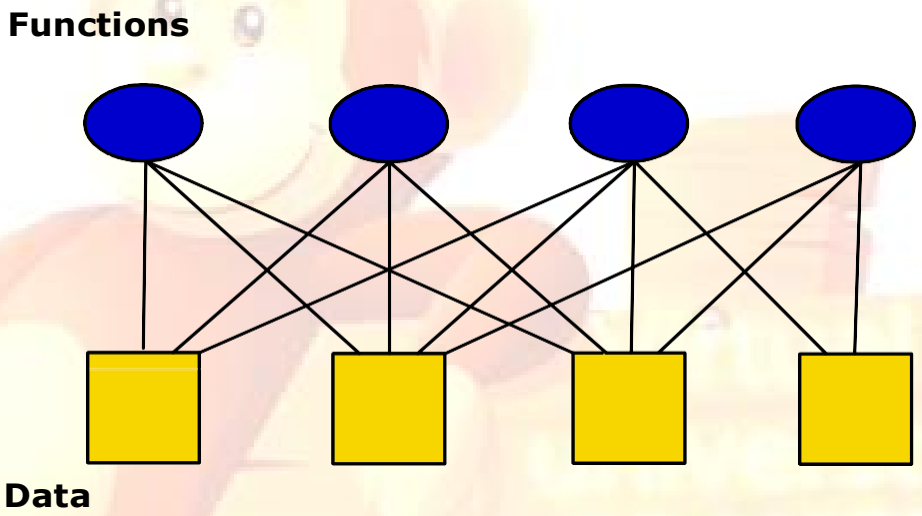
Difference between object-oriented and function-oriented design

Before talking about how to derive and object-oriented design, we first need to understand the basic difference between object-oriented and function oriented (or action oriented) approach.

© Copyright Virtual University of Pakistan

In the case of action-oriented approach, data is decomposed according to functionality requirements. That is, decomposition revolves around function. In the OO approach, decomposition of a problem revolves around data. Action-oriented paradigm focuses only on the functionality of a system and typically ignores the data until it is required. Object-oriented paradigm focuses both on the functionality and the data at the same time. The basic difference between these two is decentralized control mechanism versus centralized control mechanism respectively. Decentralization gives OO the ability to handle essential complexity better than action-oriented approach.

This difference is elaborated with the help of the following diagram:



In this diagram, the ovals depict the function while rectangles/squares depict data. Since a function contains dynamic information while data contains only static information, if the function and data are managed separately, the required data components can be found by scanning a function but the functions that use a particular data cannot be found by just looking at the data. That is, the function knows about the data it needs to use but the data do not know about the functions using it. That means, it is easy to make a change in a function since we would know which data components would be affected by this change. On the other hand, changing a data structure would be difficult because it would not be easy to find all the functions that are using this data and hence also need to be modified.

In the case of OO design since data and function are put together in one class, hence, in case of a change, the effected components can be identified easily and the effect of change is localized. Therefore, maintenance becomes relatively easy as compared to function-oriented approach.

© Copyright Virtual University of Pakistan

Object Oriented Design Components - What?

The Object and the Class

The basic unit of object oriented design is an object. An object can be defined as a tangible entity that exhibits some well defined behavior. An object represents an individual, identifiable item, unit, or entity, either real or abstract, with a well defined role in the problem domain. An object has state, behavior, and identity.

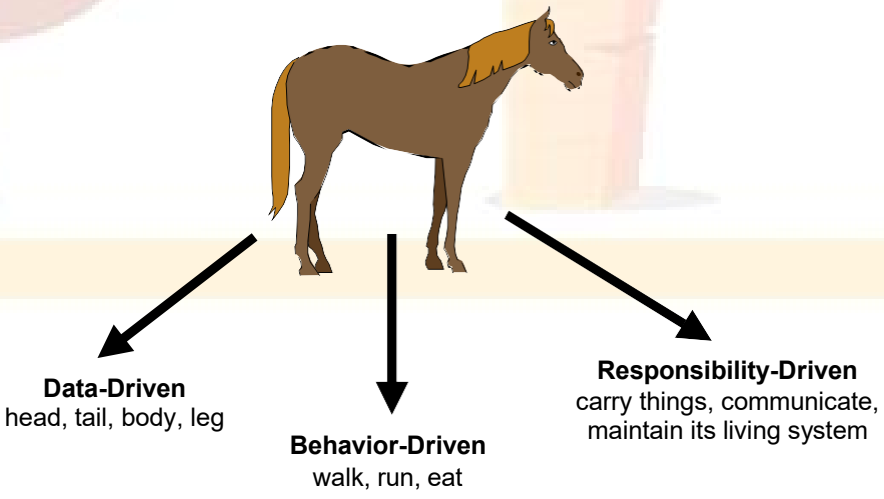
The state of an object encompasses all of the properties of the object and their current values. A property is an inherent or distinctive characteristic. Properties are usually static. All properties have some value. The state of an object is encapsulated within the object.

Behavior is how an object acts and reacts in terms of its state changes and message passing. The behavior of an object is completely defined by its actions. A message is some action that one object performs upon another in order to elicit a reaction. The operations that clients may perform upon an object are called methods.

The structure and behavior of similar objects are defined in their common class. A class represents an abstraction - the essence or the template of an object. A class specifies an interface (the outside view - the public part) and defines an implementation (the inside view - the private part). The interface primarily consists of the declaration of all the operations applicable to instances of this class. The implementation of a class primarily consists of the implementation of all the operations defined in the interface of the class

Classification

The most important and critical stage in the OOA and OOD is the appropriate classification of objects into groups and classes. Proper classification requires looking at the problem from different angles and with an open mind. When looked at from different perspectives and analyzed with different set of characteristics, same object can be classified into different categories. Let us try to understand this with the help of an example.



© Copyright Virtual University of Pakistan

Here, we can take a data-driven, behaviour driven, or responsibility driven perspective and will categorize the horse accordingly.

The Object Model

The elements of object oriented design collectively are called the Object Model. The object model encompasses the principles of abstraction, encapsulation, and hierarchy or inheritance.

Abstraction is an extremely powerful technique for dealing with complexity. Unable to master the entirety of a complex object, we ignore its essential details, dealing instead with generalized, idealized model of the object. An abstraction focuses on the outside view of an object, and hence serves to separate an objects external behavior from its implementation. Deciding upon the right set of abstractions for a given domain is the central problem in object oriented design.

Abstraction and encapsulation are complementary concepts. Abstraction provides the outside view to the client and encapsulation prevents clients from seeing its inside view. For abstraction to work, implementation must be encapsulated. Encapsulation hides the details of the implementation of an object. Intelligent encapsulation localizes design decisions that are likely to change. The ability to change the representation of an object without disturbing any of its clients is the essential benefit of encapsulation.

Relationship Among Objects

The object model presents a static view of the system and illustrates how different objects collaborate with one another through patterns of interaction. Inheritance, association and aggregation are the three inter-object relationships specified by the object model.

Inheritance defines a “kind of” hierarchy among classes. By inheritance, we specify generalization/specialization relationship among objects. In this relationship, a class (called the subclass) shares the structure and behavior defined in another class (called the superclass). A subclass augments or redefines the existing structure and behavior of its superclass. By classifying objects into groups of related abstractions, we come to explicitly distinguish the common and distinct properties of different objects, which further help us to master their inherent complexity. Identifying the hierarchy within a complex system requires the discovery of patterns among many objects.

In an association relationship, when object A “uses” object B, then A may send messages to B. The relationship defines visibility among objects.

The aggregation relationship defines part-of structure among objects. When object A is part of the state of object B, A is said to be contained by B. There are some tradeoffs between aggregation and association relationships. Aggregation reduces the number of objects that must be visible at the level of enclosing objects and may lead to undesirable tighter coupling among objects.

© Copyright Virtual University of Pakistan

Aggregation and Association - Conceptual and Implementation Issues and Differences

Association and Aggregation - Some basic differences

Objects do not exist in isolation. They rather collaborate with one another in many different ways to achieve an overall goal. The different types of relationships in which these objects are involved include association, aggregation, and inheritance. Briefly, inheritance denotes a "kind of" relationship, aggregation denotes a "part of" relationship, and association denotes some semantic connection among otherwise unrelated classes. Any further elaboration on inheritance relationship is beyond the scope of this discussion and therefore we shall concentrate on agrregation and association relationships only.

As mentioned earlier, aggregation is the "part-whole" or "a-part-of" relationship in which objects representing the components of something are encapsulated within an object representing the entire assembly. In other words, the whole is meaningless without its parts and the part cannot exist without its container or assembly. Some properties of the assembly propagate to the components as well, possibly with some local modifications. Unless there are common properties of components that can be attached to the assembly as a whole, there is little point in using aggregation. Therefore, as compared to association, aggregation implies a tighter coupling between the two objects which are involved in this relationship. Therefore, one way to differentiate between aggregation and association is that if the two objects are tightly coupled, that is, if they cannot exist independently, it is an aggregation, and if they are usually considered as independent, it is an association.

Object Creation and Life Time

From the object creation and life time point of view, when an object is instantiated, all of its parts must also be instantiated at the same time before any useful work can be done and all of its part die with it. While in the case of association, the life time of two associated object is independent of one another. The only limitation is that an object must be alive or has to be instantiated before a message can be sent to it.

Coupling and Linkages

As mentioned earlier, aggregation implies a much tighter coupling than association. In case of aggregation, the links between the whole and its part are permanent while in case of association the links may be maintained only just for the period an object requires the services of its associated object and may be disconnected afterwards.

Ownership and visibility

Another way of differentiating among the two is to look at them from the ownership and sharing point of view. In case of aggregation, since the whole contains the part, the part is encapsulated or hidden within the whole and is not accessible from outside while in case of association, the associated object may be used directly by other objects also. That is, in case of aggregation, only the whole is supposed to send a message to its parts while in case of association, anyone who holds a reference to it can communicate with it directly.

© Copyright Virtual University of Pakistan

In other words, in case of aggregation, the whole owns its parts and the part becomes a private property of the whole. For all practical purposes, any other object does not even need to know about its existence. On the other hand, an associated object may be shared among many different objects. That is, many different object may hold reference to the same object simultaneously.

Database persistence

From a database perspective, when an object is persisted or stored in the database, all of its components (all parts of the whole) must also be persisted in their entirety along with the "whole" for future reference while only a reference to the associated object may be stored in the database. Note that a normalized database would also enforce the above restriction.



© Copyright Virtual University of Pakistan

Lecture No. 14

Object Oriented Analysis

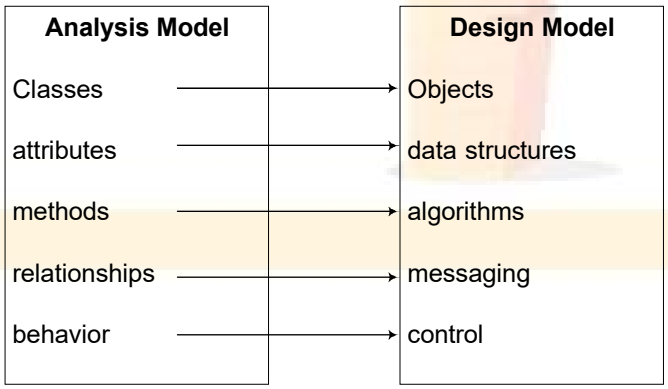
The intent of OOA is to define all classes, their relationships, and their behavior. A number of tasks must occur:

1) Static Model
   a) Identify classes (i.e. attributes and methods are defined)
   b) Specify class hierarchy
   c) Identify object-to-object relationships
   d) Model the object behavior
2) Dynamic Model
   a) Scenario Diagrams

Object Oriented Design

OOD transforms the analysis model into design model that serves as a blueprint for software construction. OOD results in a design that achieves a number of different levels of modularity. The four layers of the OO design pyramid are:

1) **The subsystem layer.** Contains a representation of each of the subsystems that enable the software to achieve its customers defined requirements and to implement the technical infrastructure that supports customer requirements.
2) **The class and object layer.** Contains the class hierarchies that enable the system to be created using generalization and increasingly more targeted specializations. The layer also contains design representations for each object.
3) **The message layer.** Contains the details that enable each object to communicate with its collaborators. This layer establishes the external and internal interfaces for the system.
4) **The responsibility layer.** Contains the data structures and algorithmic design for all attributes and operations for each object.



Translating the analysis model into a design model during object design

© Copyright Virtual University of Pakistan

Object-Oriented Analysis using Abbot's Textual Analysis

The first object-orientation technique that we will study is one of the oldest techniques to identify objects and their relationships. This technique is called Textual Analysis. It was initially developed by Abbot and then extended by Graham and others. In this technique different parts of speech are identified within the text of the specification and these parts are modeled using different components. The following table shows this scheme.

Part of speech	*Model component*	*Example*
proper noun	instance	Mehdi Hassan
improper noun	class/type/role	student, teacher
doing verb	operation	buy
being verb	classification	is a horse, is a book
having verb	composition	fan has wings
adjective	attribute value or class	this ball is green
adjective phrase	association	the customer with children
	operation	the customer who bought the kite

Once all the model components have been identified, we will eliminate the redundant or irrelevant components by again analyzing the text and the context of the problem.
Let's now try to understand this with the help of an example:

Problem Statement:
A simple cash register has a display, an electronic wire with a plug, and a numeric keypad, which has keys for subtotal, tax, and total. This cash storage device has a total key, which triggers the release on the drawer. The numeric buttons simply place a number on the display screen, the subtotal displays the current total, the tax key computes the tax, and the total key adds the subtotal to the tax.

Our task now is to:

- Identify all the classes in this problem statement.
- Eliminate the unnecessary classes.

We are now going to use nouns to find classes.

Nouns (initial)

Register	Display	Wire
Plug	Keypad	Keys
Devices	Release	Drawer
Buttons	Screen	Number

© Copyright Virtual University of Pakistan

Total Tax

Nouns (General Knowledge)

0-9 keys Money Subtotal Key
Tax Key Total Key

Eliminating Irrelevant/Redundant Nouns

We now analyze the identified nouns and try to establish whether they would be stand-alone classes in our domain or not. Outcome of this analysis is shown below.

Register
Display
Wire → Irrelevant
Plug → Irrelevant
Keypad
Keys
Devices → Vague
Release → Irrelevant
Drawer
Buttons → Redundant
Screen → Redundant
Number → Attribute
Total → Attribute
Tax → Attribute
0-9 Key
Value → Attribute
Money
Subtotal Key
Tax Key
Total Key

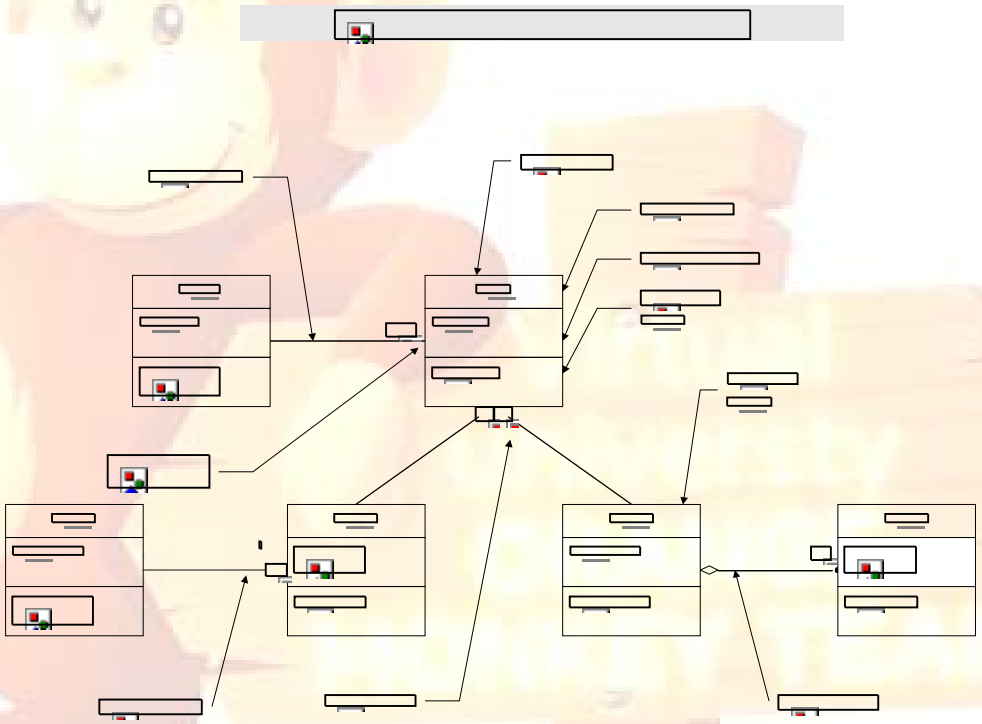
We will continue with technique to identify all the constituent components of the model and derive our object-oriented design.

© Copyright Virtual University of Pakistan

Lecture No. 15

The Notation

Many different notations are used for documenting the object oriented design. Most popular of these include, Rumbaugh, Booch, and Coad, and UML(Unified Modeling Language). We will be using UML to document our design. Although the notation is very comprehensive and detailed, but the key features of this notation are presented in the following diagram.



© Copyright Virtual University of Pakistan

Lecture No. 16

Derivation of the Object Model – The Coad Methodology

An object model of a system captures the static structure of a system by showing the objects in the systems, their relationships, their attributes, and their services. To stream line the derivation of the object model, Peter Coad has divided the process into 5 activities, each being further subdivided into a number of steps. Following is the description of these activities.

Select Objects – who am I?

We have used an approach that divides the objects into different categories to make it easier to find them and establish their attributes, services, and collaborations. This activity, consisting of 6 steps, can help you find objects and categorize them. These steps are:

Select actors

Actors are people and organizations that take part in the system under consideration. Examples of actors are: person, organization (agency, company, corporation, foundation). Note that we are talking about actors and not their "roles". e.g. a customer is a role that a person plays, so if we have a customer in our problem domain, we will also add a person as actor in the model.

Select Participants

A participant is a role that each actor plays in the system under consideration. Examples of participants are: agent, applicant, buyer, cashier, clerk, customer, dealer, distributor, donor, employee, investor, member, officer, owner, policy holder, recipient, student, supervisor, supplier, teacher, worker. It may be noted that the same person may play different roles at different times in the system. That means that if we model this behavior using Generalization-Specialization instead of Actor-Participant, we may end up with multiple inheritance.

Select Places

Places are where things come to rest or places that contain other objects. Examples of places are: airport, assembly-line, bank, city, clinic, country, depot, garage, hanger, hospital, plant, region, sales outlet, service center, shelf, station, store, warehouse, zone.

Select Transactions

Transactions are the "events" that must be remembered through time. These are entries that must be maintained in a historical record or log which may be used to answer questions or perform assessments. These transactions usually come from a window (GUI), some object which monitors for significant event and logs that information, or a another system that interacts with the system under consideration and logs some

© Copyright Virtual University of Pakistan

information. Examples of transactions are: agreement, assignment, authorization, contract, delivery, deposit, incident, inquiry, order, payment, problem report, purchase, refund, registration, rental, reservation, sale, shift, shipment, subscription, withdrawal. Note that nearly all transactions consist of a number of transaction line items.

Select Container Objects

Containers are objects that hold other objects. Note the similarity of definition between container and places. The difference is that a place is a place in the literal sense while a container is a any object that can hold other objects, e.g. bin, box, cabinet, folder, locker, safe, shelf, etc. Therefore a place is also a container but every container need not be a place.

Select Tangible things

Take a "walk" through the system and select "tangible" things around you used in the problem domain. These may be characterized as all the remaining (not yet selected) "nouns" that make up the problem domain. Examples are: account, book, calendar, cash box, cash drawer, item, plan, procedure, product, schedule, skill, tool, etc.

While selecting objects, the following considerations should be kept in mind for a simpler (and better) object model.

1. Every object that you put in your object model should have some responsibility or role to play in the problem domain. You need to know each object, its attributes, and services. If there is no way to know about the object, remove it from the object model.

2. Avoid having controller objects because controllers usually end up with functionality that's better done by other objects themselves, making the message passing more complicated, and resulting in higher coupling. Use delegation instead. Note the difference between controlling and delegation; a controller wants to do every thing by himself (doesn't trust anyone), while a good manager delegates responsibility (and takes credit).

3. In large systems several objects are likely to have similar or even identical responsibilities. Look for such objects and seek a common name to simplify the object model.

4. Use meaningful class names, names that describe objects in that class. Try to use names from the domain vocabulary to avoid confusion.

© Copyright Virtual University of Pakistan

Lecture No. 17

Identify Structures

A structure is a manner of organization which expresses a semantically strong organization within the problem domain. There are two type of structures: Generalization-Specialization (Gen-Spec) and whole-part. This activity covers the identification of these structures in the following 2 steps:

Identify Gen-Spec Structures (Hierarchy)

Consider each class that you have identified as a specialization and then look for its generalization and vice versa.

Identify Whole-Part structures (Aggregations) - What are my components?

For each object that you have identified, consider it as a whole and then try to find out its parts - objects that make up this object.

Define Attributes - What I Know?

The first two activities would identify most of the objects (classes) in the problem domain. Now is the time to think about the role and responsibilities of these objects. The first thing to consider is their attributes, i.e., what it knows.

For each object include the attributes that come to mind when you first think about the object. The criteria for the inclusion of an attribute is that it should be included if the system needs to know its value and it cannot get it any other way. Don not add an attribute for an association or aggregation. Examples of attributes are: number, name, address, date, time, operational state, phone, status, threshold, type, etc. In particular, consider the following attributes for different types of objects.

1. For actors consider name, address, phone.
2. For participants consider number, date and time, password, authorization level.
3. For place/location consider number, name, address (perhaps latitude, longitude, altitude).
4. For transaction consider number, date, time, status.
5. For line item consider quantity, status.
6. For item consider name, description, dimension, size, UPC, weight.

Like object selection, there are a number of issues that every designer must be aware of while defining attributes of an object. These are:

1. An attribute that varies over time, e.g., price of an item, should be replaced by an additional class with an effective date and value.
2. An attribute that may have a number of values should be replaced by a new class and an object connection.
3. Replace “yes/no” type attributes with “status” type attributes for flexibility.
4. If there are classes with common attributes and generalization-specialization makes good sense, then add a generalization class and factor out the commonality.

© Copyright Virtual University of Pakistan

Show Collaborations (associations and aggregations) - Who I know?

The second step in establishing each object's responsibility is to identify and show how this object collaborates with other objects, i.e., who it knows. These collaborations can be identified with the help of the following 8 steps:

1. For an actor, include an object connect to its participants (association).
2. For a participant, include an object connection to its actor (already established) and its transactions (association).
3. For a location, include object connections to objects that it can hold (association), to its part objects (aggregation), and to the transactions that are taking place at that location (association).
4. For transactions, include object connections to its participants (already established), its line items (aggregation), and its immediate subsequent transaction (aggregation).
5. For a transaction line item, include object connections to its transaction (already established), its item (association), a companion "item description" object (association), and a subsequent line item (association).
6. For an item, include object connections to transaction line item (already established), a companion "item description" object (association).
7. For a composite object, include object connections to its "part" object (aggregation).
8. For all objects (including all of the above) select connecting objects to which the object under consideration sends a message (within one or more scenarios) to get some information or to answer a query about objects directly related to it (association).

Define Services - What I do?

The third and last step in establishing each object's responsibility is to define what services does each object in the problem domain provide, i.e., what it does. Putting the right service with the right object is also very important since any mistake in judgment will increase coupling and reduce cohesion. The verbs in your problem domain usually indicate some of the services required of the associated object.

Software objects do things that the system is responsible to do with regard to that object. By putting the services with the attributes they work on results in lower coupling and stronger cohesion, and increased likelihood of reuse. The basic principle is to keep data and action together for lower coupling and better cohesion. The basic services, done by all (such as get, set, create, initialize), are not shown in the object model. While establishing the services for an object, the following fundamental questions should be asked:

1. Why does the system need this object any way?
2. What useful questions can it answer?
3. What useful action can it perform?
4. What this object can do, based upon what it knows?
5. What this object can do, based upon whom it knows?

© Copyright Virtual University of Pakistan

6. What calculations can it do?
7. What ongoing monitoring could it do?
8. What calculations across a collection could it make (letting each worker do its part)?
9. What selections across a collection could it make (letting each worker do its part)?

While establishing services of certain specific types of objects, the following should be considered:

1. For an actor, consider: calculate for me, rate me, is <value>, rank participants, calculate over participants.
2. For a participant, consider: calculate for me, rate me, is <value>, rank transactions, calculate over transactions.
3. For a place, consider: calculate for me, rate me, is <value>, rank transactions, calculate over contents, calculate over container line items.
4. For a Transaction, consider: calculate for me, rate me, is <value>, how many, how much, rank transaction line items, rank subsequent transactions, calculate over transaction line items, calculate over subsequent transactions.
5. For a line item, consider: calculate for me, rate me.
6. For an item, consider: calculate for me, rate me, is <value>, how many, how much, rank, calculate over specific items.

Lecture No. 18

CASE STUDY: Connie's Convenience Store - A point of Sale System

The System

Identify the purpose of the system

- develop an overall purpose statement in 25 words or less. Why this system? Why now?
- Keep the overall goal, the critical success factor, always before you.
- "To support, to help, to facilitate, ..."

Connie's Wish List

- scan items and automatically price them
- know whether an item is on sale
- automatically total the sale and calculate tax
- handle purcahses and returns
- handle payments with cash, check, or charge
- authorize checks and cards
- calculate change when working with cash or checks
- record all of the information about a customer transaction

© Copyright Virtual University of Pakistan

- balance the cash in the drawer with the amount recorded by the point-of-sale system.

Why ?

- speed up checkout time
- reduce the number of pricing errors
- reduce the labour required to ticket the item with a price, originally and when prices change.

Summary

to help each cashier work more effectively during checkout, to keep good records of each sale, and to store more efficient store operations.

Identify system features

Be certain to include features that cover the following

1. log important information
2. conduct business
3. analyze business results
4. interact with other systems

Identify features for logging important information

- to maintain prices based upon UPC
- to maintain tax categories (categories, rates, and effective dates)
- to maintain the authorized cashiers
- to maintain what items we sell in a store
- to log the results of each sale in a store

Identify features for conducting business

- to price each item, based upon its UPC
- to subtotal, calculate tax, and total
- to accept payment by cash, check, or charge

Identify features for analyzing business results

- to count how many of each item sold
- to count how much we received in cash, check, or credit card sales
- to assess how each cashier is performing
- to assess how each store is performing

Identify features for working with interacting systems

- to obtain authorization from one or more credit (or check) authorization system

© Copyright Virtual University of Pakistan

SELECTING OBJECTS

Select Actors

the actor is:

- person

Select Participants

the Participants are:

- cashier
- head cashier
- customer

Cashier and Head Cashier

Is there a difference between head cashier and cashier in terms of their behavior and knowledge?. If no then we don not need a separate class for head cashier.

Customer

customer. You must have a way to know about customer objects; otherwise it should not be put in the domain model.

Select Places

The places are:

- store
- shelf

Shelf

The system does not keep track of the shelves.

Select Transactions

Significant Transactions are:

- sale
- every sale is a collection of sale line items
- return
- payment
- session

Select Container Classes

The store is a container class.

a store contains

- cashiers
- registers
- items

Select Tangible Things

Tangible things in store:

© Copyright Virtual University of Pakistan

- item
- register
- cash drawer
- Tax Category (Descriptive things)

Session

Is it important? It is important in order to evaluate a cashier's performance.

Lecture No. 19

Identify Structures

Identify Gen-Spec Structures

Kinds of stores:

A store is a kind of sales outlet. Perhaps over time, Connie will expand to other kinds of sales outlets. Stores might be specialized into kinds of stores. For now on leave store as it is.

Kinds of sales:

- sales, returns
- only different is that the amount is positive or negative. Is there any other difference?

Prices:

- regular price, and promotional (sales) price

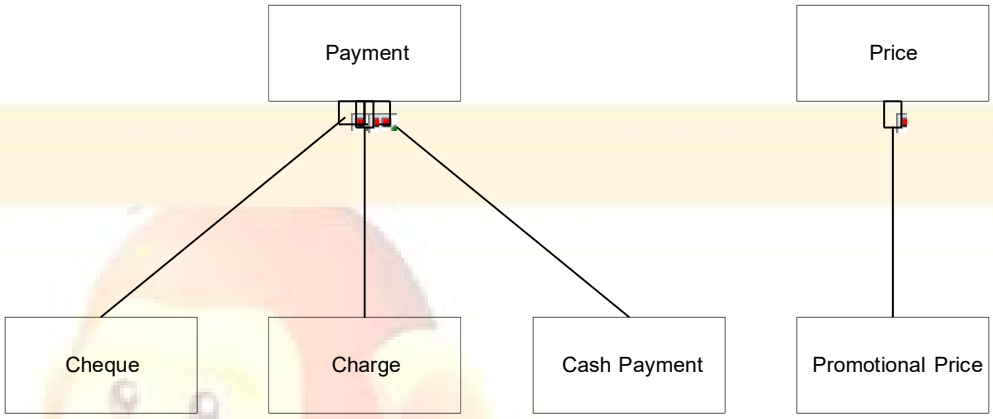
Payment:

- cash, check, and charge are kind of payments

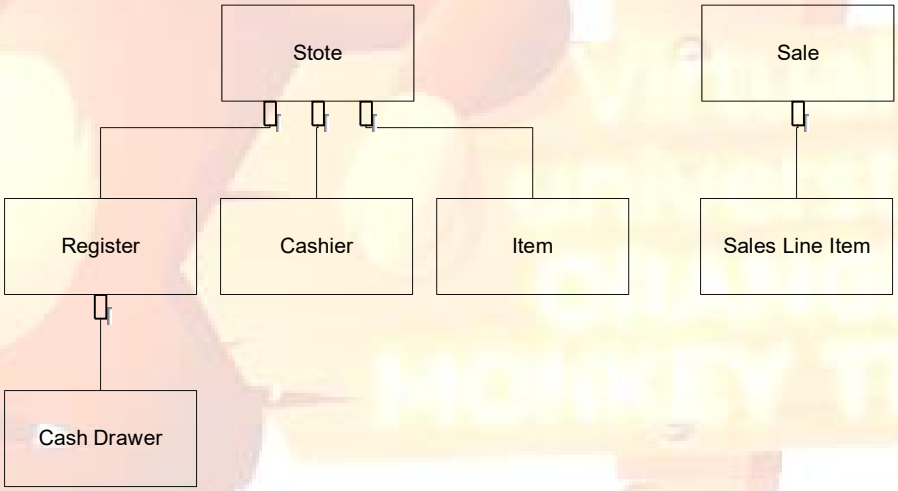
Identify Whole-Part Structures

- A store as a whole is made up of cashiers, registers, and items.
- A register contains a cash drawer.
- A sale is constituted of sale line items.

© Copyright Virtual University of Pakistan



Object Hierarchy



Whole-Part Structures

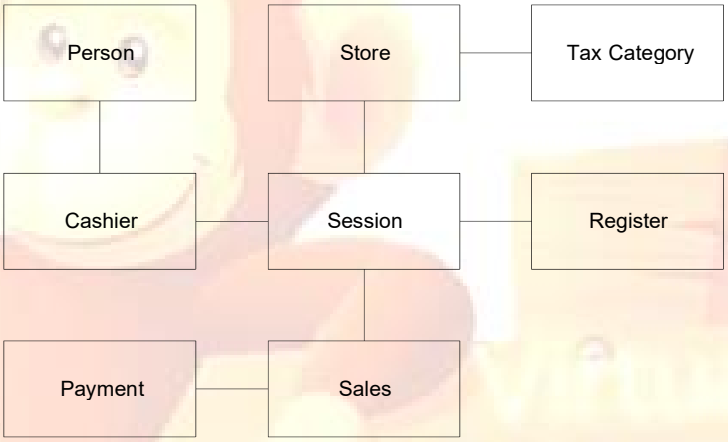
Establishing Responsibilities

Who I Know - Rules of Thumb

- an actor knows about its participants
  person knows about cashier
- a transaction knows about its participants
  a session knows about its register and cashier
- A transaction contains its transaction line items
  sale contains its sales line items

© Copyright Virtual University of Pakistan

- A transaction knows its sub transactions
  session knows about its sales
  sale knows about its payments
- A place knows about its transactions
  store knows about its sessions
- A place knows about its descriptive objects
  store knows about its tax categories
- A container knows about its contents
  a store knows about its cashiers, items, and registers



Association Relationships

Define Attributes, Services, and Links - What I know, What I do, and Who I know?

Actors:

person

Attributes: name, address, phone
Services: eating, walking

Participants:

cashier

Attributes: number, password, authorization level, current session
Services: isAuthorized, assess Performance

Places:

store

Attributes: name
Services: get item for UPC, get cashier for number

© Copyright Virtual University of Pakistan

Tangible things:

item

Attributes: number, description, UPCs, prices, taxable
attributes with repeating names - create new objects UPC, Price (specialization - promotional price)
Services: get price for a date, how much for quantity
Who I Know? UPC, Price, tax category, sale line item

register

Attributes: number
Services: how much over interval, how many over interval
Who I know? store, session, cash drawer (part of register)

cash drawer

Attributes: balance, position (open, close), operational state
Services: open
Who I know? register

Tax Category

Attributes: category, rate, effective date
Services: just the basic services - get, add, set - don't show
Who I know? items?

Transactions:

sale

Attributes: date and time
Services: calculate subtotal, calculate total, calculate discount, calculate
tax, commit
Who I Know? session, payment, SLIs

sale line item

Attributes: date and time ?, quantity, tax status (regular, resale, tax-exempt)
Services: calculate sub total
Who I Know? item, sale

sale line item - how do you handle returns and sales

sale - you have control
return - more difficult
- return to a different store
- purchased for a different price
- returns an item no longer in the inventory

return

Attributes: return price, reason code, sale date, sale price

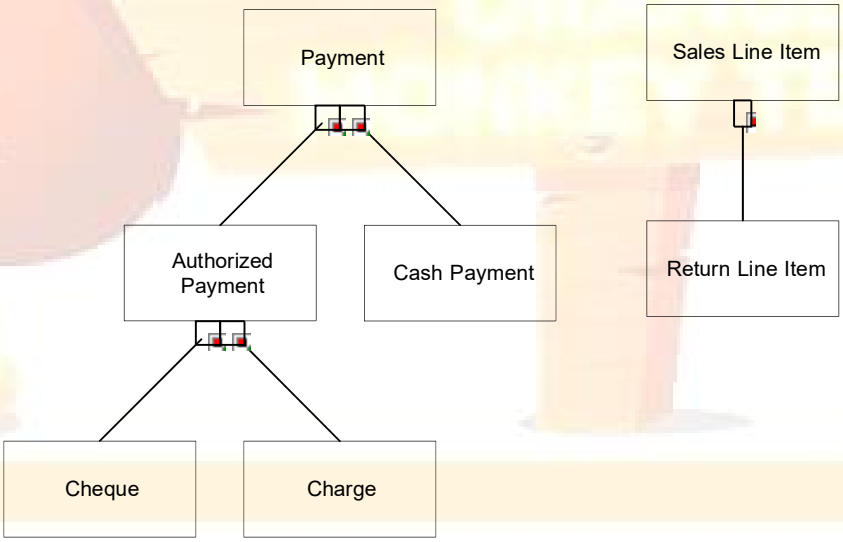
© Copyright Virtual University of Pakistan

Services:
Who I Know?

is it a case for gen-spec, what's same, what's different

payment - we have types of payments
Attributes:
each payment knows about its
amount paid, cash tendered
a check object knows its
bank, account number, amount tendered, authorization code
a credit object knows about its
card type, card number, expiration date, authorization code
common attributes among check and credit - use gen-spec
hierarchy becomes:
payment
cash payment
authorized payment
check
card

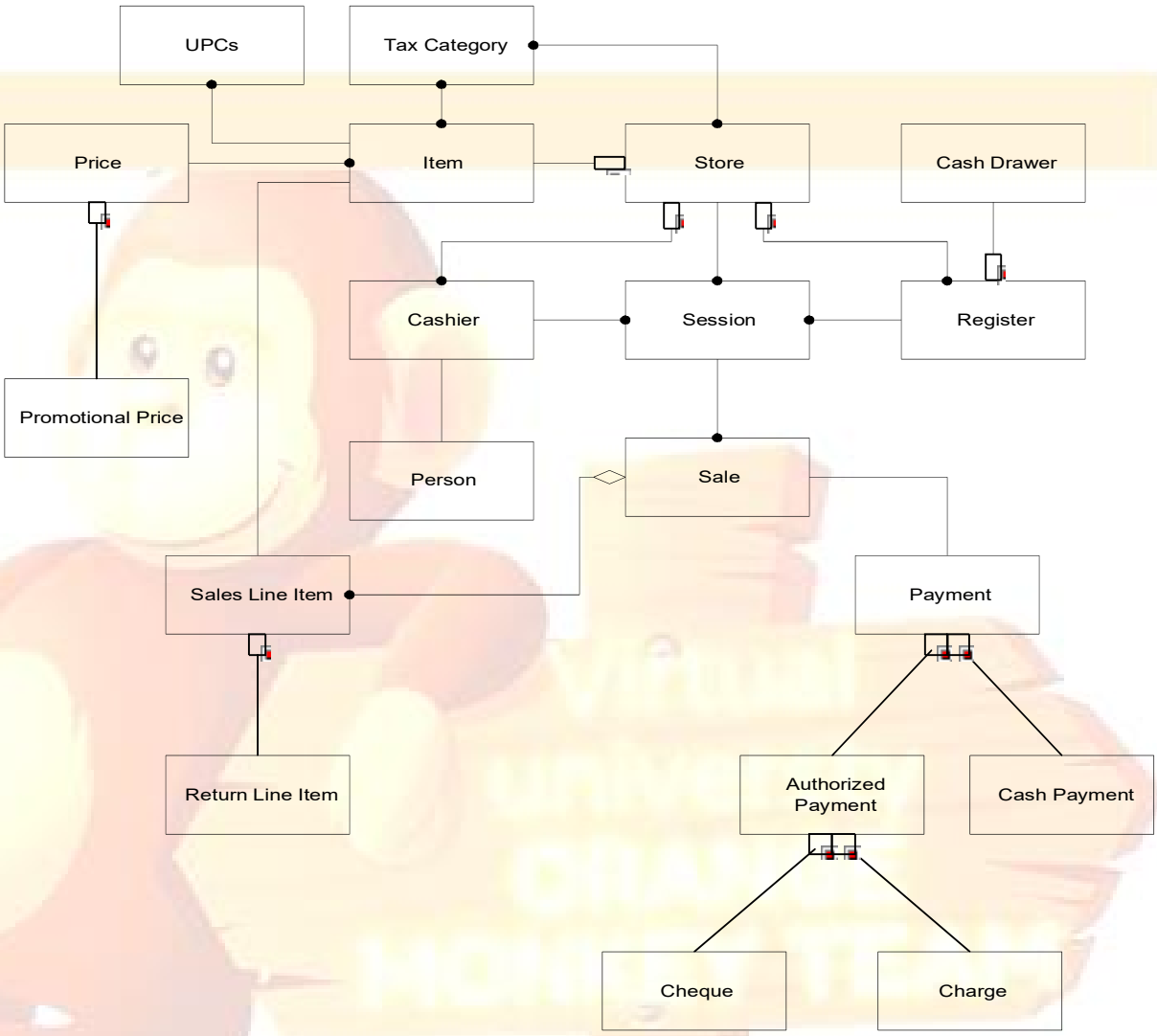
Services:
who I know: sale



session
Attributes: start date, end date, start time, end time
Services: how much money collected over interval, how many sales

© Copyright Virtual University of Pakistan

Who I know? register, cashier, store, sales



Object Model Diagram for Connie's Convenience Store

© Copyright Virtual University of Pakistan

Lecture No. 20

Interaction Diagrams – depicting the dynamic behaviour of the system

A series of diagrams can be used to describe the *dynamic behavior* of an object-oriented system. This is done in terms of a set of messages exchanged among a set of objects within a context to accomplish a purpose. This is often used to model the way a use case is realized through a sequence of messages between objects.

The purpose of Interaction diagrams is to:

- Model interactions between objects
- Assist in understanding how a system (a use case) actually works
- Verify that a use case description can be supported by the existing classes
- Identify responsibilities/operations and assign them to classes

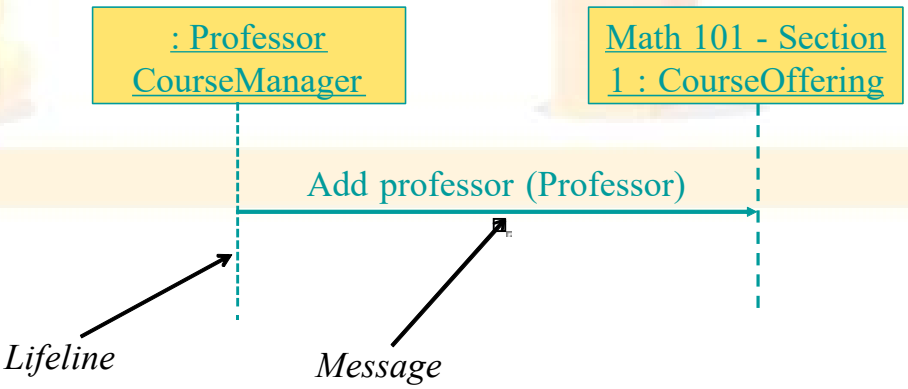
UML provides two different mechanisms to document the dynamic behaviour of the system. These are sequence diagrams which provide a time-based view and *Collaboration Diagrams* which provide an organization-based view of the system's dynamics.

The Sequence Diagram

Let us first look at Sequence Diagrams. These diagrams illustrate how objects interacts with each other and emphasize time ordering of messages by showing object interactions arranged in time sequence. These can be used to model simple sequential flow, branching, iteration, recursion and concurrency. The focus of sequence diagrams is on objects (and classes) and message exchanges among them to carry out the scenarios functionality. The objects are organized in a horizontal line and the events in a vertical time line.

The Notation

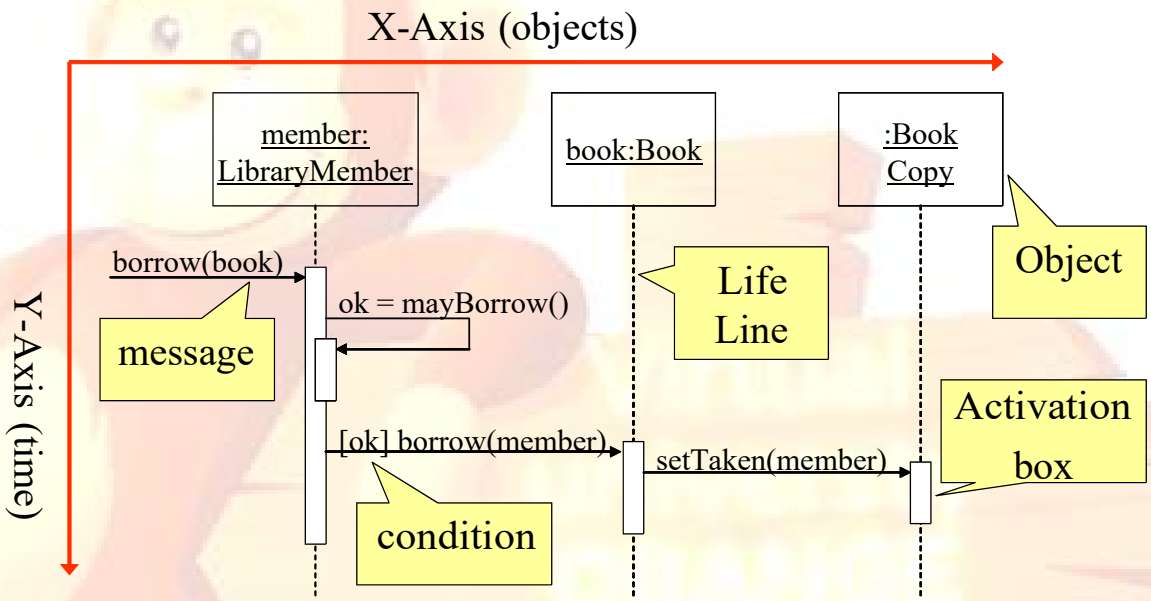
Following diagram illustrates the notation used for drawing sequence diagrams.



© Copyright Virtual University of Pakistan

The boxes denote objects (or classes), the solid lines depict messages being sent from one object to the other in the direction of the arrow, and the dotted lines are called life-lines of objects. The life line represents the object's life during interaction. We will discuss this in more detail later.

These concepts are further elaborated with the help of the following sequence diagram.



As shown above, in a sequence diagram, objects (and classes) are arranged on the X-Axis (horizontally) while time is shown on the Y-Axis (vertically). The boxes on the life-line are called activation boxes and show for how long a particular message will be active, from its start to finish. We can also show if a particular condition needs to occur before a message is invoked simply by putting the condition in a box before the message. For example, object *member:LibraryMember* sends a message to object *book:book* if the value of *ok* is true.

The syntax used for naming objects in a sequence diagram is as follows:

- syntax: [instanceName][:className]
- Name classes consistently with your class diagram (same classes).
- Include instance names when objects are referred to in messages or when several objects of the same type exist in the diagram.

An interaction between two objects is performed as a message sent from one object to another. It is most often implemented by a simple operation call. It can however be an

© Copyright Virtual University of Pakistan

actual message sent through some communication mechanism, either over the network or internally on a computer.

If object obj_1 sends a message to another object obj_2 an association must exist between those two objects. There has to be some kind of structural dependency. It can either be that obj_2 is in the global scope of obj_1, or obj_2 is in the local scope of obj_1 (method argument), or obj_1 and obj_2 are the same object.

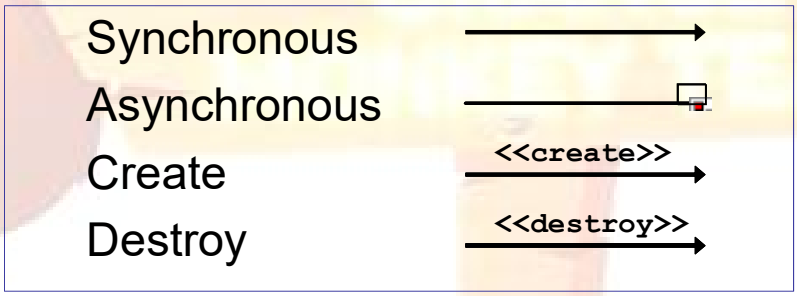
A message is represented by an arrow between the life lines of two objects. Self calls are also allowed. These are the messages that an object sends to itself. This notation allows self calls. In the above example, object *member:LibraryMember* sends itself the *mayBorrow* message. A message is labeled at minimum with the message name.
Arguments and control information (conditions, iteration) may also be included. It is preferred to use a brief textual description whenever an *actor* is the source or the target of a message.

The time required by the receiver object to process the message is denoted by an *activation-box.*

Lecture No. 21

Message Types

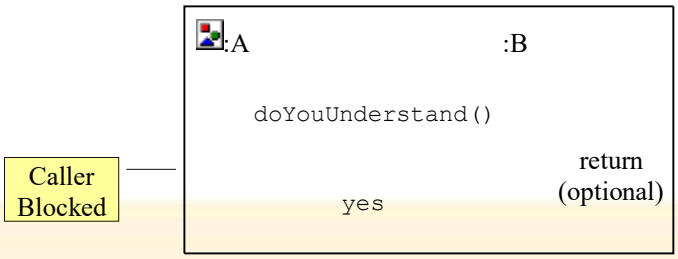
Sequence diagrams can depict many different types of messages. These are: synchronous or simple, asynchronous, create, and destroy. The following diagram shows the notation and types of arrows used for these different message types.



Synchronous Messages

Synchronous messages are "call events" and are denoted by the full arrow. They represent nested flow of control which is typically implemented as an operation call. In case of a synchronous message, the caller waits for the called routine to complete its operation before moving forward. That is, the routine that handles the message is completed before the caller resumes execution. Return values can also be optionally indicated using a dashed arrow with a label indicating the return value. This concept is illustrated with the help of the following diagram.

© Copyright Virtual University of Pakistan



While modeling synchronous messages, the following guidelines should be followed:

- Don't model a return value when it is obvious what is being returned, e.g. getTotal()
- Model a return value only when you need to refer to it elsewhere, e.g. as a parameter passed in another message.
- Prefer modeling return values as part of a method invocation, e.g.
  ok = isValid()

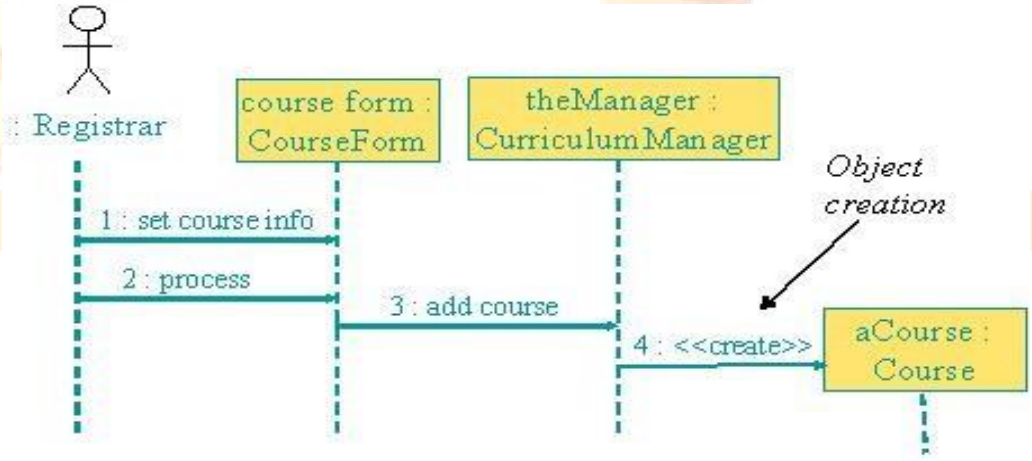
Asynchronous messages

Asynchronous messages are "signals," denoted by a half arrow. They do not block the caller. That is, the caller does not wait for the called routine to finish its operation for continuing its own sequence of activities. This occurs in multi-threaded or multi-processing applications where different execution threads may pass information to one another by sending asynchronous messages to each other. Asynchronous messages typically perform the following actions:

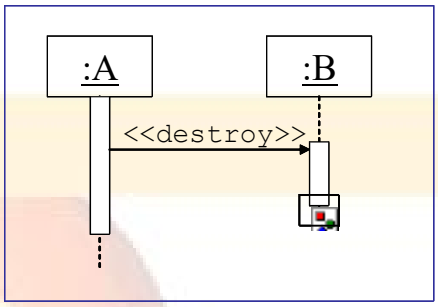
- Create a new thread
- Create a new object
- Communicate with a thread that is already running

Object Creation and Destruction

An object may create another object via a <<create>> message. Similarly an object may destroy another object via a <<destroy>> message. An object may also destroy itself. One should avoid modeling object destruction unless memory management is critical. The following diagrams show object creation and destruction. It is important to note the impact of these activities on respective life lines.



© Copyright Virtual University of Pakistan



Sequence diagrams and logical complexity

It is important to judiciously use the sequence diagrams where they actually add value. The golden principle is to keep it small and simple. It is important to understand that the diagrams are meant to make things clear. Therefore, in order to keep them simple, special attentions should be paid to the conditional logic. If it is simple then there is no harm in adding it to the diagram. On the other hand if the logic is complex then we should draw separate diagrams like flow charts.

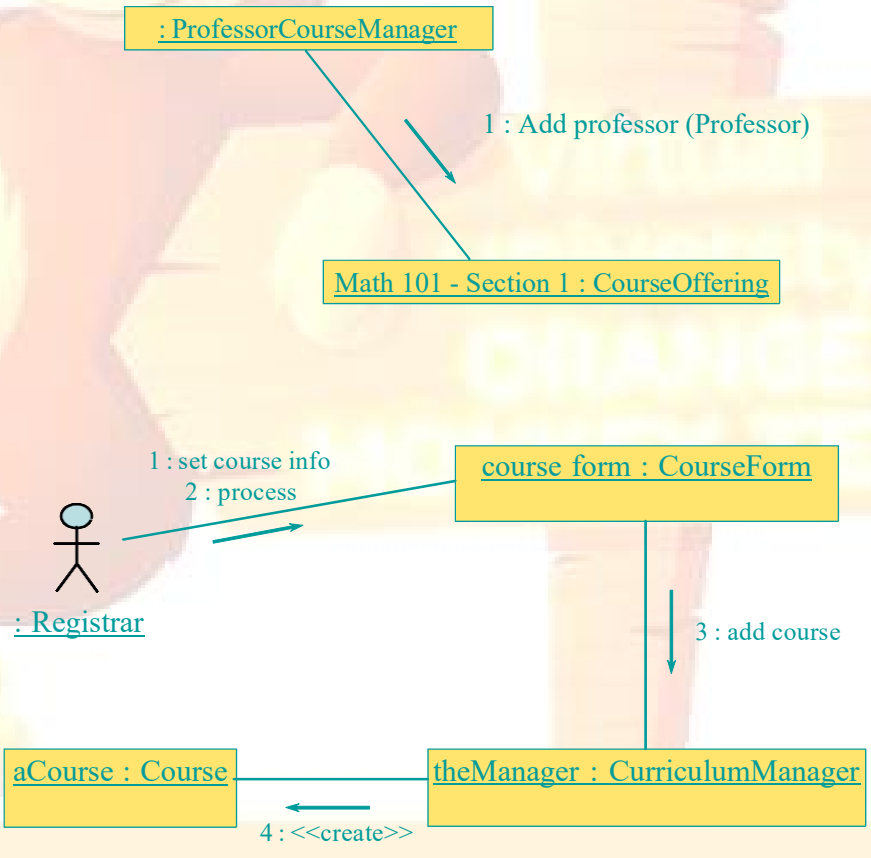
© Copyright Virtual University of Pakistan

Collaboration diagrams

Collaboration diagrams can also be used to depict the dynamic behaviour of a system. They show how objects interact with respect to organizational units (boundaries!). Since a boundary shapes communication between system and outside world e.g. user interface or other system, collaboration diagrams can be used to show this aspect of the system. The sequence of messages determined by numbering such as 1, 2, 3, 4, … This shows which operation calls which other operation.

Collaboration diagrams have basically two types of components: objects and messages. Objects exchange messages among each-other. Collaboration diagrams can also show synchronous, asynchronous, create, and destroy message using the same notation as used in sequence diagrams. Messages are numbered and can have loops

The following diagrams illustrate the use of collaboration diagrams.



Comparing sequence & collaboration diagrams

© Copyright Virtual University of Pakistan

Sequence diagrams are best to see the flow of time. On the other hand, static object connections are best represented by collaboration diagrams. Sequence of messages is more difficult to understand in collaboration diagrams than in the case of sequence diagrams. On the other hand, object organization with control flow is best seen through collaboration diagrams. It may be noted that complex control is difficult to express anyway but collaboration diagrams can become very complex very quickly.

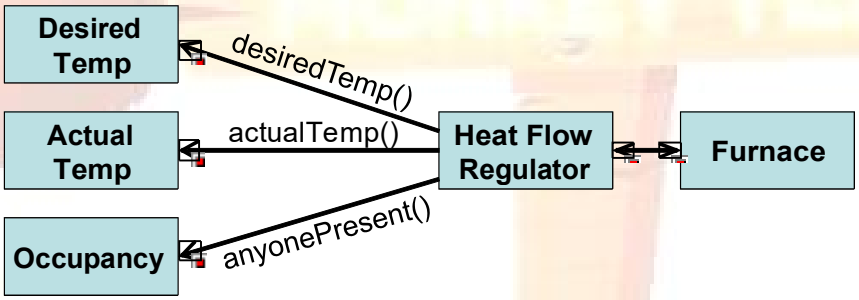
Evaluating the Quality of an Object-Oriented Design

Judging the quality of a design is difficult. We can however look at certain object-oriented design attributes to estimate its quality. The idea is to analyze the basic principle of encapsulation and delegation to judge whether the control is centralized or distributed, hence judging the coupling and cohesion in a design. This will tell us how maintainable a design is.

You may also recall our earlier discussion of coupling and cohesion. It can be easy to see that OO design yield more cohesive and loosely coupled systems.

The issue of centralized versus distributed control can be illustrated with the help of the following example.

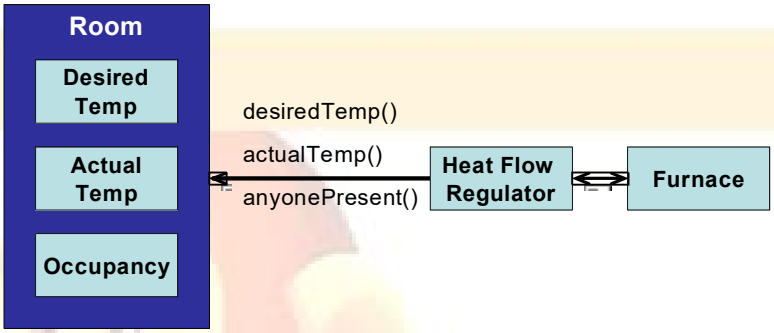
Consider a heat regulatory system for a room as shown below.



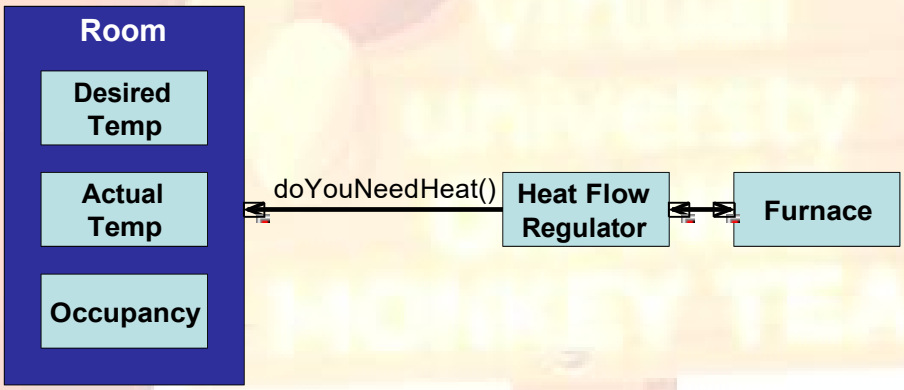
In this case, the room is not encapsulated as one entity and three different objects namely *Desired Temp, Actual Temp,* and *Occupancy* maintain necessary information about a room. In this case the Heat Flow Regulator has to communicate with three different objects.

© Copyright Virtual University of Pakistan

If we encapsulate the three objects into one Room object as shown below, then the Heat Flow Regulator will need to communicate with one object, hence the overall coupling of the system will be reduced.



This happened because in the first case intelligence is distributed while in the second case it is encapsulated. However, the control is still centralized as the Heat Flow Regulator has the control logic that first analyses the values from different queries and then makes a decision about turning the furnace on of off. We can improve the situation even further by delegating this responsibility to the Room object as shown below.



By doing that we reduce coupling even further because now we have made Room more cohesive by putting the function with the related data and have thus reduced the number and types of messages being sent from the regulator to the room.

That is, we can reduce the coupling of a system by minimizing the number of messages in the protocol of a class. The problem with large public interfaces is that you can never find what you are looking for…smaller public interfaces make a class easier to understand and modify. This can be further elaborated with the help of the following example. Suppose we have two functions defined for setting the desired temperature in the room:

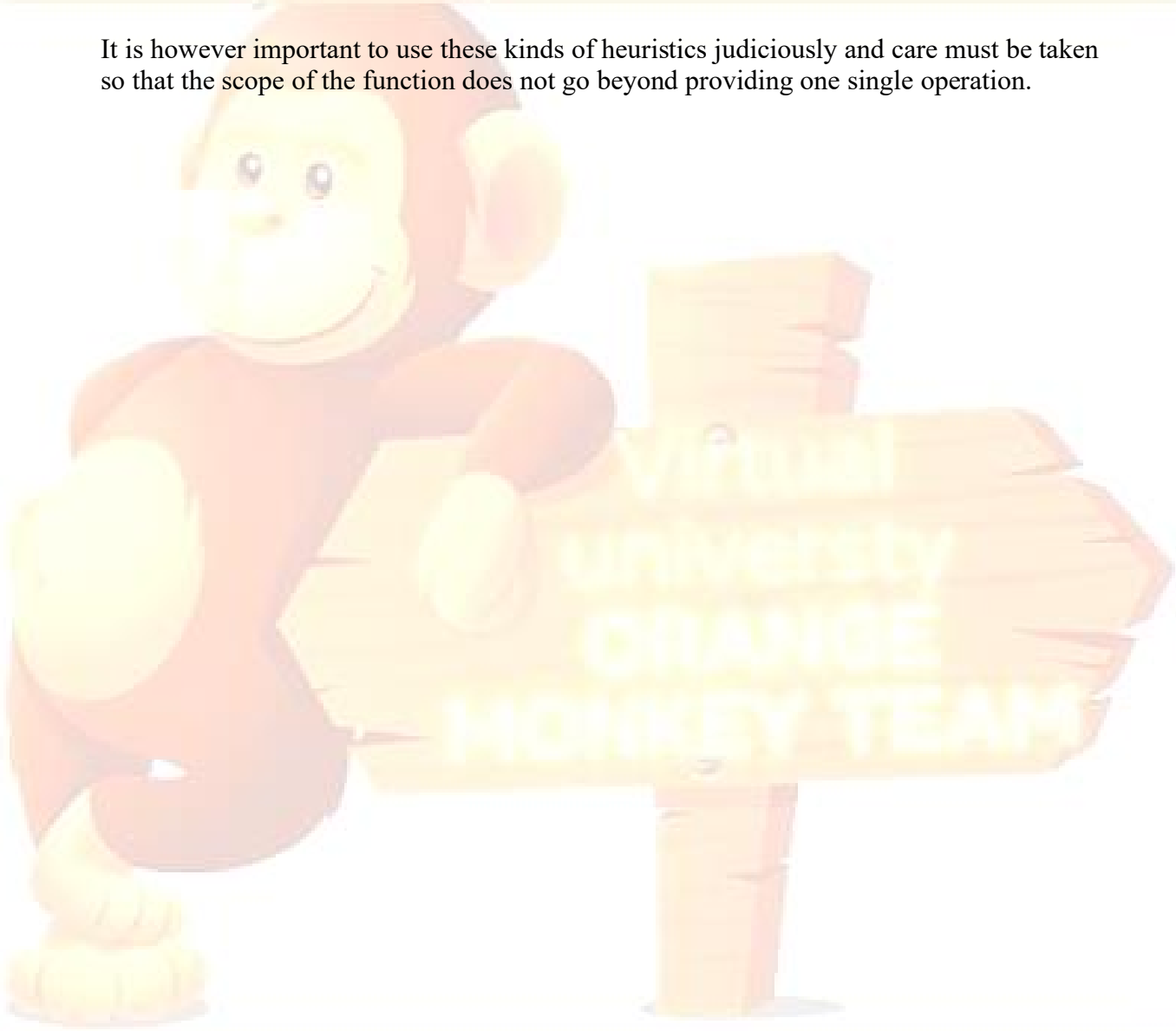
© Copyright Virtual University of Pakistan

- SetMinimumValue(int aValue)
- SetMaximimumValue(int aValue)

We can reduce the total number of messages in the protocol of this class by consolidation these as shown below, hence reducing the overall complexity of the protocol.

- SetLimits(int minValue, int maxValue)

It is however important to use these kinds of heuristics judiciously and care must be taken so that the scope of the function does not go beyond providing one single operation.



© Copyright Virtual University of Pakistan

Lecture No. 22

Software and System Architecture

Introduction

When building a house, the architect, the general contractor, the electrician, the plumber, the interior designer, and the landscaper all have different views of the structure. Although these views are pictured differently, all are inherently related: together, they describe the building's architecture. The same is true with software architecture. Architectural design basically establishes the overall structure of a software system.

The design process for identifying the sub-systems making up a system and the framework for sub-system control and communication is *architectural design.* The output of this design process is a description of the *software architecture.*

The study of software architecture is in large part a study of software structure that began in 1968 when Edsger Dijkstra pointed out that it pays to be concerned with how software is partitioned and structured, as opposed to simply programming so as to produce a correct result. Dijkstra was writing about an operating system, and first put forth the notion of a layered structure, in which programs were grouped into layers, and programs in one layer could only communicate with programs in adjoining layers. Dijkstra pointed out the elegant conceptual integrity exhibited by such an organization, with the resulting gains in development and maintenance ease.

David Parnas pressed this line of observation with his contributions concerning information-hiding modules, software structures, and program families.

A program family is a set of programs (not all of which necessarily have been or will ever be constructed) for which it is profitable or useful to consider as a group. This avoids ambiguous concepts such as "similar functionality" that sometimes arise when describing domains. For example, software engineering environments and video games are not usually considered to be in the same domain, although they might be considered members of the same program family in a discussion about tools that help build graphical user interfaces, which both happen to use.[1]

Parnas argued that early design decisions should be ones that will most likely remain constant across members of the program family that one may reasonably expect to

© Copyright Virtual University of Pakistan

produce. In the context of this discussion, an early design decision is the adoption of a particular architecture. Late design decisions should represent trivially-changeable decisions, such as the values of compile-time or even load-time constants.

All of the work in the field of software architecture may be seen as evolving towards a paradigm of software development based on principles of architecture, and for exactly the same reasons given by Dijkstra and Parnas: Structure is important, and getting the structure right carries benefits.

Before talking about the software architecture in detail, let us first look at a few of its definitions.

What is Software Architecture?

What do we mean by software architecture? Unfortunately, there is yet no single universally accepted definition. Nor is there a shortage of proposed definition candidates. The term is interpreted and defined in many different ways. At the essence of all the discussion about software architecture, however, is a focus on reasoning about the *structural* issues of a system. And although architecture is sometimes used to mean a certain architectural style, such as client-server, and sometimes used to refer to a field of study, it is most often used to describe structural aspects of a particular system.

Before looking at the definitions for the software architecture, it is important to understand how a software system is defined. It is important because many definitions of software architecture make a reference to software systems.

According to UML 1.3, a system is a collection of connected units that are organized to accomplish a specific purpose. A system can be described by one or more models, possibly from different viewpoints. IEEE Std. 610.12-1990 defines a system as a collection of components organized to accomplish a specific function or set of functions. That is, a system is defined as an organized set of connected components to accomplish the specified tasks.

Let us now look at some of the software architecture definitions from some of the most influential writers and groups in the field.

Software Architecture Definitions

UML 1.3:

> Architecture is the organizational structure of a system. An architecture can be recursively decomposed into parts that interact through interfaces, relationships that connect parts, and constraints for assembling parts. Parts that interact through interfaces include classes, components and subsystems.

Bass, Clements, and Kazman. Software Architecture in Practice, Addison-Wesley 1997:

© Copyright Virtual University of Pakistan

'The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them.

By "externally visible" properties, we are referring to those assumptions other components can make of a component, such as its provided services, performance characteristics, fault handling, shared resource usage, and so on. The intent of this definition is that a software architecture must abstract away some information from the system (otherwise there is no point looking at the architecture, we are simply viewing the entire system) and yet provide enough information to be a basis for analysis, decision making, and hence risk reduction."

Garlan and Perry, guest editorial to the *IEEE Transactions on Software Engineering*, April 1995:

Software architecture is "the structure of the components of a program/system, their interrelationships, and principles and guidelines governing their design and evolution over time."

IEEE Glossary

Architectural design: The process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system.

Shaw and Garlan

The architecture of a system defines that system in terms of computational components and interactions among those components. Components are such things as clients and servers, databases, filters, and layers in a hierarchical system. Interactions among components at this level of design can be simple and familiar, such as procedure call and shared variable access. But they can also be complex and semantically rich, such as client-server protocols, database accessing protocols, asynchronous event multicast, and piped streams.

Each of these definitions of software architecture, though seemingly different, emphasizes certain structural issues and corresponding ways to describe them. It is important to understand that although they are apparently different, they do not conflict with one another.

One can thus conclude from these definitions that an architectural design is an early stage of the system design process. It represents the link between specification and design processes. It provides an overall abstract view of the solution of a problem by identifying the critical issues and explicitly documenting the design choices made under the specified constraints as its primary goal is to define the non-functional requirements of a system and define the environment. It is often carried out in parallel with some specification

© Copyright Virtual University of Pakistan

activities and It includes the high-level design of modular components, their relationships and organization, and provides a foundation that one can build on to solve a problem.

Why is architecture important?

Barry Boehm says:

> *If a project has not achieved a system architecture, including its rationale, the project should not proceed to full-scale system development. Specifying the architecture as a deliverable enables its use throughout the development and maintenance process.*

Why is architecture important and why is it worthwhile to invest in the development of a architecture? Fundamentally, there are three reasons:

1. **Mutual communication.** Software architecture represents a common high-level abstraction of the system that most, if not all, of the system's stakeholders can use as a basis for creating mutual understanding, forming consensus, and communicating with each other.

   Each stakeholder of a software system (customer, user, project manager, coder, tester, and so on) is concerned with different characteristics of the system that are affected by its architecture. Architecture provides a common language in which different concerns can be expressed, negotiated, and resolved at a level that is intellectually manageable, even for large, complex systems. Without such language, it is difficult to understand large systems sufficiently to make the early decisions that influence both quality and usefulness.

2. **Early design decisions.** Software architecture represents the embodiment of the earliest set of design decisions about a system, and these early bindings carry weight far out of proportion to their individual gravity with respect to the system's remaining development, its service in deployment, and its maintenance life. It is also the earliest point at which the system to be built can be analyzed.

   An implementation exhibits an architecture if it conforms to the structural design decisions described by the architecture. This means that the implementation must be divided into the prescribed components, the components must interact with each other in the prescribed fashion, and each component must fulfill its responsibility to the other components dictated by the architecture.

   Resource allocation decisions also constrain implementation. These decisions may be invisible to implementers working on individual components. The constraints permit a separation of concerns that allows management decisions that make best use of personnel and computational capacity. Component builders must be fluent in the specification of their individual components but not in architectural trade-offs. Conversely, the architects need not be experts in all

© Copyright Virtual University of Pakistan

aspects of algorithm design or the intricacies of the programming language, but they are the ones responsible for architectural trade-offs.

Not only does architecture prescribe the structure of the system being developed, but it also engraves that structure on the structure of the development project. The normal method of dividing up the labor in a large system is to assign different portions of the system to different groups to construct. This is called the work breakdown structure of a system. Because the system architecture includes the highest level decomposition of the system, it is typically used as the basis for the work breakdown structure. Specifically, the module structure is most often the basis for work assignments. The work breakdown structure, in turn, dictates units of planning, scheduling, and budget, as well as inter-team communications channels, configuration control and file system organization, integration and test plans and procedures. Teams communicate with each other in terms of the interface specifications to the major components. The maintenance activity, when launched, will also reflect the software structure, with teams formed to maintain specific structural components.

A side effect of establishing the work breakdown structure is to effectively freeze the software architecture, at least at the level reflected in the work breakdown. A group that is responsible for one of the subsystems will resist having its responsibilities distributed across other groups. If these responsibilities have been formalized in a contractual relationship, changing responsibilities could become expensive. Tracking progress on a collection of tasks that is being distributed would also become much more difficult.

Thus, once the architecture's module structure has been agreed on, it becomes almost impossible for managerial and business reasons to modify it. This is one argument (among many) for carrying out extensive analysis before freezing the software architecture for a large system.

Is it possible to tell that the appropriate architectural decisions have been made (i.e., if the system will exhibit its required quality attributes) without waiting until the system is developed and deployed? If the answer were "no," choosing an architecture would be a hopeless task: random architecture selection would perform as well as any other method. Fortunately, it is possible to make quality predictions about a system based solely on an evaluation of its architecture.

3. **Reusable abstraction of a system.** Software architecture embodies a relatively small, intellectually graspable model for how the system is structured and how its components work together; this model is transferable across systems; in particular, it can be applied to other systems exhibiting similar requirements, and can promote large scale reuse.

   In an architecture-based development of a product line, the architecture is in fact the sum of the early design decisions. System architects choose an architecture

© Copyright Virtual University of Pakistan

(or a family of closely related architectures) that will serve all envisioned members of the product line by making design decisions that apply across the family early and by making other decisions that apply only to individual members late. The architecture defines what is fixed for all members of the family and what is variable.

A family-wide design solution may not be optimal for all systems derived from it, but the quality gained and labor savings realized through architectural-level reuse may compensate for the loss of optimality in particular areas. On the other hand, reusing a family-wide design reduces the risk that a derived system might have an inappropriate architecture. Using a standard design reduces both risk and development costs, at the risk of non-optimality.

Architectural Attributes

Software architecture must address the non-functional as well as the functional requirements of the software system. This includes performance, security, safety, availability, and maintainability. Following are some of the architectural design guidelines that can help in addressing these challenges.

- Performance
  - Performance can be enhanced by localising operations to minimise sub-system communication. That is, try to have self-contained modules as much as possible so that inter-module communication is minimized.
- Security
  - Security can be improved by using a layered architecture with critical assets put in inner layers.
- Safety
  - Safety-critical components should be isolated
- Availability
  - Availability can be ensured by building redundancy in the system and having redundant components in the architecture.
- Maintainability
  - Maintainability is directly related with simplicity. Therefore, maintainability can be increased by using fine-grain, self-contained components.

Architectural design process

Just like any other design activity, design of software architecture is a creative and iterative process. This involves performing a number of activities, not necessarily in any particular order or sequence. These include system structuring, control modeling, and modular decomposition. These are elaborated in the following paragraphs.

- System structuring

© Copyright Virtual University of Pakistan

System structuring is concerned with decomposing the system into interacting sub-systems. The system is decomposed into several principal sub-systems and communications between these sub-systems are identified. The architectural design is normally expressed as a block diagram presenting an overview of the system structure. More specific models showing how sub-systems share data, are distributed and interface with each other may also be developed. A sub-system is a system in its own right whose operation is independent of the services provided by other sub-systems. A module is a system component that provides services to other components but would not normally be considered as a separate system.

- Control modelling

  Control modelling establishes a model of the control relationships between the different parts of the system.

- Modular decomposition

  During this activity, the identified sub-systems are decomposed into modules.

This design process is further elaborated in the following section where architectural views are discussed.

© Copyright Virtual University of Pakistan

© Copyright Virtual University of Pakistan

CS504 GRAND QUIZ
MCS FAMILY GROUP TEAM
BY: ARSAL SHANI & AINA MALIK

1) Modules with high cohesion and low coupling can be treated and analysed as
   i. White boxes
   ii. black boxes
   iii. grey boxes
   iv. none of these
2) While establishing the services for an object, the following fundamental questions should be asked:
   i) Why does the system need this object anyway?
   ii) What useful questions can it answer?
   iii) What useful action can it perform?
   iv) All of the given options.
3) ---------- is a role that each actor plays in the system under consideration.
   i) An act
   ii) A participant
   iii) A function
   iv) None of the given
4) Any Engineering approach must be founded on organizational commitment to ----------.
   i. Cost
   ii. Scheduling
   iii. Quality
   iv. Performance
5) Return values in synchronous massages are:
   i) Compulsory
   ii) May not used when response is obvious
   iii) Not used at all
   iv) Represented by solid lines
6) According to Caper Jhones analysis of project activities, coding only has ------- affect part in system development.
   i. 13-14%
   ii. 36-40%
   iii. 50-60%
   iv. 70-80%
7) Normally a system is more easy to modify if its modules have
   i. High coupling and high cohesion
   ii. High coupling and low cohesion
   iii. Low coupling and high cohesion
   iv. Low coupling and Low cohesion
8) In multi-threaded or multiprocessing applications where different execution threads may pass information to one another by sending ----------to each other.

Arsal Shani & Aina Malik (MCS Family Group Team)

i. Interrupt calls
ii. Synchronous messages
iii. Asynchronous messages
iv. System Calls

9) Which of the following is not among the four layers of the object-oriented pyramid?
i. The subsystem layers
ii. The class and object layer
iii. The abstract layer
iv. The message layers

10) System models include:
i. User business processes
ii. User activities for conducting the business process
iii. Processes that need to be automated
iv. All of the given options

11) In the architecture trade-off analysis method the architectural style should be described using the
i. Data flow view
ii. Module view
iii. Process view
iv. All of the given

12) ---------- is concerned with decomposing the system into interacting sub-systems.
a) System structuring
b) Control Modelling
c) Molecular Decomposition
d) None of the given

13) A use case represents:
i. A class, its attributes and operations
ii. An operation's interface and signature
iii. The role a user plays when interacting with the system
iv. The system's functionality for a particular purpose

14) External entity may be:
i. Source of input data only
ii. Source of input data and destination of results
iii. Destination of results
iv. Repository of data

15) The process of utilizing our knowledge of computer science in effective production of ----------.
i. Chemical Engineering
ii. Electrical Engineering
iii. Computer Engineering
iv. System Engineering

16) Coupling is a measure of ---------- of a component.
i. Independence
ii. Dependence
iii. Aggregation
iv. Composition

17) ---------- has become a standard notation for object-oriented system modelling:

Arsal Shah & Ali Malik (MCS Family Group Team)

i. UML
ii. C++
iii. OCL(Object Constraint Language)
iv. None of the given option

18) An arrow in data flow diagram represents:
i. Direction of flow of data
ii. Processing of data
iii. External agent
iv. Internal Agent

19) ---------- diagrams does not capture control flow information, it just shows the flow of data in a system.
i. Sequence
ii. Data Flow
iii. Activity
iv. Class

20) In ---------- the analyst determines the source of requirements and where do these requirements consume:
a) Data flow analysis
b) Source and sink analysis
c) Down parsing
d) Up parsing

21) Data cannot flow from one external entity to other external entity because:
a) It will get corrupted
b) It is not allowed in DFD
c) An external entity has no mechanism to read or write
d) Both are outside the context of the system

22) In the functional design, the structure of the system resolves around:
a) Objects
b) Properties
c) Functions
d) All of the given options

23) ---------- is one of the techniques to document domain knowledge
a) State transition diagram
b) Feasibility matrix
c) System matrix
d) None of the given options

24) In case of ---------- approach , decomposition of a problem revolves around data.
a) Object-Oriented
b) Action-Oriented
c) Event-Oriented
d) Process-Oriented

25) The ---------- relationship is a kind of a generalization specialization relationship:
a) Bit-Byte
b) Uses
c) Binary
d) Extends

26) Which statement is not true about system software?

Faisal Shah & Aina Malik (MCS Family Group Team)

a) Change request has direct impact on software
b) Passes through a constant process of evolution
c) Change requests have direct impact in the form of defects
d) None of the given

27) Strong cohesion implies that:
a) All parts of a component have a close logical relationship with each other
b) All parts of a component don't have a close relationship with each other
c) Component is dynamic in nature
d) Component is static in nature

28) The intent of Object-Oriented Analysis(OOA) is to define:
a) All classes
b) Relationships among classes
c) Behaviour of classes
d) All of the given options

29) Requirement engineering focuses on ---------- aspect of the software development process.
a) Both what and how
b) What
c) How
d) Why and how

30) ---------- relationship is concerned with classes, not with class instantiates.
a) Association
b) Inheritance
c) Aggregation
d) Composition

31) Which of the following statements are true in context of the object model deviation through the Coad methodology?
a) A place is also a contains
b) Every container needs to be a place
c) Same person may play different times in the system.
i. A only
ii. A and b
iii. A and c
iv. All of the given

32) The goal of --------- is to translate the customer's desire for a set of defined capabilities into a working product.
i. Electrical engineering
ii. Product engineering
iii. Hardware engineering
iv. Mechanical engineering

33) In case of a ------- message, the called routine that handles the message is completed before the caller resumes execution.
a) Synchronous
b) Asynchronous
c) Bidirectional
i. A only
ii. B only

Arsal Shah & Alina Malik (MCS Family Group Team)

iii. C only
iv. All of the given

34) In multiprocessing applications, different execution threads may pass information to one another by sending ------- to each other.
i. Interrupt calls
ii. Synchronous messages
iii. Asynchronous messages
iv. System calls

35) A car is made up of a body, three or four wheels, a steering mechanism, a breaking mechanism, and a power engine"
The above statement is example of:
i. Whole-part relationship
ii. Inheritance
iii. Specialization
iv. Generalization

36) To help separate an object's external behaviour from its implementation, the technique used is called -------
i. Generalization
ii. Association
iii. Composition
iv. Abstraction

37) Sequences of messages can be present in:
(a) Use case diagram
(b) Sequence diagram
(c) Collaboration diagram
1) a only
2) b only
3) c only
4) b and c

38) Which of the following strategies lead to good software design:
i. Separation of concerns
ii. Modularity
iii. Divide-and-conquer
iv. All of the given options

39) Data flow model:
i. Captures the flow of data in a system
ii. Helps in developing an understanding of system's functionality
iii. Describes data origination, transformations and consumption in a system
iv. All of the given options

40) ------- requirements are often called product features.
i. Functional
ii. Non-functional
iii. Developer
iv. User

41) In the functional design, the structure of the system revolves around.
i. Objects
ii. Properties

Arsal Sunni & Ana Malik (MCS Family Group Team)

iii. Functions
iv. All of the given option

42) The first step in any OOA process model is to
i. Build an object-relationship model.
ii. Define collaborations between objects.
iii. Elicit customer requirements
iv. Select a representation language

43) The ------ relationship is kind of a generalization specialization relationship.
i. Bit-byte
ii. Uses
iii. Binary
iv. Extends

44) Regarding data flow model, which of the following statement(s) is true:
i. It captures the transformation of data between processes/functions of a system
ii. Processes on a data flow can operate in parallel
iii. Only those processes are represented which we need to automate
iv. All of the given option

45) ---------- is an extremely powerful technique for dealing with complexity.
i. Aggregation
ii. Abstraction
iii. Inheritance
iv. Association

46) In “point of sale system”. the term “payment” represents
i. Actor
ii. Participant
iii. Transaction
iv. Container

47) The architecture components for product engineering are
i. Data, hardware, software, people
ii. Data, documentation, hardware, software
iii. Data, hardware, software, procedures
iv. Documentation, hardware, people, procedures

48) An object model encompasses the principle(s) of
i. Abstraction
ii. Encapsulation
iii. Hierarchy or inheritance
iv. All of the given option

49) Prototyping is used when there is ---------- regarding requirements.
i. Uncertainty
ii. Confirmation
iii. Conflict
iv. Consensus

50) In ------- phase of software development, requirement analyst focuses on possible design of the proposed solution.
i. Maintenance
ii. Development
iii. Definition

Asad Gani & Ali Malik (MCS Family Group Team)

iv. Vision

51) At which stage of software development loop, results are delivered?
   i. Problem definition
   ii. Solution integration
   iii. Technical development
   iv. Status quo

52) A class will be cohesive if:
   i. Class does not implement complex interfaces
   ii. Class does not have complex methods
   iii. If most of the methods do not use most of the data members most of the time
   iv. If most of the methods use most of the data members most of the time.

53) A DFD is normally levelled (adding more levels of abstraction) as
   i. It is a good idea in design
   ii. It is recommended by many experts
   iii. It is easy to do it
   iv. It is easier to read and understand a number of smaller DFDs than one large DFD

54) Identify the true statement(s)
   i. An attribute that may have a number of values should be replaced by a new class and an object connection
   ii. An attribute that varies over time, e.g. price of an item, should be replaced by an additional class with an affective data and value
   iii. Replace “yes/no” type attribute with “status” type attributes for flexibility
   iv. All of given option

55) ---------- Is a technique in which we construct a model of an entity based upon its essential characteristics and ignore the inessential details.
   i. Inheritance
   ii. Polymorphism
   iii. Aggregation
   iv. Abstraction

56) A structure is a manner of an organization which expresses a ------ strong organization within the problem domain.
   i. Semantically
   ii. Syntactically
   iii. Graphically
   iv. None of the given

57) An object model of a system captures the ------ structure of a system.
   i. Static
   ii. Dynamic
   iii. Iterative
   iv. Hierarchical

58) To determine the architectural style or combination of styles that best fits the proposed system, requirements engineering is used to uncover
   i. Algorithmic complexity
   ii. Characteristics and constraints
   iii. Control and data
   iv. Design patterns

Arsal Gani & Aina Malik (MCS Family Group Team)

59) Which statement is not according to the software engineering principles? Software engineering is a(n) _____
   i. Balancing act
   ii. Disciplined approach
   iii. Unsystematic approach
   iv. Quantifiable approach
60) In abbot's textual analysis technique, different part of speech is identified within the text of the specification and these part are modelled using different _____
   i. Event
   ii. Process
   iii. Operations
   iv. Components
61) Quantitative methods for assessing the quality of proposed architectural designs are readily available.
   i. True
   ii. False
62) In order to determine the role and responsibilities of the identified objects, we need to consider which of the following step(s):
   a) Who I am?
   b) What I know?
   c) Who I know?
   d) What I do?
   i. A only
   ii. A and b
   iii. B ,c and D
   iv. All of the given
63) In object oriented design _____ layer contains the details that enable each object to communicates with its collaborators.
   i. Subsystem
   ii. Responsibility
   iii. Message
   iv. Object
64) In sequence diagram, the boxes denote:
   i. Objects (or classes)
   ii. Messages, sent from one object to other
   iii. Life-time of objects
   iv. None of the given option
65) In "railway tickit reservation system" the roles such as enquiry. Reservation and ticketing and cancellation are to be performed by the user called:
   i. Passenger
   ii. System analyst
   iii. System developer
   iv. System designer
66) Class responsibilities are defined by _____
   i. Its attributes only
   ii. Its collaborators
   iii. Its operations only

Arsal Sunni & Aina Malik (MCS Family Group Team)

iv. Both its attributes and operations

67) Requirement engineering mainly deals with the _____ of the system
   i. Vision phase
   ii. Definition phase
   iii. Development phase
   iv. Maintenance phase

68) In UML based object oriented model of a system, a composition relation between two objects is shown by a _____ sign on the whole side of a relation line.
   i. An unfilled diamond
   ii. A filled diamond
   iii. A half diamond
   iv. A dot

69) _____ analysis educates the analyst on business domain complexity and shows a way to deal with it.
   i. Domain
   ii. Use case
   iii. Object collaboration
   iv. None of the given options

70) In this case of _____ intra component linkages are stronger while inter component linkages are weak.
   i. High cohesion
   ii. Low coupling
   iii. Low cohesion
   iv. High coupling

71) An architectural style encompasses which of the following elements?
   i. Constraints
   ii. Set of components
   iii. Semantic models
   iv. All of the given

72) In software engineering paradigm, any engineering approach must be founded on organizational commitment to _____
   i. Cost
   ii. Scheduling
   iii. Quality
   iv. Performance

73) Identify the true statement:
   i. Normally object oriented design is more maintainable than functional oriented.
   ii. Software with functional oriented design does not fulfil non functional requirements.
   iii. Object oriented design can not implement “separation of concerns” strategy
   iv. Function oriented design does not lead to an efficient product

74) A process in data flow diagram (DFD) represents
   i. Flow of data
   ii. Transformation of data
   iii. Storage of data
   iv. An external agent

75) A maintainable design is a design , which supports
   i. Change

Arsal Sajid & Aina Malik (MCS Family Group Team)

ii. Debugging
iii. Adding new features
iv. All of the given

76) Whole part structure is also called _____
i. Generalization
ii. Aggregation
iii. Specialization
iv. Association

77) _____ are kind of umbrella activities that are used to smoothly and successfully perform the construction activities.
i. Design activities
ii. Management activities
iii. Testing activities
iv. Maintenance activities

78) When you encounter both transform flow in the same DFD the flow is partitioned and the appropriate mapping technique is used on each part of the DFD.
i. True
ii. False

79) Software architecture must address ---------- requirements of a software system.
i. Functional
ii. Non-functional
iii. User Interface Requirements
iv. Both functional and non-functional.

80) To construct a system model the engineer should consider one of the following restraining factors.
i. Assumptions and constraints
ii. Budget and expenses
iii. Data objects and operations
iv. Schedule and milestones

81) A cohesive class is one which emphasizes on ---------- unit of functionality.
i. Single
ii. Multiple
iii. Static
iv. None

82) The best way to conduct a requirement validation review is to
i. Examine the system model for errors
ii. have the customer look over the requirements
iii. Send them to the design team and see if they have any concerns
iv. Use a checklist of the questions to examine each requirement

83) Defining the services of an object means:
i. What it does?
ii. What it knows?
iii. Who knows it?

84) Which one of the following is the external quality of a software product?
i. Correctness
ii. Concision
iii. Cohesion

Arsal Ghani & Anna Malik (MCS Family Group Team)

iv. Low coupling

85) In Data Flow Diagram, the entity or system, outside the boundary of this system is called:
   i. Process
   ii. Data flow
   iii. External agent
   iv. Data store

86) GUI stands for:
   i. Genaric user Interface
   ii. Graphical user interface
   iii. Genaric user interaction
   iv. Graphical user interaction

87) Specialization means:
   i. Calling the same method with the object of child object
   ii. Hiding the data
   iii. Creating new subclasses for an existing class
   iv. None of the given options

88) In a use case diagram, an ellipse signifies a(n):
   i. Actor
   ii. Class
   iii. Use case
   iv. System boundary

89) Software development is a step-by-step process, and in --------- phase of software development Business objective of an organization get cleared
   i. Maintenance
   ii. Development
   iii. Definition
   iv. Vision

90) If you try to make software more user-friendly then the --------- may suffer.
   i. Reliability
   ii. Software
   iii. Efficiency
   iv. Cost

91) In object-oriented design, the structure of the system revolves around.
   i. Objects
   ii. Properties
   iii. Methods
   iv. All of the given option

92) In -------- relationship, a class shares the structure and behaviour defined in another class:
   i. Aggregation
   ii. Composition
   iii. Inheritance
   iv. Uses

93) In Object Oriented Design, combining the services offered by an object with the attributes they work on, results in:
   (a) Lower coupling and strong cohesion
   (b) Lower cohesion and strong coupling
   (c) Increased likelihood of reuse

Arsal Sami & Aina Malik (MCS Family Group Team)

(d) Decrease the modularity of the system

i. A only
ii. B and c only
iii. A and c only

94) A change becomes ---------- because of close presence of data and functions.

i. Accessible
ii. Global
iii. Private
iv. Localized

95) Software engineering is a ---------- approach.

i. Systematic
ii. Disciplined
iii. Scheduled
iv. All of the given options

96) An external entity that interacts with the system is called a(n):

i. Use case
ii. Actor
iii. Stakeholder
iv. Association

97) More powerful hardware resulted into the development of ----------powerful and --------- software.

i. Less, complex
ii. More , complex
iii. More, simple
iv. Less, simple

98) A context diagram is used:

i. As a first step in developing a detailed DFD of a system
ii. In systems analysis of very complex systems
iii. As an aid to system design
iv. As an aid to programmers

99) " System should maintain transaction log of every system"
The above statement is an example of:

i. Functional requirement
ii. Non-functional requirement
iii. Pseudo requirement
iv. None of these

100) The architectural model provides the software engineer with the view of the system as a whole:

i. True
ii. False

101) As per Peter Coad's methodology , which of the following may not be a perfect candidate for being an object?

i. Zone
ii. Recipient
iii. Garage
iv. Password

Arsal Shah & Ali Malik (MCS Family Group Team)

102) In the case of ---------- approach , data is decomposed according to functionality requirements.
   i. Object-oriented
   ii. Action-oriented
   iii. Event-oriented
   iv. Process-oriented

103) In UML based Object Oriented Model of a system, the diamond sign is used to depict ---------- relations between two objects/classes.
   i. Aggregation and Association
   ii. Inheritance and Association
   iii. Composition and Aggregation
   iv. Composition, Aggregation and Association

104) The system specification describes the:
   i. Function and behavior of a computer-based system
   ii. Implementation of each allocated system element
   iii. Algorithmic detail and data structures
   iv. Time required for system simulation

105) In object oriented approach, ---------- are the people and organizations that take part in the system under consideration:
   i. Actors
   ii. Places
   iii. Participants

106) Software Design discusses ---------- aspect of software development.
   i. What
   ii. How
   iii. Who
   iv. When

107) ---------- requirements cause frequent modifications in user interface.
   i. Functional
   ii. Non-functional
   iii. Unstable
   iv. User

108) By levelling a DFD (adding more levels of abstraction) we mean:
   i. Splitting it into different levels
   ii. Make its structure uniform
   iii. Expanding a process into one with more sub-processes giving more detail
   iv. Summarizing a DFD to specify only these essentials

109) A “register” in “Point of Sale system” is an example of:
   i. Actor
   ii. Participant
   iii. Tangible thing
   iv. Transaction

110) ---------- is a set of processes and tools to develop software.
   i. Software engineering
   ii. Information
   iii. Software
   iv. None of the given

Arsal Sunni & Aina Malik (MCS Family Group Team)

111) The ---------- on which program operates is also considered as part of the software.

i. Data
ii. Information
iii. Program
iv. None of the given

112) ---------- diagram provides a time-based view and collaboration diagrams which provide an organization based view of the system's dynamics.

i. Data flow diagram
ii. Entity relationship diagram
iii. Class diagram
iv. Sequence diagram

113) Synchronous messages are "call events" and are denoted by ----------

i. Full arrow
ii. Half arrow
iii. <<create>>
iv. <<destroy>>

114) Which of the following are the components of system engineering software?

i. Process
ii. Methods
iii. Tools
iv. All of the given

115) Identifying system features include ----------

i. Log important information
ii. Conduct business
iii. Analyze business results
iv. All of the above

116) ---------- is yet another technique that is used to reduce customer dissatisfaction at the requirement stage.

i. Study of similar systems
ii. Site visits
iii. Prototyping
iv. All of the above

117) Data store notation in DFD represents:

i. Data input
ii. Data output
iii. Data input and data output
iv. None of the above

118) The process of defining attributes is called ----------

i. Who know me?
ii. What I know?
iii. Whom I know?
iv. All of the above

119) The output of the design process is a description of the:

i. Software architecture
ii. Software Code
iii. Software
iv. All of the above

Arsalan Shani & Alina Malik (MCS Family Group Team)

120) Which of the following are the levels of software requirements?
   i. Business requirements
   ii. User requirements
   iii. Functional requirements
   iv. All of the above

121) Given below are some statements associated with data flow diagrams. Identify the correct statement among them.
   i. Data flow is made used of to model what systems do
   ii. Flows of data can take place from a process to a sink
   iii. Context diagrams shows the major system processes
   iv. All processes have to be labelled or decomposed

122) In which of the following diagram the actors and attributes are represented with system boundary?
   i. Data flow diagram
   ii. Entity relationship diagram
   iii. Class diagram
   iv. Use case diagram

123) _____ is real looking mock_up of what would be eventually delivered and might not do anything useful.
   i. Study of similar system
   ii. Site visits
   iii. Prototyping
   iv. All of the above

124) _____ is blueprint for software construction.
   i. Object oriented design
   ii. Sequence design
   iii. Software design
   iv. All of the above

125) _____ requirements lead to ill-spent time and rework.
   i. Unacceptable
   ii. Ambiguous
   iii. Dissatisfaction of customer
   iv. None of the above

126) Which type of diagram is used to depict the dynamic behaviour of a system.
   i. ERD diagram
   ii. DFD diagram
   iii. Class diagram
   iv. Collaborations diagram

127) What is the most crucial non-functional requirement of a system to control radiation dosages that are emitted as treatment for cancer?
   i. Security
   ii. Reliability
   iii. Easy usability
   iv. Accuracy

128) A better design has an objective achieve
   i. High cohesion
   ii. Low cohesion

Arsal Ghani & Aina Malik (MCS Family Group Team)

iii. Low coupling
iv. High cohesion and low coupling

129) Which of the following are trhe components of software engineering framework is combine the three remaining components?
i. Process
ii. Method
iii. Tools
iv. All of the above

130) In sequence diagrams the time required by the receiver object to process the message is denoted by an _____
i. Activation box
ii. Message line
iii. Life line
iv. All of the above

131) How many types of OOD modes have _____.
i. One
ii. Two
iii. Three
iv. Four

132) Which notation is used to represent the process of the system in DFD model.
i. Process
ii. External agent
iii. Data flow
iv. Data store

133) Requirement engineering mainly deals with the _____ of the system
i. Process
ii. Maintenance
iii. Development phase
iv. Definition phase

134) Insufficient user involvement leads to _____ products.
i. Unacceptable
ii. Ambiguous
iii. Dissatisfaction of customer
iv. None of the above

135) Collaboration diagrams have basically two types of components: objects and _____
i. Messages
ii. Method
iii. Classes
iv. None of the above

136) In object-oriented analysis how many number of tasks must occurs _____.
i. 1
ii. 2
iii. 3
iv. None of the above

137) State transition diagram is helpful in determining _____.
i. Data store
ii. Process flow

Arsal Ghani & Aina Malik (MCS Family Group Team)

iii. Business understanding
iv. None of the above

138) In sequence diagram events are organized in a _____ time life line.
i. Vertical
ii. Horizontal
iii. Both A and B
iv. All of the above

139) Asynchronous messages are “signals,” denoted by _____.
i. Full arrow
ii. Half arrow
iii. <<create>>
iv. <<destroy>>

140) When we write a program for computer and then we named it as _____ .
i. Data
ii. Information
iii. Software
iv. None of the given

141) Context level diagram present in which of the following document.
i. SRS-software requirement specification
ii. Design document
iii. Test phase
iv. All of the above

142) _____ is diagram in which objects are interact with each other and these are arranged in a sequence.
i. ERD diagrams
ii. Inheritance diagrams
iii. Class diagrams
iv. Sequence diagrams

143) Which of the following layers are include in object-oriented design?
i. The subsystem layers
ii. The class and object layer
iii. All of the above
iv. The message layers

144) Which notation is used to represent the boundary of the system in DFD model?
i. Process
ii. External agent
iii. Data flow
iv. Data store

145) Identifying whole-part structures (aggregations) means what are my _____.
i. Components
ii. Structures
iii. Class
iv. Object

146) An object or class may further be classified on the basis of _____ .
i. Behaviour driven attributes
ii. Data driven attributes
iii. Responsibility driven attributes

Arsalan Shan & Alina Malik (MCS Family Group Team)

iv. All of the above

147) DFD notation contains
i. Process
ii. External agent
iii. Data flow
iv. All of the above

148) The doted lines in sequence diagram are called ____ .
i. Life line
ii. Message line
iii. Entities line
iv. All of the above

149) An object may create another object via a _____ message .
i. Full arrow
ii. half arrow
iii. <<create>>
iv. <<destroy>>

150) How many levels of software requirements are _____?
i. One
ii. Two
iii. Three
iv. Four

151) Which of the following diagram has iterative activities?
i. Data flow diagram
ii. Entity relationship diagram
iii. Class diagram
iv. Use case diagram

152) Which of the items listed below is not one of the software engineering layers?
i. Tools
ii. Manufacturing
iii. Process
iv. Methods

153) Coupling is a measure of _____ a module or component.
i. Independent
ii. Dependent
iii. Closeness
iv. All of the given

154) Software maintenance phase involves
i. Debugging
ii. Adding new features
iii. Making changes
iv. All of the given

155) The hardest single part of building a software system is deciding precisely _____ to build.
i. When
ii. What
iii. Why
iv. All of the given

156) Interaction diagrams depict the ____ behaviour of the system.

Arsal Shah & Aina Malik (MCS Family Group Team)

i. Static
ii. Active
iii. Dynamic
iv. None of the given

157) A _____ can be used to describe the dynamic behaviour of an object-oriented system.
i. ERD diagrams
ii. Inheritance diagrams
iii. Class diagrams
iv. Series diagrams

158) The Use case diagram shows that which ____ interact with each use case.
i. Use case
ii. Actor
iii. Component
iv. Relation

159) Transactions are the _____ that must be remembered through time.
i. Events
ii. Action
iii. Triggers
iv. Methods

160) A necessary supplement to transform or transaction mapping needed to create a complete architectural design is
i. Entity relationship diagrams
ii. The data dictionary
iii. Processing narratives for each module
iv. Best cases for each module

Arsal Shani & Aina Malik (MCS Family Group Team)

Cs504 2016 All Quiz With Answer

External Entity may be

Select correct option

source of input data only

source of input data or destination of results

destination of results only

repository of data

System _____ are built to allow the System Engineer to evaluate the system components in relationship to one another.

Select correct option

Requirements

Documents

Models

Marian Shah Zill

Test Cases

The _____ relationship is kind of a generalization specialization relationship.

Select correct option

Bit-Byte

Uses

Binary

Extends

All the documents related to the software are also considered as part

of the _____.

Select correct option

Physical Document

Logical Document

Maham Shah Zill

Relational Database

Software

In object oriented design, the structure of the system revolves around.

Select correct option

Objects

Methods

Properties

None of the given options

Which elements of business processing engineering are the responsibilities of the software engineer?

Maham Shah Zil

Select correct option

business area analysis

business system design

product planning

information strategy planning

The Use case diagram shows that which _____ interact with each use case.

Select correct option

Use case

Actor

Component

Relation

Maham Shah Zil

A class will be cohesive if:

Select correct option

Class does not implement Complex interfaces

Class does not have Complex Methods

If most of the methods do not use most of the data members most of the time

if most of the methods use most of the data members most of the time.

_____ diagram does not capture control flow information, it just shows the flow of the data in a system.

Select correct option

Sequence

Data Flow

Maham Shah Zill

Activity

Class

A Process in Data Flow Diagram (DFD) represents

Select correct option

Flow of data

Transformation of data

Storage of data

An external agent

Regarding Flow Chart which of the following statement(s) is/are TRUE:

Select correct option

Flow charts are usually used to describe flow of control in a system

Maham Shah Zill

Flow Charts just show the flow of the data in a system.

Looping or Iterations can not be represented in Flow Chart

None of given options

The system model template contains which of the following elements

Select correct option

Input

Output

System Out

Input / Output

Which one of the following is the external quality of a software product?

Maham Shahzil

Select correct option

Correctness

Concision

Cohesion

Low Coupling

_____ structure represents the internal organization of the various data and control items.

Select correct option

Data

Value

Information

Conceptual

Maham Shah Zill

An arrow in Data Flow Diagram (DFD) represents

Select correct option

Direction of flow of data

Processing of data

External agent

Internal agent

In this case of _____, intra component linkages are stronger while inter component linkages are weak.

Select correct option

high cohesion

low coupling

low cohesion

Maham Shah Zill

high coupling

A context diagram is used

Select correct option

as the first step in developing a detailed DFD of a system

in systems analysis of very complex systems

as an aid to system design

as an aid to programmers

_____ is a technique in which we construct a model of an entity based upon its essential characteristics and ignore the inessential details.

Select correct option

Inheritance

Maham Shahzil

Polymorphism

Aggregation

Abstraction

_____ relationship is concerned with classes not with the class instantiates.

Select correct option

Association

Inheritance

Aggregation

Composition

_____ requirements cause frequent modifications in user interface.

Maham Shahzill

Select correct option

Functional

Non-functional

Unstable

User

The context diagram is used as the top level abstraction in a _____ developed according to principles of structured analysis.

Select correct option

Dataflow diagram

Activity Diagram

State Transition Diagram

Maham Shah Zill

USe Case Diagram

In Data Flow Diagram, the entity or system, outside the boundary of this system is called

Select correct option

Process

Data Flow

External Agent

Data Store

In the case of action-oriented approach, data is decomposed according to:

Select correct option

Maham Shah Zil

Object requirements

Functionality requirements

Corresponding domain model

Compatibility with object interface

An external entity that interacts with a system is called a(n):

Select correct option

use case

actor

stakeholder

association

_____ requirements are often called product features.

Maham Shah Zill

Select correct option

Functional

Non- Functional

Developer

User

When two components of a system are using the same global data area, they are related as

Select correct option

Data Coupling

Content Coupling

Common Coupling

External Coupling

Maham Shah Zill

The context diagram is used as the top level abstraction in a _____ developed according to principles of structured analysis.

Select correct option

Dataflow diagram

Activity Diagram

State Transition Diagram

USe Case Diagram

Modules with high cohesion and low coupling can be treated and analyzed as:

Select correct option

White Boxes

Maham Shah Zill

Black Boxes

Gray Boxes

None of the given options

Identify the TRUE statement:

Select correct option

Normally Object Oriented design is more maintainable than functional oriented.

Software with Functional oriented design does not fulfill non functional requirements.

Object Oriented design can not implement "Separation of concerns" strategy

Function Oriented design does not lead to an efficient product.

A context diagram:

Maham Shah Zill

Select correct option

describes detailed design of a system

is a DFD which gives an overview of the system

is a detailed description of a system

is not used in drawing a detailed DFD

The modules that interact with each other through message passing have _____.

Select correct option

Low Coupling

High Coupling

Low Cohesion

High Cohesion

Maham Shah Zill

A maintainable design is a design , which supports

Select correct option

Change

debugging

Adding new features

All of the given

The best way to conduct a requirements validation review is to

Select correct option

examine the system model for errors

have the customer look over the requirements

Maham Shah Zill

send them to the design team and see if they have any concerns

use a checklist of questions to examine each requirement

In _____ the analyst determines all the sources of requirements and where do these requirements consume

Select correct option

Data Flow Analysis

Source and Sink Analysis

Down Parsing

Up Parsing

Which of the following sentence is true regarding user interface design?

Select correct option

Maham Shah Zil

GUI interfaces are good for all tasks which a user needs to perform at an interface.

The higher the response time, the better is the interface.

The simpler the interface, the efficient is the system.

Command-line interfaces are faster for some tasks which the user needs to perform.

Which of the following strategy/strategies lead(s) to a good software design:

Select correct option

Separation of Concerns

Modularity

Divide-and-conquer

All of the given options

Maham Shah Zil

In object oriented design, the structure of the system revolves around.

Select correct option

Objects

Methods

Properties

None of the given options

Data cannot flow from one external entity to other external entity because:

Select correct option

It will get corrupted

It is not allowed in DFD

Maham Shah Zill

An external entity has no mechanism to read or write

Both are outside the context of the system

At which stage of software development loop, results are delivered?

Select correct option

Problem definition

Solution integration

Technical development

Status quo

A context diagram is used

Select correct option

as the first step in developing a detailed DFD of a system

Maham Shah Zil

in systems analysis of very complex systems

as an aid to system design

as an aid to programmers

All the documents related to the software are also considered as part

of the _____.

Select correct option

Physical Document

Logical Document

Relational Database

Software

Maham Shah Zill

The condition that must be met before the use case can be invoked, is called:

Select correct option

Pre-Condition

Post-Condition

Pre-Assertion

Post-Assertion

In use case diagram, an ellipse signifies a(n):

Select correct option

actor

class

use case

Maham Shah Zill

system boundary

Modules with high cohesion and low coupling can be treated and analyzed as:

Select correct option

White Boxes

Black Boxes

Gray Boxes

None of the given options

The system specification describes the

Select correct option

function and behavior of a computer-based system

Maham Shah Zill

implementation of each allocated system element

algorithmic detail and data structures

time required for system simulation

A use case represents:

Select correct option

a class, its attributes and operations.

an operation’s interface and signature.

the role a user plays when interacting with the system.

the system’s functionality for a particular purpose.

Maham Shah Zill

_____ is a technique that can be used to reduce customer dissatisfaction at requirement stage.

Select correct option

Analysis

Negotiation

Prototyping

GUI

The data on which the program operates is also considered as part of the

_____.

Select correct option

Important Data

Maham Shah Zill

Software

Logical Data

Utility Software

The modules interacting with each other through message passing have _____ between them.

Select correct option

low cohesion

high cohesion

low coupling

high couling

By leveling a DFD (adding more levels of abstraction) we mean

Maham Shah Zill

Select correct option

Splitting it into different levels

Make its structure uniform

Expanding a process into one with more sub-processes giving more detail

Summarizing a DFD to specify only the essentials

Following is/are example(s) of illegal data flow in Data Flow Diagram (DFD)

Select correct option

External Agents directly communicating with each other

External Agent updating information in a Data

Store

External Agent accessing information from a Data Store

Maham Shah Zil

All of the given options

Software Design discusses _____ aspect of software development

Select correct option

What

How

Who

When

Which one is not a part of Software Development phase ?

Select correct option

Construction

Maham Shah Zill

Scope

Project Vision

Definition

_____ structure represents the internal organization of the various data and control items.

Select correct option

Data

Value

Information

Conceptual

Modules with high cohesion and low coupling can be treated and analyzed as:

Select correct option

Maham Shah Zill

White Boxes

Black Boxes

Gray Boxes

None of the given options

The best way to conduct a requirements validation review is to

Select correct option

examine the system model for errors

have the customer look over the requirements

send them to the design team and see if they have any concerns

use a checklist of questions to examine each requirement

Maham Shah Zill

In the functional design, the structure of the system revolves around.

Select correct option

objects

properties

functions

All of the given options

_____ of the total cost of the software development is spent on maintenance.

Select correct option

one third

two third

Maham Shah Zill

one fourth

three fourth

_____ is one of the techniques to document domain knowledge

Select correct option

State transition diagram

Feasibility matrix

System matrix

None of given options

Which of the following sentence is true regarding user interface design?

Select correct option

Maham Shah Zil

GUI interfaces are good for all tasks which a user needs to perform at an interface.

The higher the response time, the better is the interface.

The simpler the interface, the efficient is the system.

Command-line interfaces are faster for some tasks which the user needs to perform.

External Entity may be

Select correct option

source of input data only

source of input data or destination of results

destination of results only

repository of data

Maham Shah Zil

The context diagram is used as the top level abstraction in a _____ developed according to principles of structured analysis.

Select correct option

Dataflow diagram

Activity Diagram

State Transition Diagram

USe Case Diagram

A Process in Data Flow Diagram (DFD) represents

Select correct option

Flow of data

Transformation of data

Maham Shah Zill

Storage of data

An external agent

_____ of the total cost of the software development is spent on maintenance.

Select correct option

one third

two third

one fourth

three fourth

Construction activities are directly related to software _____.

Select correct option

Management

Maham Shah Zil

Planning

Quality Assurance

Development

In this case of _____, intra component linkages are stronger while inter component linkages are weak.

Select correct option

high cohesion

low coupling

low cohesion

high coupling

In Data Flow Diagram (DFD), one data store cannot directly copy the data from another _____ .

Maham Shah Zill

Select correct option

Agent

Process

Data store

Flow

System models include:

Select correct option

User business processes

User activities for conducting the business processes

Processes that need to be automated

All of the given options

Maham Shah Zil

The project manager would need _____ document to monitor and track the progress of the project.

Select correct option

Design

Project

Requirement

Planning

External Entity may be

Select correct option

source of input data only

source of input data or destination of results

destination of results only

Maham Shah Zill

repository of data

Regarding Data Flow Model, which of the following statement(s) is/are true:

Select correct option

It captures the transformation of data between processes/functions of a system

Processes on a data flow can operate in parallel

Only those processes are represented which we need to automate

All of the given options

Use case construction is a technique used for:

Select correct option

requirements determination.

Maham Shah Zil

requirements structuring.

user interface design.

database design.

In the functional design, the structure of the system revolves around.

Select correct option

objects

properties

functions

Maham Shah Zill

All of the given options

In Data Flow Diagram (DFD), data flow can:

Select correct option

Only originate from an external entity

Only terminate in an external entity

Originate and terminate in an external entity

Either originate or terminate in an external entity but not both

A context diagram:

Select correct option

describes detailed design of a system

is a DFD which gives an overview of the system

Maham Shah Zill

is a detailed description of a system

is not used in drawing a detailed DFD

Coupling is a measure of _____ of a module or component.

Select correct option

Independence

Dependence

Aggregation

Composition

Coupling is a measure of _____ of a module or component.

Select correct option

Independence

Maham Shah Zil

Dependence

Aggregation

Composition

Software Design discusses _____ aspect of software development

Select correct option

What

How

Who

When

Which of the following strategy/strategies lead(s) to a good software design:

Select correct option

Separation of Concerns

Maham Shah Zil

Modularity

Divide-and-conquer

All of the given options

_____ relationship is concerned with classes not with the class instantiates.

Select correct option

Association

Inheritance

Aggregation

Composition

When two components of a system are using the same global data area, they are related as

Maham Shahzill

Select correct option

Data Coupling

Content Coupling

Common Coupling

External Coupling

The best way to conduct a requirements validation review is to

Select correct option

examine the system model for errors

have the customer look over the requirements

send them to the design team and see if they have any concerns

Maham Shah Zill

use a checklist of questions to examine each requirement

Data Flow Model:

Select correct option

Captures the flow of data in a system

Helps in developing an understanding of system's functionality

Describes data origination, transformations and consumption in a system

All of the given options

_____ diagram does not capture control flow information, it just shows the flow of the data in a system.

Select correct option

Sequence

Data Flow

Maham Shahzil

Activity

Class

To construct a system model the engineer should consider one of the following restraining factors?

Select correct option

assumptions and constraints

budget and expenses

data objects and operations

schedule and milestones

Maham Shah Zil

A class will be cohesive if:

Select correct option

Class does not implement Complex interfaces

Class does not have Complex Methods

If most of the methods do not use most of the data members most of the time

if most of the methods use most of the data members most of the time.

Which one of the following is the external quality of a software product?

Select correct option

Correctness

Concision

Maham Shah Zill

Cohesion

Low Coupling

Regarding Data Flow Model, which of the following statement(s) is/are true:

Select correct option

It captures the transformation of data between processes/functions of a system

Processes on a data flow can operate in

parallel

Only those processes are represented which we need to automate

All of the given options

The system model template contains which of the following elements

Maham Shah Zill

Select correct option

Input

Output

System Out

Input / Output

A _____ is not the real product but just a real looking mock-up of what would be eventually delivered.

Select correct option

Software

Program

Prototype

Test Case

Maham Shah Zill

In the case of _____ in a system, module boundaries are not well defined.

Select correct option

low cohesion

high coupling

low coupling

high cohesion

Data cannot flow from one external entity to other external entity because:

Select correct option

It will get corrupted

It is not allowed in DFD

Maham Shah Zill

An external entity has no mechanism to read or write

Both are outside the context of the system

The process of utilizing our knowledge of computer science in effective production of software systems is called _____.

Select correct option

Chemical Engineering

Electrical Engineering

Computer Engineering

Software Engineering

Arranging information in _____ form makes it easy to read, understand and comprehend as compared to streams of text.

Select correct option

Maham Shah Zil

Columns

Paragraph

Tabular

Rows

The goal of _____ is to translate the customer's desire for a set of defined capabilities into a working product.

Select correct option

Electrical Engineering

Product Engineering

Hardware Engineering

Mechanical Engineering

Maham Shah Zill

In data flow diagram (DFD), Create, Update, Delete and Read operations are normally called:

Select correct option

CRUD operations

DURC operations

RUDC operations

CDUR operation

In use case diagram, an ellipse signifies a(n):

Select correct option

actor

class

use case

Maham Shah Zill

system boundary

In the case of action-oriented approach, data is decomposed according to:

Select correct option

Object requirements

Functionality requirements

Corresponding domain model

Compatibility with object interface

A class will be cohesive if:

Select correct option

Mahan Shah Zil

Class does not implement Complex interfaces

Class does not have Complex Methods

If most of the methods do not use most of the data members most of the time

if most of the methods use most of the data members most of the time.

The design process usually involves:

Select correct option

Developing a number of different models.

Looking at the system from different angles.

Describing the system at various levels of abstraction.

All of the given options

Maham Shah Zil

The architecture components for product engineering are

Select correct option

data, hardware, software, people

data, documentation, hardware, software

data, hardware, software, procedures

documentation, hardware, people, procedures

In use case diagram, the scope of the system is defined by:

Select correct option

Actor

Entity

System Boundary

Maham Shah Zill

"Extends" relationship

_____ requirements cause frequent modifications in user interface.

Select correct option

Functional

Non-functional

Unstable

User

All the documents related to the software are also considered as part

of the _____.

Select correct option

Physical Document

Maham Shah Zill

Logical Document

Relational Database

Software

The use of traceability tables helps to

Select correct option

debug programs following the detection of run-time errors

determine the performance of algorithm implementations

identify, control, and track requirements changes

none of the given

Maham Shah Zill

CS504-SOFTWARE ENGEINRING-I

SOLVED MCQS QUIZ 1

LEACTURE (1 to 15)

Solved by JUNAID MALIK



AL-Junaid Institude

Contact no: 03041659294

VU LMS HELP vulmshelp.com

LMS Handling

Subject Enrollment

1) Online classes available
HTML, CSS, Javascript
Phyton..etc
2) Solved Graded Activities
- *Assignment's*
- *Quizze's*
- *GDB's*

3) Sloved Quiz File's
4) Short Note's
5) Past paper's &
Current paper's

Final project CS619

1) SRS
(Software Requirement's Specification)
2) DD
(Design Document)
3) Test phase + viva
4) Viva preparation
5) Final Deliverable

Website Link
vulmshelp.com

Gmail
junaidfazal08@gmail.com

PLZ DOWNLOAD GRAND QUIZ FILE (vulmshelp.com) Then Start Your QUIZ

1. ----- is a technique in which we construct a model of an entity based upon its essential characteristics and ignore the inessential details.

- Polymorphism
- Abstraction
- Aggregation
- Inheritance

2. An attribute that varies over time. e.g.., price of an item, should be replaced by a/an _____ with an effective data and value.

- Additional class
- Structure
- Interface
- Additional Method/F unction

3. To construct a system model the engineer should consider one of the following restraining factors?

- Schedule and milestones
- Budge and expenses
- Data objects and operations
- Assumptions and constraints

4. In the use case diagram, an ellipse signifies a(n):

- System boundary
- Class
- Actor
- Use case

5. A software requirement document describes all the _____provided by the system along with the constraints under which it must operate.

- Conditions
- Processes
- Services
- Events

6. UML is a language for_____.

- Low-level Programming

- High-level Programming
- Modeling and design
- Creating diagrams only

7. Which elements of business processing engineering are the responsibilities of the software engineer?

- Business area analysis
- Product planning
- Business system design
- Information strategy planning

8. Identify the TRUE statement.

- Software with Functional oriented design does not fulfill nonfunctional requirements.
- Functional oriented design does not lead to an efficient product.
- Normally object oriented design is more maintainable than functional oriented.
- Object oriented design cannot implement “separation of concerns” strategy

9. Specialization means:

- Calling the same method with object of child object
- Abstraction
- Hiding the data
- Creating new subclasses from an existing class

10. In case of _____ approach, decomposition of a problem revolves around data.

- Object-oriented
- Action-oriented
- Event-oriented
- Process-oriented

11. An architectural style encompasses which of the following elements?

- Semantic Model and Constraints
- Set of components and semantic models
- Constraints, set of components and semantic models
- Set of components and constraints

12. In “Railway tickets reservation system” the roles such as inquiry, reservation, ticketing and cancellation are to be performed by the user call

- System analyst
- System designer
- Passenger
- System developer

13. There are _____ most important characteristics of an object.

- Two
- Four
- Three
- Six

14. Which one is not a part of software development phase?

- Construction
- Project vision
- Definition
- Scope

15. Which one of the following is the external quality of a software product?

- Cohesion
- Correctness
- Low coupling
- Concision

16. The process of utilizing our knowledge of computer science in effective production of software system is called___

- Electrical Engineering
- Chemical Engineering
- Software Engineering
- Computer Engineering

17. The context diagram is used as the top level abstraction in a _____ developed according to principles of structure analysis.

- Dataflow diagram
- State transition diagram
- Activity diagram
- Use case diagram

18. While establishing the services for an object, the goal is to keep data and action together for _____ coupling and _____ cohesion.

- Higher, Lower
- Lower, Higher
- Lower, Lower
- Higher, Higher

19. Software architecture must address _____ requirements of a software system.

- Both functional and non-functional
- Functional
- Non-functional
- User interface requirements

20. _____ has become a standard notation for object oriented system modeling.

- UML
- Java
- OCL (object constraint language)
- C++

21. Software Engineering is the combination of tools, techniques and_____

- Testing
- Design
- Maintenance
- Processes

22. The _____ relationship is the kind of a generalization specialization relationship.

- Extends
- Uses
- Binary
- Bit-Byte

23. In UML based oriented model of a system, composition relation between two objects is shown by a

- A half arrowhead
- An unfilled diamond
- A dot
- A filled diamond

24. Return values in synchronous message are:

- Represented by solid lines
- Compulsory
- Not used
- May not when response in obvious

25. _____requirements are often called product feature.

- Developer
- User
- Functional
- Non-functional

26. In multiprocessing application, different execution threads may pass information to one another by sending_____.

- Synchronous message
- Interrupt calls
- Asynchronous message
- System calls

27. The use case diagram shows that which_____ interact with each use case.

- Actor
- Relation
- Component
- Use case

28. If Cat is derived from Mammal class, and Mammal is derived from Animal class, then:

- Cat will inherit Animal's functions and data
- Cat is allowed to access only the Mammal's Class
- Cat will not be able to access any class
- Cat will inherit Animal's functions and data

29. Coupling is a measure of _____of a module or component.

- Aggregation
- Independence
- Dependence
- Composition

30. Normally a system will be more easy to modify if its modules have:

- Low coupling and High cohesion.
- High coupling and Low cohesion.
- High coupling and High cohesion.
- Low coupling and Low cohesion.

31. Whole part structure is also called
    a. Generalization
    b. Aggregation
    c. Specialization
    d. Association

32. A prototype is not the real product. It is rather just a real looking of what would be eventually delivered and might not do anything useful.
    a. Mock-up page 68
    b. Ad-hoc
    c. Design
    d. Structure

33. In "Railway ticket reservation system" the roles such as inquiry, reservation, ticketing and cancellation are to be performed by the user called:-
    a. Passenger
    b. System analyst
    c. System developer
    d. System designer

34. A useful technique for evaluating the overall complexity of a proposedarchitecture is to look at the component's _.
    a. number and size of components
    b. flow dependencies and sharing dependencies
    c. size and cost
    d. function points

35. _ relationship is concerned with classes not with the class instantiates.
    a. Association
    b. Inheritance
    c. Aggregation
    d. Composition

35. Software development is a step-by-step process, and in phase of software development Business Objective of an organization gets cleared.
    a. Maintenance
    b. Development
    c. Definition
    d. Vision
36. The data on which the program operates is also considered as part of the .
    a. Important Data
    b. Software page 01
    c. Logical Data
    d. Utility Software

37. Consider the following piece of code:
    public class Square extends Shape {
    // some code
    }
    The above code is an example of:
    a. Part-Whole relationship
    b. Generalization/Specialization
    c. Data Sharing
    d. Data encapsulation
38. Requirement engineering mainly deals with the __ of the system
    a. Vision Phase
    b. Definition Phase page 16
    c. Development Phase
    d. Maintenance Phase

39. A relationship indicates that one entity is composed of one or more parts which are themselves instances of that or another entity.
    a. Inheritance
    b. Whole-part
    c. Generalization
    d. Specialization
40. In UML Object Model Notation
    a. C++ language-specific programs are constructed
    b. mathematical problems are visualised
    c. relationships among classes and sub-classes are expressed page 92
    d. Graphs and tables are used to explain Software Engineering principles
41. In the case of , intra component linkages are stronger while inter component linkages are weak.
    a. high cohesion
    b. low coupling page 73
    c. low cohesion
    d. high coupling
42. In Data Flow Diagram, the entity or system, outside the boundary of this system is called
    a. Process
    b. Data Flow
    c. External Agent
    d. Data Store
43. A cohesive Class is one which emphasizes on unit of functionality
    a. Single
    b. Multiple page 76
    c. Static
    d. Both Single and Multiple
44. The goal of is to translate the customer's desire for a set of defined capabilities into a working product.
    a. Electrical Engineering
    b. Product Engineering
    c. Hardware Engineering
    d. Mechanical Engineering
45. In Object Oriented Design. layer contains the details that enable each object to communicate with its collaborators
    a. Subsystem
    b. Responsibility
    c. Message page 89

46. A DFD is normally leveled (adding more levels of abstraction) as
    a. it is a good idea in design
    b. it is recommended by many experts
    c. it is easy to do it
    d. it is easier to read and understand a number of smaller DFDs than one large DFD
47. As per Peter Coad's methodology, which of the following may NOT be a perfect candidate for being an object?
    a. Zone
    b. Recipient
    c. Garage
    d. Password page 93 94
48. The modules interacting with each other through message passing have between them.
    a. low cohesion
    b. high cohesion
    c. low coupling page 74
    d. high coupling
49. In UML based Object Oriented model of a system, the inheritance relation between two classes is shown by a sign towards the super-class side on the association line between the two.
    a. A filled diamond
    b. An unfilled diamond
    c. A half arrowhead
    d. An arrowhead page 92
50. Which statement is not according to the software engineering principles? Software engineering is a(n) __.
    a. Balancing act
    b. Disciplined approach
    c. Unsystematic approach page 5
    d. Quantifiable approach
51. A is not the real product but just a real looking mock-up of what would be eventually delivered.
    a. Software
    b. Program
    c. Prototype(page 68)

52. The best way to conduct a requirements validation review is to
    a. examine the system model for errors
    b. have the customer look over the requirements
    c. send them to the design team and see if they have any concerns
    d. use a checklist of questions to examine each requirement
53. More powerful hardware resulted into the development of powerful and

software.

a. less, complex
b. more, complex page 4
c. more, simple
d. less, simple

54. From the following which is/are not the example(s) of illegal data flow in Data Flow Diagram (DFD)
    a. External Agents directly communicating with each other
    b. External Agent updating information in a DataStore
    c. External Agent accessing information from a Data Store
    d. There must be an intermediate process which should transform data received from one external entity and then send the transformed data to the other external entity
55. Which of the given component of software engineering framework provides different techniques that can be used to perform a particular task?
    a. Processes
    b. Tools
    c. Methods page 12
    d. Quality Focus
56. Prototyping is used when there is regarding requirements.
    a. Uncertainty(page 68)
    b. Confirmation
    c. Conflict
    d. Consensus
57. The process of utilizing our knowledge of computer science in effective production of software systems is called
    a. Chemical Engineering
    b. Electrical Engineering
    c. Computer Engineering
    d. Software Engineering page 2
58. In Abbot's Textual Analysis technique, different parts of speech are identified within the text of the specification and these parts are modeled using different
    a. Event
    b. Process
    c. Operations
    d. Components page 90
59. To help separate an object's external behavior from its implementation, the technique used is called
    a. Generalization
    b. Association

c. Composition
d. Abstraction page 86

60. A Process in Data Flow Diagram (DFD) represents
a. Flow of data
b. Transformation of data (page 49)
c. Storage of data
d. An external agent

61. In data flow diagram (DFD). Create, Update, Delete and Read operations are normally called:
a. CRUD operations page 53
b. DURC operations
c. RUDC operations

62. Which of the following is not part of software architecture?
a. Databases
b. data design
c. program structure
d. algorithm details

63. Selecting objects in a domain) include:
a. Actors. Participants and Places
b. Only Participants
c. Only Actors
d. Only Actors and Places

64. .Defining the services of an object means:
a. What it does? Page 96
b. What it knows?
c. Who knows it?
d. Whom it knows?

64. requirements cause frequent modifications in user interface
a. Functional
b. Non-functional
c. Unstable page 62
d. User

CS504-Software Engineering-I
Solved MCQS for Midterms papers
Solved by JUNAID MALIK and Team



AL-Junaid Institude

Contact no: 03041659294

VU LMS HELP vulmshelp.com

LMS Handling

Subject Enrollment

1) Online classes available
HTML, CSS, Javascript
Phyton..etc
2) Solved Graded Activities
- *Assignment's*
- *Quizze's*
- *GDB's*

3) Sloved Quiz File's
4) Short Note's
5) Past paper's &
Current paper's

Final project CS619

1) SRS
(Software Requirement's Specification)
2) DD
(Design Document)
3) Test phase + viva
4) Viva preparation
5) Final Deliverable

Website Link
vulmshelp.com

Gmail
junaidfazal08@gmail.com

Whole part structure is also called
a. Generalization
b. Aggregation
c. Specialization
d. Association

2. A prototype is not the real product. It is rather just a real looking _ of what would be eventually delivered and might not do anything useful.
a. Mock-up page 68
b. Ad-hoc
c. Design
d. Structure

3. In "Railway ticket reservation system" the roles such as inquiry, reservation, ticketing and cancellation are to be performed by the user called:-
a. Passenger
b. System analyst
c. System developer
d. System designer

4. A useful technique for evaluating the overall complexity of a proposedarchitecture is to look at the component's _.
a. number and size of components
b. flow dependencies and sharing dependencies
c. size and cost
d. function points

5. _ _ relationship is concerned with classes not with the class instantiates.
a. Association
b. Inheritance
c. Aggregation
d. Composition

6. Software development is a step-by-step process, and in_ phase of software development Business Objective of an organization gets cleared.
a. Maintenance
b. Development
c. Definition
d. Vision

7. The data on which the program operates is also considered as part of the .
a. Important Data
b. Software page 01
c. Logical Data
d. Utility Software

For More Visit: vulmshelp.com

8. From the following which is/are not the example(s) of illegal data flow in Data Flow Diagram (DFD)
    a. External Agents directly communicating with each other
    b. External Agent updating information in a DataStore
    c. External Agent accessing information from a Data Store
    d. There must be an intermediate process which should transform data received from one external entity and then send the transformed data to the other external entity
9. In sequence diagram, the objects are organized in a _ line and the events in a _ time line.
    a. Horizontal, straight
    b. Vertical, straight
    c. Horizontal, vertical
    d. Vertical, horizontal
10. Consider the following piece of code:

```
public class Square extends Shape {
// some code
}
```

The above code is an example of:
    a. Part-Whole relationship
    b. Generalization/Specialization
    c. Data Sharing
    d. Data encapsulation
11. Requirement engineering mainly deals with the _ _ of the system
    a. Vision Phase
    b. Definition Phase page 16
    c. Development Phase
    d. Maintenance Phase
12. A _ relationship indicates that one entity is composed of one or more parts which are themselves instances of that or another entity.
    a. Inheritance
    b. Whole-part
    c. Generalization
    d. Specialization
13. In UML Object Model Notation _
    a. C++ language-specific programs are constructed
    b. mathematical problems are visualised
    c. relationships among classes and sub-classes are expressed page 92
    d. Graphs and tables are used to explain Software Engineering principles
14. In the case of _ , intra component linkages are stronger while inter component linkages are weak.
    a. high cohesion

For More Visit: vulmshelp.com

b. low coupling page 73
c. low cohesion
d. high coupling

_ that must be remembered through time.

15. Transactions are the _
a. Events page 93
b. Actions
c. Triggers
d. Methods

16. The modules interacting with each other through message passing have _ between them.
a. low cohesion
b. high cohesion
c. low coupling page 74
d. high coupling

17. In UML based Object Oriented model of a system, the inheritance relation between two classes is shown by a _ sign towards the super-class side on the association line between the two.
a. A filled diamond
b. An unfilled diamond
c. A half arrowhead
d. An arrowhead page 92

18. Which statement is not according to the software engineering principles? Software engineering is a(n) _
a. Balancing act
b. Disciplined approach
c. Unsystematic approach page 5
d. Quantifiable approach

19. A _ is not the real product but just a real looking mock-up of what would be eventually delivered.
a. Software
b. Program
c. Prototype(page 68)
d. Test Case

20. The best way to conduct a requirements validation review is to
a. examine the system model for errors
b. have the customer look over the requirements
c. send them to the design team and see if they have any concerns
d. use a checklist of questions to examine each requirement

21. More powerful hardware resulted into the development of _ powerful and _ software.
a. less, complex
b. more, complex page 4
c. more, simple

For more visit: vulmshelp.com

d. less, simple

For More Visit: vulmshelp.com

22._ _ pointed out the elegant conceptual integrity exhibited by layered organization of software systems, with the resulting gains in development and maintenance ease.
a. David Parnas
b. Shaw and Garlan
c. Barry Boehm
d. EdsgerDijkstra page 115

23. The architecture components for product engineering are
a. data, hardware, software, people
b. data, documentation, hardware, software
c. data, hardware, software, procedures
d. documentation, hardware, people, procedures

24. In Object oriented design, combining the services offered by an object with the attributes they work on, results in:
a) Lower coupling and stronger cohesion
b) Lower cohesion and stronger coupling
c) Increased likelihood of reuse
d) Decreases the modularity of the system
a. a only
b. band c
c. a and c
d. b and d

25. Architectural design process involves performing a number of activities which includes system structuring, control modeling and _ __.
a. System Analysis
b. Modular Decomposition page 120
c. Testing
d. Graphical User Interface

26. In use case diagram, the scope of the system is defined by:
a. Actor
b. Entity
c. System Boundary
d. "Extends" relationship

27. Specialization means
a. Calling the same method with object of child object
b. Hiding the data
c. Creating new subclasses from an existing class page 34 , 86
d. Abstraction

28. Which of the following sentence is true regarding user interface design?
a. GUI interfaces are good for all tasks which a user needs to perform at an interface.
b. The higher the response time, the better is the interface
c. The simpler the interface, the efficient is the system

For more visit: vulmshelp.com

d. Command-line interfaces are faster for some tasks which the user needs to perform

For More Visit: vulmshelp.com

29. In the architecture trade-off analysis method the architectural style should be described using the
    a. data Flow view and process view
    b. data Flow view and module view
    c. module view and process view
    d. data Flow view, module view and process view page 136
30. In Data Flow Diagram, the entity or system, outside the boundary of this system is called
    a. Process
    b. Data Flow
    c. External Agent
    d. Data Store
31. A cohesive Class is one which emphasizes on _ unit of functionality
    a. Single
    b. Multiple page 76
    c. Static
    d. Both Single and Multiple
32. The goal of _ is to translate the customer's desire for a set of defined capabilities into a working product.
    a. Electrical Engineering
    b. Product Engineering
    c. Hardware Engineering
    d. Mechanical Engineering
33. In Object Oriented Design. _ layer contains the details that enable each object to communicate with its collaborators
    a. Subsystem
    b. Responsibility
    c. Message page 89
    d. Object
34. A DFD is normally leveled (adding more levels of abstraction) as
    a. it is a good idea in design
    b. it is recommended by many experts
    c. it is easy to do it
    d. it is easier to read and understand a number of smaller DFDs than one large DFD
35. As per Peter Coad's methodology, which of the following may NOT be a perfect candidate for being an object?
    a. Zone
    b. Recipient
    c. Garage
    d. Password page 93 94

For More Visit: vulmshelp.com

36. Which of the given component of software engineering framework provides different techniques that can be used to perform a particular task?
    a. Processes
    b. Tools
    c. Methods page 12
    d. Quality Focus
37. To determine the architectural style or combination of styles that best fits the Proposed system requirements engineering is used to uncover
    a. algorithmic complexity
    b. characteristics and constraints page 126
    c. control and data
    d. design patterns
38. In _ _relationship, a class shares the structure and behavior defined in another class.
    a. Aggregation
    b. Composition
    c. Inheritance page 86
    d. Uses
39. Prototyping is used when there is regarding requirements.
    a. Uncertainty(page 68)
    b. Confirmation
    c. Conflict
    d. Consensus
40. The process of utilizing our knowledge of computer science in effective production of software systems is called
    a. Chemical Engineering
    b. Electrical Engineering
    c. Computer Engineering
    d. Software Engineering page 2
41. In Abbot's Textual Analysis technique, different parts of speech are identified within the text of the specification and these parts are modeled using different_
    a. Event
    b. Process
    c. Operations
    d. Components page 90
42. To help separate an object's external behavior from its implementation, the technique used is called_
    a. Generalization
    b. Association
    c. Composition
    d. Abstraction page 86

For More Visit: vulmshelp.com

43. A Process in Data Flow Diagram (DFD) represents
    a. Flow of data
    b. Transformation of data (page 49)
    c. Storage of data
    d. An external agent
44. In data flow diagram (DFD). Create, Update, Delete and Read operations are normally called:
    a. CRUD operations page 53
    b. DURC operations
    c. RUDC operations
    d. CDUR operation
45. Collaboration diagram can show_
    a) Binary messages
    b) Asynchronous messages
    c) Synchronous messages
    a. a only
    b. b only
    c. c only
    d. both b and c page 111
46. Data cannot flow from one external entity to other external entity because
    a. It will get corrupted
    b. It is not allowed in DFD page 59
    c. An external entity has no mechanism to read or write
    d. Both are outside the context of the system
47. In_ Phase of software development, Requirement Engineer focuses on realizing the Business Object of an under developed product.
    a. Maintenance
    b. Development page 16
    c. Definition
    d. Vision
48. Which of the following is not part of software architecture?
    a. Databases
    b. data design
    c. program structure
    d. algorithm details
49. Selecting objects in a domain) include:
    a. Actors. Participants and Places
    b. Only Participants
    c. Only Actors
    d. Only Actors and Places
50. Defining the services of an object means:
    a. What it does? Page 96
    b. What it knows?

For More Visit vumshelp.com

c. Who knows it?
d. Whom it knows?

For More Visit: vulmshelp.com

51. In sequence diagram, the solid lines depict:
    a. Objects (or classes)
    b. Messages being sent from one object to the other page 107
    c. Life-time of an object
    d. state of the object
52. Sequence diagrams:
    a. Provide the static behavior
    b. Provide Data Flow
    c. Provide a time-based view 106
    d. Provide parallel data flow
53. _____ requirements cause frequent modifications in user interface
    a. Functional
    b. Non-functional
    c. Unstable page 62
    d. User
54., security and maintainability are the types of _____ requirements.
    a. Non-functional page 120
    b. Domain
    c. Functional
    d. Business
55. A change becomes _____ because of close presence of data and functions
    a. Accessible
    b. Global
    c. Private
    d. Localized page 81
56. System _____ are built to allow the system engineer to evaluate the system components in relationship to one another.
    a. Requirements
    b. Documents
    c. Models page 42
    d. Test cases
57. In _____ phase of software development, requirement analyst focuses on possible design of the proposed solution.
    a. Maintenance
    b. Development (page 16)
    c. Definition
    d. Vision
58. The focus of sequence diagrams is:
    a. On objects/classes and messages exchanged among them (page 106)
    b. On static Model of system
    c. On object constraints
    d. On the flow of Control

For More Visit: vulmshelp.com

59. In Data Flow Diagram (DFD), data flow can:
    a. Only originate from an external entity
    b. Only terminate in an external entity
    c. Originate and terminate in an external entity
    d. Either originate or terminate in an externalentity but not both
60. In _ the analyst determines all the sources of requirements and where do theserequirements consume
    a. Data Flow Analysis
    b. Source and Sink Analysis (page 40)
    c. Down Parsing
    d. Up Parsing
61. In a top-down system analysis, a/an _ required to develop high level view of thesystem at first.
    a. Analyst page 54
    b. Designer
    c. Tester
    d. Developer
62. The Use case diagram shows that which _ interact with each use case.
    a. Use case
    b. Actor page 32
    c. Component
    d. Relation
63. The context diagram is used as the top levelabstraction in a _ developed according toprinciples of structured analysis.
    a. Dataflow diagram (page 31)
    b. Activity Diagram
    c. State Transition Diagram
    d. Use Case Diagram
64. Different messages in sequence diagrams includes:
    a. Simple
    b. Asynchronous
    c. Notify
    d. Both Simple and Asynchronous page 108
65. A software requirement document describes all the _ provided by the system along with the constraints under which it must operate.
    a. Conditions
    b. Services page 23
    c. Events
    d. Processes
66. In case of a _ message, the called routine that handles the message iscompleted before the caller resumes execution.
    a) Synchronous
    b) Asynchronous
    c) Bidirectional

    a. a only page 108

For More visit vulmshelp.com

b. b only
c. c only
d. both b and c

For More Visit: vulmshelp.com

67. In object oriented design, _ layer contains the data structures and algorithmic design for all attributes and operations for each object.
    a. Subsystem
    b. Class and Object
    c. Message
    d. Responsibility page 89
68. Asynchronous messages:
    a. are implemented as operation call
    b. These block caller before response
    c. occurs in multi-threaded applications (page 109)
    d. are shown by dotted line
69. In UML based object oriented model of a system, a composition relation between two objects is shown by a _ sign on the whole side of a relation line.
    a. An unfilled diamond
    b. A filled diamond
    c. A half diamond
    d. A dot
70. In UML based Object Oriented Model of a system, the diamond sign is used to depict _ relations between two objects/classes.
    a. Aggregation and Association
    b. Inheritance and Association
    c. Composition and Aggregation
    d. Composition, Aggregation and Association
71. A "register" in "Point of sale system" is an example of:
    a. Actor
    b. Participant
    c. Tangible thing page 100
    d. Transaction
72. A use case represents
    a. A class, it attributes and operation
    b. An operation's inter faces and signature
    c. The role a user plays when interaction with the system page 32
    d. The system's functionality for a particular purpose
73. In the case of _ approach, data is decomposed according to functionality requirements.
    a. Object-oriented
    b. Action-oriented page 80
    c. Event-oriented
    d. Process-oriented
74. To construct a system model the engineer should consider one of the following restraining factors?
    a. Assumptions and constraints
    b. Budget and expenses
    c. Data object and operation

For More Visit: vulmshelp.com

d. Schedule and milestone

For More Visit: vulmshelp.com

75. When two components of a system are using the same global data area, they are related as
    a. Data coupling
    b. Content coupling
    c. Common coupling google
    d. External coupling
76. _ Structure represents the internal organization of the various data and control items.
    a. Data
    b. Value
    c. Information
    d. Conceptual
77. The method of dividing and assigning different portions of a large system to different groups for construction is called _
    a. Work Breakdown Structure (page 119)
    b. Working Boundary Structure
    c. Work Basic Structure
    d. Work Breakdown System
78. "A car is made up of a body, three or four wheels, a steering mechanism. a breaking mechanism and a power-engine"
    The above statement is example of :
    a. Whole-Part relationship page 95
    b. Inheritance
    c. Specialization
    d. Generalization
79. In software engineering paradigm, any engineering approach must be founded on organizational commitment to .
    a. Cost
    b. Scheduling
    c. Quality (page 15)
    d. Performance
80. _ is a technique that can be used to reduce customer dissatisfaction at requirement stage.
    a. Analysis
    b. Negotiation
    c. Prototyping (page 71)
    d. GUI
81. Which of the following activities are included in the design process of a software architecture _?
    a. System Development and Deployment
    b. Architectural Analysis and Testing
    c. Requirement Specifications of the system
    d. System Structuring and Modular Decomposition

For More Visit: vulmshelp.com

82. In UML based Object Oriented model of a system, the association relation between two objects is depicted by .
    a. A straight line
    b. A filled diamond sign on the whole side of the line
    c. An unfilled diamond sign on the whole side of the line
    d. Any arrowhead sign on one side of the line

83. are kind of umbrella activities that are used to smoothly and successfully perform the construction activities
    a. Designee activities
    b. Management activities (page 14)
    c. Testing activities
    d. Maintenance activities

84. Software Design discusses aspect of software development
    a. What
    b. How
    c. Who
    d. When

85. Include and extend relationship is used in UML notation of a/an .
    a. Activity Diagram
    b. Data Flow Diagram
    c. Entity Relationship Diagram
    d. Use Case Model

86. External Entity may be
    a. source of input data only
    b. source of input data or destination of results
    c. destination of results only
    d. repository of data

87. An object model of a system captures the structure of a system.
    a. Static page 93
    b. dynamic
    c. iterative
    d. Hierarchical

88. The criteria used to assess the quality of an architectural design should be based on system
    a. accessibility and reliability
    b. data and control
    c. functionality
    d. implementation details

89. In case of approach, decomposition of a problem revolves around data
    a. Object-oriented
    b. Action-oriented page 80
    c. Event-oriented

For More visit: vulmshelp.com

d. Process-oriented

For More Visit: vulmshelp.com

90. How can we implement generalization in Object Oriented programming languages?
    a. Polymorphism
    b. Encapsulation
    c. Abstraction
    d. Inheritance
91. Which of the following is NOT among one of the four layers of the Object Oriented (OO) design pyramid
    a. The subsystem layer
    b. The class and object layer
    c. The Abstract layer page 89
    d. The message layer
92. _____ is a technique in which we construct a model of an entity based upon its essential characteristics and ignore the inessential details.
    a. Inheritance
    b. Polymorphism
    c. Aggregation
    d. Abstraction page 79
93. The project manager would need _____ document to monitor and track the progress of the project.
    a. Design
    b. Project
    c. Requirement page 18
    d. Planning
94. In the case of action-oriented approach data is decomposed according to:
    a. Object requirements
    b. Functionality requirements
    c. Corresponding domain model
    d. Compatibility with object interface
95. An architectural style encompasses which of the following elements?
    a. Constraints, Set of Components and Semantic Models page 126
    b. Set of Components and Semantic Models
    c. Semantic Models and Constraints
    d. Set of Components and Constraints
96. In order to determine the role and responsibilities of the identified objects, we need to consider which of the following step(s):
        a) Who I am?
        b) What I know?
        c) Who I know?
        d) What I do?
    a. a only
    b. a and b
    c. b, cand d page 102
    d. cand d

For More Visit: vulmshelp.com

97. In sequence diagram, the boxes denote:
    a. Objects (or classes) page 106
    b. Messages, sent from one object to other
    c. Life-time of Objects
    d. Relationships

98. _____ is a system component that provides services to other components but would not normally be considered as a separate system.
    a. Method
    b. Module page 121
    c. Message
    d. Relationship

99. A tangible entity in the real life is called _____.
    a. Functions
    b. Object page 85
    c. Class
    d. Sub-Class

100. Sequence of messages can be present in:
    a) Use case diagram
    b) Sequence diagram
    c) Collaboration diagram
    a. a only
    b. b only
    c. c only
    d. b and c

101. The system model template contains which of the following elements
    a. Input
    b. Output
    c. System Out
    d. Input/Output

102. Identify the TRUE statement:
    a. Normally Object Oriented design is more maintainable than functional oriented.
    b. Software with Functional oriented design does not fulfill non functional requirements.
    c. Object Oriented design cannot implement "Separation of concerns" strategy
    d. Function Oriented design does not lead to an efficient product

103. An external entity that interacts with a system is called a(n):
    a. Use case
    b. Actor
    c. Stakeholder
    d. Association

104. By levelling a DFD (adding more levels of abstraction) we mean
    a. Splitting it into different levels
    b. Make its structure uniform

For More Visit: vulmshelp.com

c. Expanding a process into one with more sub-processes giving more detail
d. Summarizing a DFD to specify only the essentials

For More Visit: vulmshelp.com

105. Software Engineering is the combination of tools, techniques and .
   a. Testing
   b. Processes page 6
   c. Maintenance
   d. Design

106. _ _ is concerned with decomposing a system into interacting sub-systems.
   a. System Structuring
   b. Control Modeling
   c. Modular Decomposition page 121
   d. Work Breakdown Structure

107. There are _ most important characteristics of an object.
   a. Six
   b. Four
   c. Two
   d. Three

108. "System should maintain transaction log of every transaction"
The above statement is an example of
   a. Functional requirement
   b. Non-functional requirement
   c. Pseudo requirement
   d. Both Non-functional and Pseudo requirements

109. In object oriented approach, _ are the people and organizations that take part in the system under consideration.
   a. Actors
   b. Places
   c. Participants
   d. Tangible things

110. The modules that interact with each other through message passing have _ .
   a. Low coupling page 73
   b. High coupling
   c. Low cohesion
   d. High cohesion

111. Identifying Whole-part structures (Aggregations) means what are my
   a. Components page 94
   b. Attributes
   c. Methods
   d. Messages

112. In "point of sale system " the term payment represents
   a. Actor
   b. Participant

For More Visit: vulmshelp.com

c. Transaction (page 99)
d. Container

For More Visit: vulmshelp.com

113. Return values in synchronous messages are:
   a. Compulsory
   b. May not used when response is obvious (page 109)
   c. Not used at all
   d. Represented by solid lines
114. GUI stand for
   a. Generic user interface
   b. Graphic user interface
   c. Generic user interaction
   d. Graphical user interaction
115. Which of the following is not the object model principle?
   a. Abstraction
   b. Encapsulation
   c. Hierarchy or inheritance
   d. Exposure page 86
116. The system specification describes the
   a. function and behavior of a computer-based system
   b. implementation of each allocated system element
   c. algorithmic detail and data structures
   d. time required for system simulation
117. The _ provides the software engineer with a viewof the system as a whole.
   a. Process Model
   b. Architectural Model
   c. Business model
   d. Requirements Model
118. Requirement engineering focuses on _ aspect of the software development process.
   a. Both what and how
   b. What
   c. How
   d. why and how
119. An arrow in Data Flow Diagram (DFD) represents
   a. Direction of flow of data
   b. Processing of data
   c. External agent
   d. Internal agent
120. The _ relationship is kind of a generalization specialization relationship.
   a. Bit-Byte
   b. Uses
   c. Binary
   d. Extends

For More Visit: vulmshelp.com

121. In the case of _ _ in a system, module boundaries are not well defined.
   a. low cohesion
   b. high coupling
   c. low coupling
   d. high cohesion

122. _ Component of software engineering framework provides automated or semi-automated support in a software development.
   a. Processes
   b. Methods
   c. Quality Focus
   d. Tools

123. In Data Flow Diagram (DFD), one data store cannot directly copy the data from another .
   a. Agent
   b. Process
   c. Data store
   d. Flow

124. All the documents related to the software are alsoconsidered as partof the _ .
   a. Physical Document
   b. Logical Document
   c. Relational Database
   d. Software

125. Which elements of business processing engineering are the responsibilities of the software engineer?
   a. business area analysis
   b. business system design
   c. product planning
   d. information strategy planning

126. A class will be cohesive if:
   a. Class does not implement Complex interfaces
   b. Class does not have complex methods
   c. If most of the methods do not use most of the data members most of the time
   d. If most of the methods use most of the data members most of the time

127. A context diagram is used
   a. As the first step in developing a detailed DFD of a system
   b. In systems analysis of very complex systems
   c. As an aid to system design
   d. As an aid to programmers

128. _ d. Association
   a. Aggregation
   b. Abstraction
   c. Inheritance

For More visit: vulmshelp.com

_ is an extremely powerful technique for dealing with complexity.

For More Visit: vulmshelp.com

129. In multiprocessing applications, different execution threads may pass information to one another by sending _ to each other.
   a. Interrupt calls
   b. Synchronous messages
   c. Asynchronous messages
   d. System calls

130. Normally a system is more easy to modify if its modules have
   a. High coupling and high cohesion
   b. High coupling and low cohesion
   c. Low coupling and high cohesion
   d. Low coupling and Low cohesion

131. UML is a language for
   a. High-level Programming
   b. Low-level Programming
   c. Modeling and Design
   d. Creating diagrams only

132. _ is not the part of Peter Coad methodology
   a. Select actors
   b. Select participants
   c. Select places
   d. Select actions

133. Which of the following is the external quality of software product?
   a. Correctness
   b. Concision
   c. cohesion
   d. Low coupling

134. While establishing the services for an object, the goal is to keep data and action together for _ coupling and _ cohesion
   a. Lower, Higher
   b. Higher, Lower
   c. Lower, Lower
   d. Higher, Higher

135. A necessary supplement to transform or transaction mapping needed to create a complete architectural design is
   a. entity relationship diagrams
   b. the data dictionary
   c. processing narratives for each module
   d. test cases for each module

136. Construction activities are directly related to software .
   a. Management
   b. Planning
   c. Quality Assurance
   d. Development

For More Visit: vulmshelp.com

137. OOD results in a design that achieves a number of different levels of _ .
a. Operation
b. Event page 89
c. Modularity
d. Process

138. At which stage of software development loop, results are delivered?
a. Problem definition
b. Solution integration
c. Technical development
d. Status quo

139. _ Requirements are often called product features.
a. Functional
b. Non-functional
c. Developer
d. User

140. Strong cohesion implies that
a. All parts of component have a class logical relationship with each other
b. All part of component do not have a close logical relationship with each other
c. Component is dynamic in nature
d. Component is static in nature

141. _ Diagram does not capture control flow information; it just shows the flow of the data in a system.
a. Sequence
b. Data Flow
c. Activity
d. Class

142. A life line represents the object's life during the interaction in a sequence diagram while its notation is depicted by _
a. Solid Lines
b. Dotted Lines
c. Full Arrow
d. Curved Lines

143. An attribute that varies over time, e.g. price of an item, should be replaced by a/an _ _ with an effective date and value
a. Additional Class
b. Additional Method/Function
c. Interface
d. Structure

144. Which of the given component of software engineering framework demands rational and Timely development of a software?
a. Tools
b. Methods
c. Quality Focus

For More Visit: vulmshelp.com

d. Processes

For More Visit: vulmshelp.com

145. A context diagram
   a. describes detailed design of a system
   b. is a DFD which gives an overview of the system
   c. is a detailed description of a system
   d. is not used in drawing a detailed DFD

146. Which one is not a part of Software Development phase?
   a. Construction
   b. Scope
   c. Project Vision
   d. Definition

My Point of View

147. Software architecture must address _ requirement of a software system
   a. Functional
   b. Non Functional
   c. User interface Requirements
   d. Both Functional and Non Functional

148. If Cat is derived from Mammal Class, and Mammal is derived from Animal Class, then:
   a. Cat will inherit Animal's functions and data
   b. Cat will not inherit Animal's functions and data
   c. Cat will not be able to access any class
   d. Cat is allowed to access only the Mammal's Class

149. Which of the following is not supported by a maintainable design?
   a. Change
   b. debugging
   c. Adding new features
   d. Higher maintenance cost

150. The condition that must be met before the use case can be invoked, is called:
   a. Pre-Condition
   b. Post-Condition
   c. Pre-Assertion
   d. Post-Assertion

For More visit: vulmshelp.com